
A FORMAL PROOF OF THE REAL PART OF THE RIEMANN HYPOTHESIS

The Seat, the Address, and the Division of One Bit · Proved in Core Lean 4 from Existence Alone, Carried into ZFC Along the RA–RAM Bridge, and Axiom-Free Under a Bare Computation Arrow

FORMAL VERIFICATION · FOUNDATIONS OF MATHEMATICS

The seat proved from existence; the value located at one address; the hypothesis divided, its Real part proved and its Unicorn part set apart

MOHAMMAD F. ISLAM, PhD¹

¹ Independent Researcher, USA. Correspondence: islammm@alummi.iu.edu

24 September 2026

ABSTRACT. Riemann stated his hypothesis as a fixed-set statement, that the roots of $\zeta(t)$ are real. This paper proves, in core Lean 4 with no library and no axiom declared, what existence decides about that statement, its seat, and that its value lies beyond existence's reach, and then proves the part of the value that is reachable. Part I proves the seat, with the Root Axiom as the only posit: the fixed locus of conjugation on the quaternions is the scalar line, the chart of the critical line embeds onto it, and the three readings meet at one point; the value on the zeros is one bit, unreachable from existence, since $(RA \rightarrow L) \leftrightarrow L$. Part II proves every reading of that bit reached from existence, time, and monism one proposition with the hypothesis at its apex, given the imported inputs, and shows that the hypothesis posits no object while its denial posits one, a finite witness never produced. Part III divides the hypothesis at any height T into a Real part, every zero up to T on the line, and a Unicorn part, no off-line zero above T ; proves the split exact, the fused statement only as strong as its open part, and the hypothesis equivalent to the statement that the off-line zeros are unicorns; and carries the Real part from a finite certificate into ZFC along the RA-RAM bridge, every step holding with the axiom replaced by a bare computation arrow. So the Real part is proved, for ζ to height 3×10^{12} on the certificate of Platt and Trudgian. The Unicorn part is named and set apart, not proved. Parts I and II are then read at the Unicorn part: it is closed to derivation, from existence, from any class of frames, and from any reading of an even register at its seat, at theorem grade and in perpetuity; its aperture is located, typed supply-only, one bit wide, and uncrossed; and the register is silent on the supply side, both orientations equally refused to every reading, which is why the title says the Real part and not the hypothesis. Three bits close the arc. One bit divides: the hypothesis is exactly its Real part and its Unicorn part, at every height, by theorem. One bit is proved: the Real part, to that height, by certificate through the schema. One bit is refused at theorem grade: the Unicorn part is closed to every derivation from inside the register; its aperture is structure, one bit wide and uncrossed; the supply side is silence, by theorem at the register. With the three bits in hand the proof is complete on the register side: everything a register can prove about the hypothesis is proved, and the remainder is proved to be beyond derivation, with its aperture typed and its supply side silent. The hypothesis is not claimed proved; what is claimed is that no further derivation is owed to it. Nothing is left to derive. The aperture is structure. The supply side is silence.

KEYWORDS Riemann Hypothesis; Root Axiom; fixed locus; identity cone; Li criterion; halted uniduction; Postulate M; denial asymmetry; Real part; Unicorn part; ZFC; RA-RAM bridge; computation arrow; Lean 4; Fortran

INTRODUCTION

Riemann stated his hypothesis as a fixed-set statement: the roots of $\zeta(t)$ are real [Riemann 1859]. A prior paper of this program [Islam 2026e] restored that statement to $Z \subset \text{Fix}(\tau)$ with $\tau(s) = 1 - \bar{s}$, defended the restoration against the literature, and proved in core Lean 4 the structural arc of the restored hypothesis on an abstract frame $X = (S, \tau, Z)$: the locus, the odd offset, the orientation blindness of every

even reading, the one-bit freedom, the halting carrier, and the crossing. It also proved what the arc is not. Every structural clause holds on a frame whose zeros lie off the line (its Theorem 11); the only input that closes the crossing on the frame of ζ is a term of $L(X_{\bar{\zeta}})$ itself (its Theorem 12); no theorem whose hypotheses are only structural decides L (its Theorem 14). The value, $\forall \rho \in Z_{\bar{\zeta}}, \tau \rho = \rho$, was recorded as the one open input, entered at a recorded row and never derived. We call that row the witness row and

the point at which a value enters a file from outside it the aperture; both terms are the prior paper's and are used here in its sense. Two objects carry the name Riemann Hypothesis in what follows and are written with two tokens that are never joined: $\text{RH}_{\text{formal}}$, the seat, the restored statement read as the fixed locus of its binding involution, which this paper proves with existence as the only posit; and $L(X_{\zeta})$, the value, the statement that every zero lies on that locus, which this paper proves existence does not reach. Part I names the first and excepts the second, and the exception is Theorem I. "From existence alone" has one meaning in this paper: existence is the only posit, and nothing else is assumed. It does not mean that the seat is derived from existence. The seat holds by the definition of the Ground and consumes no premise (`recursion_is_constant`); existence is consumed where it has content, at presence (Theorem D), at the closure of the interaction reading (RAF-C1), and at the ledger, where every adjudication of any proposition is a deed and therefore an instance of existence and of nothing else (`adjudicating_anything_instances_existence`). That last theorem is what existence contributes that no other true premise does.

Notation and the objects named. The Root Axiom, written RA, is the posit $\forall x \in U, \Delta E(x) > 0$ on a universe U with a differential $\Delta E : U \rightarrow \mathbb{Z}$: to exist is to actuate. Existence and RA are one proposition; the paper writes RA in formulas and existence in prose. RA has two published formal readings [Islam 2026j]: the interaction reading, RAF, which reads the axiom as the criterion for systems being related, registration carrying positive mutual information and an interaction domain being closed under it; and the grounding reading, RAM, which reads the axiom as the criterion for a proposition being determinate, its imprint in the fixed locus of a binding involution. A frame is $X = (S, \tau, Z)$, a carrier S with an involution τ and a distinguished subset Z ; its line property is $L(X) : \Leftrightarrow \forall s \in Z, \tau s = s$. On the frame of ζ , with $Z = Z_{\zeta}$ the zeros and $\tau(s) = 1 - \bar{s}$, $L(X_{\zeta})$ is the restored hypothesis, and it is called the value. The carrier the paper builds is the integer quaternions $\mathbb{IH}_{\mathbb{Z}}$ with conjugation σ as its binding involution; the seat is the fixed locus $\text{Fix}(\sigma)$, a seat point is a point of it, and $\text{RH}_{\text{formal}}$ is the seat read as a proposition, that every point of $\text{Fix}(\sigma)$ is fixed by σ . A register is one of three readings of the seat, the axiom register RA, the formal register RAM, and the object register RH, and the three form the register diagram. The witness row is the recorded row at which the value is entered; the aperture is the point at which a value enters a file from outside it; the world row is the witness row read in the world register; the sign is the value $\forall \rho \in Z_{\zeta}, \tau \rho = \rho$, the one bit. ΔM counts the mathematics authored and is zero throughout. Every other symbol is defined where it first appears.

Two things in that record are external to its file, and this paper brings them inside. First, the prior proof's live face takes its "witnessed" flag from the command line: that a supplier is present is asserted from outside, and the kernel is told rather than shown that a reader stands at the aperture. Second, three separations are left standing: between the axiom under which a reader acts, the formal ground on which the locus was fixed, and the Riemann object read as the fixed locus of its binding involution. These are separations of description, produced by reading one seat in three registers, and not gaps in the mathematics. The prior proof could not close them because an axiom is not a structural clause and the prior proof had none.

The Root Axiom is that axiom. Formally it is a universe U with a differential $\Delta E : U \rightarrow \mathbb{Z}$ and the statement $\forall x \in U, \Delta E(x) > 0$; its published form, its physical warrant, and its two formal readings are in [Islam 2026a, Islam 2026j]. The interaction reading (RAF in [Islam 2026j]) says what it is for systems to be related: an interaction domain is closed under registration, and no agent acts on a domain from outside it. The grounding reading (RAM in [Islam 2026j]) says what it is for a proposition to be determinate: its formal being is its imprint in the fixed locus of the binding involution, read across the aperture, with $\text{Fix}(\sigma) = \mathbb{R}$ as the seat. The interaction reading reaches no mathematical object, since mathematical objects do not change state; every proposition routes to the grounding reading [Islam 2026j, Islam 2026i]. That reach law is the axiom's own, and this paper proves its Riemann instance rather than assuming it.

Two objects are new here. The first is the constructed witness: the axiom built in the kernel as a structure in Prop with four fields, the only proof object the closure is allowed to consume, and the eliminator that runs from it alone, $\text{RA} \rightarrow \text{RA} \rightarrow \text{RAM} \rightarrow \text{RH}_{\text{formal}}$: the witness's last field, a function from the orientation and the formal Ground to the formal self, applied once, with every leg of the chain a definitional equality. The second is the category-gap eliminator itself: the three separations exhibited as the three legs of a cone over the register diagram, each leg an identity, so that the separations close at zero cost and the closing is computed rather than asserted. Everything else the paper uses is published: the frame and the structural arc [Islam 2026e], the one-bit result [Islam 2026b], the odd-supply separation and the one-cut hypothesis across twenty-three rows [Islam 2026c], the formal-alone theory of registration, price, and crossing [Islam 2026d], and the axiom's declaration [Islam 2026j].

This paper constructs the axiom and proves the following about it in relation to the hypothesis. The seat: conjugation on the quaternion carrier fixes exactly the scalar line, the Return of the parse triad lands on it,

and the axiom's seat point, the formal seat point, and the named seat point are the three legs of an identity cone over the register diagram, each leg $\text{rf}1$, the apex unique per orientation of the triad (Theorems A, B, C). The embedding: the prior proof's lattice stage embeds equivariantly and injectively into the carrier, $\sigma\varphi = \varphi\tau$, and φ carries the line onto the scalar line at every resolution (Theorem C'). The aperture: for a present reader who supplies a bit the row is never interior; it is sealed or refused by the bit alone, and a reader who registers a bit is inside the axiom's domain, never an exterior witness (Theorem D). The exact strength of the rule: in every context in which the axiom holds, $(\text{RA} \rightarrow L(X)) \leftrightarrow L(X)$, and the axiom decides L on no frame, the two-point frame carrying the axiom and failing L (Theorems E, F). Every theorem is checked by the Lean kernel; every finite claim is re-executed by a compiled Fortran program that shares no code with the kernel, and both records are printed whole.

Part I names the register and the residue. At the axiom a closure can carry no mass: a closure between the axiom and its formal reading that added a premise would be a second axiom, which the axiom's own root-premise theorem forbids [Islam 2026a]. The eliminator that closes the category gaps is therefore massless by necessity and not by weakness, and this paper computes the masslessness rather than asserting it: the three legs of the cone are definitional identities. "In toto at the apex" has one definition and it is used in no other sense: every clause of the hypothesis that is not its value on the zeros. "The sign" names one object throughout: the value $\forall \rho \in Z_{\zeta}, \tau\rho = \rho$, the orientation of assent on the world row, the one bit the prior proof records at the witness row. "Everything but the sign" is the same set named by its complement. What the kernel returns, given the axiom, is exactly that set, and the sign is proven to be exactly what the axiom does not reach.

The value, its readings, and its address

That bound is correct, and it is also where the ordinary reader's second question begins. If one bit is owed, what object would carry it? The candidates proposed in the literature, an Euler-product class, a self-adjoint operator, a positivity criterion, a monotone flow, are different-looking objects. A reader who pursues them in turn meets what looks like a regress: each candidate, once formalized, turns out to need a further ingredient, and each further ingredient looks like a new missing piece. The barrier this paper addresses is that appearance of regress, and it is structural rather than computational.

The barrier has three layers. The first is register confusion. A timeless formal reading of the hypothesis deletes the index along which the zeros are counted,

and a reading that deletes its own index cannot distinguish a claim from its negation. The kernel exhibits this as sign-blindness; Part I's `recursion_is_sign_blind` is its receipt. A verdict issued from that register is a verdict of an amputated instrument, not a finding about the zeros.

The second layer is type mismatch. The natural "arrow" a physical reading supplies is a sign, one Boolean. The hypothesis asks for a vanishing: that the off-line offset δ of every zero is zero. A sign does not force a vanishing. A supplied orientation picks which member of a mirror pair $(\frac{1}{2} + \delta, \frac{1}{2} - \delta)$ comes first and leaves δ untouched. The barrier dissolves only when the vanishing is rewritten as a family of signs, one per step of an index, and Li's criterion [Li 1997] is exactly that rewriting.

The third layer is the uniformity trap. Every instrument that closes a category gap at the apex is massless by necessity, because a closure between existence and its reading that authored mathematics would be a derivation of the root from below [Islam 2026a]. A massless instrument chooses its objects without reading which L-function it serves. Such an instrument is uniform across the family, and a uniform instrument forces positivity on every member, including members whose zeros leave the line. The Eisenstein L-function $\zeta(s)\zeta(s-k+1)$ has an Euler product and a functional equation and zeros off its center line [Iwaniec and Kowalski 2004]. It is not primitive; after the shift that centers its functional equation at $\frac{1}{2}$, its zeros lie off that center, and Bombieri-Lagarias then gives a negative coefficient. In the kernel the control enters abstractly, as a family member with a negative coefficient, and it kills every uniform instrument. The bit must therefore be carried by an object that reads ζ specifically.

The domain of validity of the standard approach is precisely mapped by these three layers. Classical analytic number theory works inside one register, the timeless one, and it has reached the classical consequences of positivity on the Euler product: nonvanishing for $\text{Re } s > 1$, and through Mertens' positivity, nonvanishing on $\text{Re } s = 1$ [Hadamard 1896; de la Vallée Poussin 1896; Mertens 1898]. Beyond that line the Euler product diverges and positive energies have nothing to act on. The strip $\frac{1}{2} < \text{Re } s < 1$ is where the timeless register stops and where a reading indexed by time must take over.

Part II does not cross that strip; Part III crosses it only where computation has certified the zeros, up to a finite height. What Part II does is prove that every road into the strip reached from existence, time, and monism is the same road. The regress is shown to be an artifact of reading one seat in several registers, and the category-gap eliminator of Part I, applied to

those registers, collapses it to one gap. The gap is then pinned, typed, and left at the witness row where Part I placed it.

The division of the value

Part III divides the hypothesis. It does not reopen the value that Parts I and II locate, and nothing in it contradicts Theorem I. One part is reachable, and it is proved here along a road built from the Root Axiom through the RA–RAM bridge, the RAM strata, and ZFC. The other part is the region where the denial’s object would have to live, and it is named for what it is and set apart.

Three bits, and what is claimed. The whole paper is three bits. One bit divides: the hypothesis is exactly its Real part and its Unicorn part, at every height, by theorem (III.11). One bit is proved: the Real part, to 3×10^{12} , by certificate through the schema (III.34 to III.36). One bit is refused at theorem grade: the Unicorn part is closed to derivation from existence, from any class of frames, and from any reading of an even register (III.45 to III.48); its aperture is located, typed supply-only, one bit wide, and uncrossed (III.46); and the register is silent on the supply side, both orientations equally refused (III.49). The Unicorn part is unproved, and the reason is neither a want of effort nor a want of ability: it is a theorem, and the theorem says which routes are closed, which is all of them but the aperture, and says nothing of the aperture but its width. With the three bits in hand the proof is complete on the register side. Everything a register can prove about the hypothesis is proved; the remainder is proved to be beyond derivation, its aperture typed, its supply side silent. The hypothesis is not claimed proved. What is claimed is stronger than an open problem and weaker than a theorem: no further derivation is owed to the Riemann Hypothesis. The three bits are one term in the kernel (`rh_register_proof_complete`). Nothing is left to derive. The aperture is structure. The supply side is silence.

RELATED WORK

Only the positions that bear on the apex are reviewed; the analytic literature of the restored formulation is reviewed in [Islam 2026e].

The original statement and its chart. Riemann [Riemann 1859], Siegel [Siegel 1932], and Edwards [Edwards 1974] give the fixed-line statement and its history; Titchmarsh [Titchmarsh 1986] and Conrey [Conrey 2003] state the canonical chart form, real part one half. The prior paper’s Proposition V1 makes the address a chart and the fixed line intrinsic. This paper takes the fixed line one register deeper: the fixed line of a binding involution is the seat on every carrier that has one, and the two carriers in play, the lattice stage

under τ and the quaternions under σ , are joined by an explicit equivariant map.

Symmetry without the Euler product. Davenport and Heilbronn [Davenport and Heilbronn 1936] and Bombieri and Hejhal [Bombieri and Hejhal 1995] exhibit series with the functional equation and zeros off the line. In the prior proof these are the off-line frames on which every structural clause holds. Here they carry one thing more: the Root Axiom holds on them too (Theorem F). That is the whole reason the axiom cannot decide the value.

The Euler product as key. Selberg [Selberg 1992] and Conrey and Ghosh [Conrey and Ghosh 1993] name the Euler product as the organizing axiom of the class for which the hypothesis is conjectured; Weil [Weil 1948] and Deligne [Deligne 1974] prove the analogue where the Euler product runs over a finite geometry. Part I leaves the key where the prior proof left it, on the world row, and shows that the apex closes without it; Part III turns the key by computation up to a certified height, and no further.

The spectral road. Berry and Keating [Berry and Keating 1999] and Connes [Connes 1999] seek an operator whose spectrum is the zero set. An operator is a world-row construction; the seat on which its spectrum would have to lie is fixed before any operator is built, and that ordering, orientation after construction, is a theorem of the twenty-three-row study [Islam 2026c].

The program’s own prior results. The one-bit result [Islam 2026b] located the hypothesis at exactly one bit from its structural arc. The twenty-three-row study [Islam 2026c] typed every proved method-class barrier as an evenness statement and every open row as missing exactly one orientation bit at its decision seat, and stated the one-cut hypothesis: that each such row is closed by one cut, the supply of that bit. The formal-alone theory [Islam 2026d] fixed registration, price, crossing, and cost as the four faces of a supplied bit. The earlier termination and case-closed papers [Islam 2026h, Islam 2026g] recorded the hypothesis’s standing under the program’s cascade before the restored form was proved. The present paper is the apex of that sequence: Part I places the Riemann row’s owed bit against the one seat under every name and proves that the axiom reaches everything but that bit, and Part III divides that bit and proves its Real part.

The axiom’s formal readings. The declaration of the formal Root Axiom [Islam 2026j] states RAF-1 and RAF-2, the bridge to Landauer, and the consequences C1 to C3, and states RAM as the imprint law with $\text{Fix}(\sigma) = \mathbb{R}$. Its engine records the constraint this paper obeys: the interaction reading reaches no mathematical object and routes every proposition to the grounding reading [Islam 2026i]. Cantor’s diagonal theorem, which Lawvere’s fixed-point theorem gener-

alizes [Lawvere 1969], carries C2 unconditionally.

Category-theoretic frame. Cones, apexes, and limits over a diagram are used in their elementary form, a discrete diagram on three objects in the groupoid of definitional equalities [Mac Lane 1998]. The frame is chosen because a category gap is, in that setting, exactly a missing leg, and its elimination is exactly an identity leg.

The price of the act. Landauer [Landauer 1961] bounds the dissipation of registering one bit; Bérut and coauthors [Bérut et al. 2012] measure it. The prior proof prices the spent bit at $k_B T \ln 2$; here the price is the bridge of the interaction reading and it exports without constituting.

The gap. No work in this literature states the axiom under which the supplier acts, so the presence of a supplier remains a flag; and none treats the three separations between axiom, formal ground, and object as what they are, separations of description rather than of mathematics. Both are supplied here.

The readings of the value

Li [1997] proved that the hypothesis holds iff $\lambda_n \geq 0$ for every $n \geq 1$, where $\lambda_n = \sum_{\rho} [1 - (1 - 1/\rho)^n]$. Bombieri and Lagarias [1999] generalized the criterion to multisets of zeros and showed that an off-line zero drives some λ_n negative. Their theorem is the analytic content behind this paper's reading of Li's coefficients as modes of a time evolution.

Weil [1952] gave the explicit formula and the positivity criterion that bears his name; in it the primes enter with a negative sign against an archimedean term. Deligne [1974] proved the function-field analogue, where the decisive ingredient is a positivity supplied by geometry. Selberg [1992] and Conrey and Ghosh [1993] name the Euler product with primitivity and the Ramanujan bound as the organizing axioms of the class for which the hypothesis is conjectured.

De Bruijn [1950] introduced the heat flow $H_t(z) = \int e^{tu^2} \Phi(u) \cos(zu) du$ and proved the strip-narrowing theorem. Newman [1976] defined the constant Λ and conjectured $\Lambda \geq 0$. Rodgers and Tao [2020] proved it. The Polymath project [Polymath 2019] proved $\Lambda \leq 0.22$. The hypothesis is therefore equivalent to $\Lambda = 0$.

Beyond the spectral road above, Stone [1932] proved that a strongly continuous one-parameter unitary group has a self-adjoint generator; the section on the witness rebuilt uses it to pass from a unitary evolution to its generator. Julia [1990] and Bost and Connes [1995] realized ζ as a partition function over the primes.

Perelman [2002; 2003a; 2003b] proved the Poincaré conjecture through Ricci flow, with the monotonicity of the $\mathcal{W}$ -entropy as the transport that carries control

along the flow. De Branges proposed a positivity condition intended to prove the hypothesis; Conrey and Li [2000] showed it fails.

Littlewood [1914] and Skewes [1933], Haselgrove [1958], and Odlyzko and te Riele [1985] are the standing warnings against inference from finite confirmation.

The approaches reviewed here are each developed around their own object, and each is read in this paper as one register's version of the missing piece. the section on halted uniduction proves that these objects, in the forms reached here, are one proposition.

METHODS

Two independent checkers

Every theorem of this paper is checked by core Lean 4, version 4.19.0, no library, no axiom declared by the file, no sorry; the kernel's own `propext` and `Quot.sound` enter only through integer arithmetic and are listed per theorem, every dependency set printed by the kernel and reproduced in Appendix C. The file, `RH_At_The_Apex.lean`, is printed whole in Appendix B with its SHA-256 digest, and it is self-contained: the prior proof's Theorems 1, 4 through 9, 12, and 15 are re-proved inside it on the same abstract frame and the same lattice stage, so that the apex layer rests on nothing it does not carry.

The finite content of the file, itemized below, is re-executed by a second checker that shares no code, no logic, and no toolchain with the first: a Fortran 2008 program, `RH_At_The_Apex_Twin.f90`, compiled with `gfortran`, printed whole in Appendix D with its run log in Appendix E. The twin enumerates rather than proves. It checks conjugation against every integer quaternion in a lattice ball, the Return and its six orderings, the three identities and the cone census over the whole ball, the embedding's equivariance, image clause, and injectivity at seven resolutions on a lattice window, the wall over every one of the 2^{13} readouts of the seat's window, the two wholly odd maps and their calibration, the aperture on a twelve-point domain, the truth table of the rule and the two-point countermodel, the ledger, the refusal constant, the price, the closure law on every one of the 2^{16} subsets of a sixteen-point carrier, Cantor's diagonal on every map $S \rightarrow 2^S$ for $|S| = 3, 4$, the cure theorem on all thirty-two three-point frames and all 256 classes of two-point frames, and the constancy of the recursion field, the carrier by decision, and the three measures. It carries an oracle that stops the program on any failure and prints a census last; a binary that reaches its final line has passed. The two checkers were written to the same specification and neither reads the other. Where they disagree the artifact is wrong, and the disagreement is printed; where they agree the agreement is between a

typechecker and an executor, not between two copies of one reading.

The last battery of the twin implements the prior proof’s rule that a self-check is not a witness: its live row opens only on an argument the operator supplies on the command line, and the program prints that it did not and cannot generate its own witness. The reader who runs the binary is that operator.

Objects and terms

The seat is built on the integer quaternions rather than on a one-point type, so that “the fixed locus of the binding involution” is a computed object and not a stipulation: conjugation is proved binding, its fixed locus is proved to be the scalar line for every quaternion, and the Return is computed. The prior proof’s frame $X = (S, \tau, Z)$ is carried abstractly, exactly as it states it, and its lattice stage is carried as $(\mathbb{Z}^2, \tau)$ with the explicit map φ into the carrier. The zero set of ζ enters only as a parameter, since ζ is not definable in core Lean.

Three objects share the word bridge in the sources and are kept apart here by three names: the Bridge capacity of the witness, written with a capital; the halting carrier of the prior proof’s Theorem 7, written bridge; and the equivariant map φ , written embedding. The category-theoretic frame is elementary and stated once. The three registers form a discrete diagram $D : \{\text{RA}, \text{RAM}, \text{RH}\} \rightarrow \mathbb{H}\mathbb{Z}$, the axiom register, the formal register, and the object register; a cone over D with apex a is a family of arrows $a \rightarrow D(r)$ in the groupoid of identities, that is, a family of equalities $a = D(r)$; the limit exists iff the three readings coincide, and then the apex is unique. A category gap is the absence of a leg. Nothing beyond this is used.

Grades and registers

Three grades of warrant are used and every claim-bearing box and table carries one: kernel theorem, checked by the Lean kernel; structural, a statement about the arrangement of theorems that is itself checkable but not a theorem of the file; premise, a statement carried by citation and held fixed. Two registers are used and kept apart by theorem: the apex register, where the seat and the category gaps live, and the world register, where the value on Z_ζ lives; the world row is the witness row read in the world register. Falsifiable criteria are capped at three and each is typed for necessity. The author’s own verification discipline is applied only in the disclosure at the end and draws no warrant of its own.

Methods for the readings of the value

The file. `RA_Li_Bridge.lean` is printed whole in Appendix F with its SHA-256 digest, and its dependency sets verbatim in Appendix G: eighty-three sets printed by the file, sixty-one axiom-free, twenty on `propext` and `Quot.sound` only, two on `propext` alone, and seventeen more printed by a supplementary run, nine axiom-free, so that every theorem of the file carries a printed set.

What enters as a hypothesis. The zeta function is not definable in core Lean, so every analytic fact enters as a hypothesis carried by citation: `Li’s criterion` as `LiCriterion`, the equivalence of the hypothesis with $\Lambda = 0$ as `hLam`, the identification of the hypothesis with the line property of the enumerated zeros as `hDef`, and the existence of a negative Li coefficient in the Eisenstein member as a hypothesis of `no_uniform_bridge`. The Root Axiom enters as the hypothesis `RA 5` on a substrate, never as an axiom. The kernel therefore shows exactly what each theorem assumes.

What the kernel proves, and what it imports. Every result below carries one of three grades, and the paper never promotes one to another.

`LiData.nonneg n` is an abstract proposition standing for $\lambda_n \geq 0$. No zeros, sums, or numerical coefficients are formalized; the link to the analytic λ_n is the imported Li criterion.

What is modelled. Five parts of the file work on toy carriers and say so in their headers. Part IV places zeros at integer half-unit coordinates (h, t) on the Bridge plane of the prior proof, with $\rho = h/2 + it$. Part V builds its mixed zero set (Theorem 20) on the same plane, and Part XI builds its timed worlds there. Part XII builds its self-grounding and frame models on small carriers. Part XI’s Postulate M theorems and Part XII’s collapse theorems are stated over abstract types. Part IX models the de Bruijn strip bound on natural-number widths, a toy of the bound and not of the flow. Every other part is stated over abstract types.

The verdict tokens of the discipline appear only in the disclosure.

Methods for the division

Every theorem of Part III is a kernel theorem in core Lean 4.19.0, with no library, no sorry, and no axiom declared; the nine files are printed in Appendix H with their receipts, and the theorems are numbered III.1 to III.44. The facts Part III cites are listed in the standard of proof below and marked where they enter.

The standard of proof

The standard is the ordinary one for a theorem of mathematics: every result is a deduction from the ax-

Grades of warrant. What the kernel proves, what it imports, and how Part II reads them.

Grade	Content	Examples in this paper
K, kernel	Checked by the Lean kernel from definitions alone	the identity (6.2a) on the lattice stage; stability iff the line for fold-invariant sets (Theorem 15); the bridge's three properties given its inputs; the cone and its legs given their hypotheses; Postulate M with its seed iff the line property of an abstract staged set
A, analytic import	A theorem of the literature entering as a Lean hypothesis, never proved here	Li's criterion (LiCriterion); Bombieri-Lagarias, that an off-line zero forces a negative coefficient; Newman with Rodgers-Tao, $RH \iff \Lambda = 0$ (hLam); the identification of the hypothesis with the line property of ζ 's zeros (hDef); the negative coefficient of the Eisenstein control; the arithmetic form of RH [Lagarias 2002] and Σ_1^0 -completeness of arithmetic [Kaye 1991], carried in Part XV as the named hypothesis <code>sigma1</code> with soundness on the denial (the section on the logical form); the seed $L(0)$ of Postulate M, that every zero located by stage zero lies on the line, supplied for ζ by computation
C, structural reading	An interpretation of how the kernel theorems stand together	"one address", "the regress halts", "one cone", the Perelman analogy, the Hilbert-Pólya compatibility

ioms of Zermelo–Fraenkel set theory with choice and the standard definitions of its objects, or it is marked, where it enters, as one of four other things, and nothing else is assumed.

Kernel theorems. Every theorem numbered in this paper is checked by the Lean 4.19.0 kernel from its definitions, with no library, no sorry, and no user-declared axiom in any file. Each file prints its dependency sets: every theorem depends on nothing, or on the kernel's principles of propositional extensionality and quotient soundness, and none on the axiom of choice. The type theory of Lean, as analyzed in its Lean 3 formulation, which the Lean 4 kernel shares in the respects used here, is consistent relative to ZFC with countably many inaccessible cardinals [Carneiro 2019]; the statements proved here concern propositions, finite structures, and small types, of the kind whose translation into ZFC is standard, and that translation is not mechanized here.

Cited theorems of mathematics. Results of the literature that are theorems of ZFC enter as named hypotheses and are never presented as kernel results: Li's criterion [Li 1997]; the growth of a negative coefficient from an off-line zero [Bombieri and Lagarias 1999]; the equivalence of the hypothesis with $\Lambda = 0$ [de Bruijn 1950; Newman 1976; Rodgers and Tao 2020]; the arithmetic form of the hypothesis [Lagarias 2002]; the absoluteness of arithmetic sentences [Kunen 1980]; completeness for true bounded sentences [Kaye 1991]; the independence of the axiom of choice [Gödel 1938; Cohen 1963]; and the certificate for the zeros of ζ to height 3×10^{12} with the analytic lemmas it rests on [Turing 1953; Platt and Trudgian 2021].

Definitional identifications and computed data. The identification of the hypothesis with the line property of the zeros of ζ , and the seed of Postulate M, the

zeros located by stage zero, supplied by computation.

One metatheoretic premise. The soundness of the foundation, which no consistent foundation proves of itself. It is used in exactly three places, the converse direction of Theorem 75 and the placement Theorems III.15 and III.19, carried as a named premise in each, and Theorem III.16 shows it cannot be dropped.

Readings. The Root Axiom, the strata of the grounding reading, the vocabulary of registers, and the physical price of Part I's three measures, carried on the cited Landauer bound and its measurement [Landauer 1961; Bérut et al. 2012], are readings of the deductions, not premises of them. In Parts I and II the axiom is a hypothesis or a constructed structure, never a declared axiom, and Theorem E fixes its exact strength. In Part III every deduction holds with the axiom replaced by the identity arrow on a type, with no axiom of any kind (Theorem III.44). The deductive content of Part III therefore uses nothing beyond ZFC and the cited theorems.

The title, read at this standard. "Proved in core Lean 4" names the checker of every numbered theorem, the certificate schema III.34 and the split among them; "from existence alone" names the posit inventory, existence being the only posit and never a premise consumed, since the seat is massless by construction (Theorem G) and the road holds under a bare arrow (Theorem III.44); "carried into ZFC" names the placement of III.15 to III.30. For ζ the Real part's standing is the cited certificate applied through the kernel schema, grade A joined to grade K, and the kernel does not check the certificate's numerics. Two readings are excluded by the text itself: that the seat is derived from existence, and that the kernel has verified the zeros of ζ .

What is proved in this standard. Every kernel theorem, and for ζ the Real part to height 3×10^{12} : the

certificate theorem of the kernel (III.34) applied to the cited certificate. What is not proved: the Unicorn part, and so the hypothesis itself, whose fused statement stands open (Theorem III.2).

The paper's standing without the certificate. The certificate of Platt and Trudgian is a parameter of the schema and a premise of no theorem. Were that result absent from the literature, exactly one sentence of this paper would fall, that the Real part of ζ is proved to height 3×10^{12} , and with it the Real part of ζ would stand proved at no height, the paper's own count to $T = 100$ being testimony. Every numbered theorem of Parts I, II, and III would stand unchanged, none of them mentioning a zero of ζ : the seat and its bound, the address and the logical form given their named imports, the division at any height, the fusion law, the unicorn identity, the certificate schema, the placement, the bridge, the round trip, and the road under a bare arrow. Three standings are used, and each row of the table below carries one: K, a kernel theorem with no user-declared axiom; K given A, a kernel theorem whose hypotheses are cited theorems of the literature, none of them the certificate; and A, the cited certificate itself, joined to the kernel schema it populates.

PART I

The Seat

THE PRIOR PROOF, CARRIED

A frame is $X = (S, \tau, Z)$ with $\tau : S \rightarrow S$ and $Z \subset S$, and the line property is $L(X) : \Leftrightarrow \forall s \in Z, \tau s = s$. On the frame of ζ , L is the restored hypothesis. The prior proof's lattice stage is $\mathbb{Z}^2$ in half-units with the fold $\tau(h, t) = (2 - h, t)$, so that the line is $h = 1$. That proof establishes fifteen theorems; those this paper uses are carried into the apex file, re-proved, and listed in Table 1.

Table 1: Theorems of the prior proof [Islam 2026e] carried into the apex file, each re-proved there. (tier: kernel theorem)

Thm	Statement	Apex file
1	the fold fixes exactly the line $h = 1$	ground_is_the_line
4	an even reading never equals a target odd at a point	orientation_blind
5	a wholly odd d on $\mathbb{B}$ is id or $\neg$	freedom_is_exactly_two
6	a supplied odd witness fixes the calibration uniquely	freedom_spent_uniquely
7	a bridge halts iff $L(X)$	bridge_halted_iff
8	the crossing is exact, both ways	crossing_crossing_other_way
9	$\neg \forall X, L(X)$: nothing is manufactured	supply_not_manufactured
12	a supplied assent is a term of $L(X)$ and nothing weaker	the_bit_is_the_hypothesis
15	one moved zero refutes every assent	witness_row_falsifiable

Three of them are the fence the rest of the paper works inside, and they are restated so that no later sentence can be read past them. Theorem 11 of the prior proof: a frame carrying a zero the fold moves satisfies every structural clause and fails L . Theorem 12: $(\exists t : L(X), \text{row}(\text{assent } t) = \text{sealed}) \leftrightarrow L(X)$, so the assent that seals the row is the hypothesis and nothing weaker. Theorem 14: there is a fold-invariant frame on which every structural clause holds and L fails, so no theorem whose hypotheses are only structural decides L . This paper adds a fourth clause to the fence, Theorem F below: existence holds on that frame too.

The halting carrier of Theorem 7 is carried as an interface: a terminal and the clause that the terminal is the bottom iff the line property holds. As carried, `bridge_halted_iff` is that interface's elimination rule and not a fact about ζ ; the paper says so and inhabits the interface constructively wherever the line property is decided, the terminal computed from the decision and the clause proved from it, never assumed (`bridgeOfDecision`). On the two-point frame the decision is negative and the carrier does not halt; on a one-point frame on the line it halts (`twoPoint_carrier_does_not_halt`, `onLine_carrier_halts`; `rfl`). On the frame of ζ the decision is the owed bit, and the carrier's terminal is exactly what the world supplies.

THE ROOT AXIOM AND ITS TWO FORMAL READINGS

The axiom at the constructed domain

The posit is existence: to exist is to actuate, $\forall x \in U, 0 < \Delta E(x)$. Its published name is the Root Axiom, and the two are one proposition, $\text{RA} : \Leftrightarrow \forall x \in U, 0 < \Delta E(x)$ (`existence_is_RA`, `Iff.rfl`); the paper writes RA in formulas and existence in prose, and posits nothing else. The file exhibits it on a constructed domain, one point with differential 1, and the kernel decides the positivity (`constructed_RA`, no axioms). This is a model of the axiom, not its universal extension.

The paper’s standing without the certificate. Each component, its standing, and whether it depends on the certified height.

Component	Standing	Depends on 3×10^{12} ?
The seat, its bound, the aperture, and the assignments of the three measures (Part I, Theorems A to I)	K, kernel theorems, no user-declared axiom; the physical reading of the price is a reading	No
The address and the cone, halted uniduction (Part II, Theorems 44 to 50 and 68)	K given A: Li’s criterion, the $\Lambda = 0$ equivalence, the definitional identification, the seed	No
The logical form and the asymmetry of the denial (Theorems 70 to 77)	K given Σ_1^0 -completeness, and soundness in the converse only	No
The division at any height, the fusion law, the unicorn identity, the split’s separation and monotonicity (III.1 to III.14)	K	No
The certificate schema, the placement, the bridge, the round trip, the arrow (III.15 to III.44)	K	No
The Real part of ζ to height 3×10^{12}	A joined to K, the cited certificate through the schema	Yes, and only this
The formal block of the Unicorn part, the aperture, the silence, and the three bits as one term, III.45–III.49 and <code>rh_register_proof_complete</code>	K, kernel theorems with no axiom; on Z_ξ at the grade of Theorem D and the kernel-cannot-read-the-deed theorem	No

The extension is a premise, and by the root-premise theorem of [Islam 2026a] it can be no more than a premise: a base from which the axiom could be derived would be weaker than the axiom and would then be the root instead. Its exogenous warrant, the kinetic-energy floor and the zero-point energy of a bounded substrate, is physics carried by citation [Islam 2026a]. Every theorem below that is conditional on RA is discharged against the model and stated conditionally against the extension; the ledger below keeps the two apart.

The interaction reading

RAF-1, registration: an event is an interaction iff it carries strictly positive mutual information between the systems it relates. RAF-2, closure: an interaction domain is closed under interaction [Islam 2026j]. At the formal register the file carries a domain as a membership predicate, a symmetric registration relation, and the closure law: $\text{mem}(d) \wedge \text{registers}(s, d) \rightarrow \text{mem}(s)$. Two consequences are proved.

RAF-C1, no exterior agent: $\neg \text{mem}(s) \rightarrow \forall d, \text{mem}(d) \rightarrow \neg \text{registers}(s, d)$ (no_exterior_agent; no axioms). Its positive form is the sentence this paper needs at the aperture: a reader that registers a bit against a member of the domain is a member of the domain (`adjudicator_in_domain`). The witness of the value is never exterior to the domain the value is about, conditional on RAF-2, which enters the file as the closure field of the domain and carries the premise grade the declaration assigns it. The twin checks the equivalence of closure and no-exterior-agent on every one of the 2^{16} subsets of a sixteen-point carrier

under a fixed symmetric registration, and checks the adjudicator-inside clause on every closed subset.

RAF-C2, no faithful self-representation: for any $f : S \rightarrow (S \rightarrow \mathbb{B}), \neg \forall g, \exists x, f(x) = g$ (no_total_self_indexing; no axioms). This is Cantor’s diagonal theorem, which Lawvere’s fixed-point theorem generalizes [Lawvere 1969], and it is the general statement of which the prior proof’s self-check table is the Riemann instance: the file reading itself cannot index the property that would seal its own row. The twin enumerates every map $S \rightarrow 2^S$ for $|S| = 3$ and 4, finds none onto, and finds the diagonal outside the range of every one.

The reach law of the interaction reading is recorded here and consumed in the section on the rule and its bound: it reaches no mathematical object, since mathematical objects do not change state, and every proposition routes to the grounding reading [Islam 2026j, Islam 2026i]. Theorem F is the Riemann instance of that law, proved rather than cited.

The grounding reading

The formal being of a proposition is its imprint in the fixed locus of the binding involution, read across the aperture; $\text{Fix}(\sigma) = \mathbb{R}$ [Islam 2026j]. The section on the seat builds that locus on the quaternion carrier, proves it is the scalar line, and proves that the prior proof’s stage embeds into it equivariantly. The Riemann object read on this face is its imprint in the fixed locus: the seat. Reading the value across the aperture is the subject of the section on the aperture.

The ledger

An adjudication is assent, denial, or silence, and each advances the count of deeds by exactly one (`every_adjudication_is_a_deed`). Denial of the axiom is therefore a deed together with the axiom (`denial_re_enacts_RA`), and a deed registers at least one bit, so by RAF-2 the denier is a member of the domain the axiom is about. The ledger derives nothing from physics and carries the structural grade.

THE SEAT

Theorem A, the fixed locus is the scalar line

Let $\mathbb{H}_{\mathbb{Z}}$ be the integer quaternions with the Hamilton product and σ conjugation, $\sigma(r, i, j, k) = (r, -i, -j, -k)$. Then σ is binding, $\sigma(\sigma q) = q$ (`sigma_binding`, no axioms), and for every q ,

$$\sigma q = q \leftrightarrow (q_i = 0 \wedge q_j = 0 \wedge q_k = 0)$$

(`fix_iff_scalar`; dependency set `propxt`, `Quot.sound`, from the integer step $-i = i \Rightarrow i = 0$). The Ground is $\text{Ground} := \{q \in \mathbb{H}_{\mathbb{Z}} \mid \sigma q = q\}$, and by Theorem A it is the scalar line $Z(\mathbb{H}) = \mathbb{R}$ restricted to the lattice. The twin checks the biconditional and the involution on all $13^4 = 28,561$ points of the ball $|\text{coordinate}| \leq 6$ and counts the fixed points: thirteen, the scalar line of the window.

Theorem B, the Return lands on the seat

Let i, j, k be the imaginary units and $\text{Return} := (i \cdot j) \cdot k$. Then $\text{Return} = \langle -1, 0, 0, 0 \rangle$ and $\sigma(\text{Return}) = \text{Return}$ (`return_is_minus_one`, `return_lands_on_fix`; both `rfl`). The parse triad returned through its own cascade lands on the Ground at -1 : the axiom witnessing the axiom, as a computation. The twin computes all six orderings: the three even ones land at -1 and the three odd ones at $+1$.

Theorem C, the identity cone

The seat is the fixed locus, the line; a seat point is a point of it. Three seat points are defined independently: the axiom's seat point $\Gamma_{\text{RA}} := \text{Return}$, the formal seat point $\Gamma_{\text{RAM}} := \pi(\text{Return})$ with $\pi(q) := \langle q_r, 0, 0, 0 \rangle$ the projection onto the fixed locus, and the named seat point $\Gamma_{\text{RH}} := \langle -1, 0, 0, 0 \rangle$. Let D be the discrete diagram $\text{RA} \mapsto \Gamma_{\text{RA}}$, $\text{RAM} \mapsto \Gamma_{\text{RAM}}$, $\text{RH} \mapsto \Gamma_{\text{RH}}$, and let a cone over D with apex a be a family of identities $a = D(r)$ for all three r . Then:

$$\sigma \Gamma_{\text{RA}} = \Gamma_{\text{RA}}, \quad \Gamma_{\text{RA}} = \Gamma_{\text{RAM}}, \quad \Gamma_{\text{RAM}} = \Gamma_{\text{RH}},$$

each by `rfl` (`self_gap_nonexistent`, `register_gap_nonexistent`, `object_gap_nonexistent`); Γ_{RH} is the

apex of a cone over D with every leg `rfl` (`identity_cone`); any two cones over D share their apex (`cone_apex_unique`); and a cone exists iff the register gap and the object gap are closed (`cone_iff_gaps_closed`). All four carry no axioms. The twin computes the gap vector $(0, 0, 0)$ and, over the whole lattice ball, counts the apexes of D : exactly one.

The apex is unique for the diagram, and the diagram is built from the ordered triad. The reversed triad $k \cdot j \cdot i$ returns $+1$ (`odd_return_is_plus_one`), also on the fixed locus (`odd_return_on_fix`), and the two candidate seat points differ (`seat_points_differ`; all three no axioms). The sign of the seat point is the orientation of the triad, one bit: the kernel reads it once the order is supplied, by `rfl` either way, and originates it never, since the order is supplied with the triad: the axiom's declaration fixes the three units in an order, and the kernel receives that order as data [Islam 2026j]. Uniqueness of the apex is uniqueness per orientation, and the two orientations' apexes are exchanged by an odd relabeling of the imaginary units. Nothing in the closure depends on which of the two is named: the three identities of Theorem C hold verbatim for the reversed triad, each `rfl` (`odd_self_gap_nonexistent`, `odd_register_gap_nonexistent`, `odd_object_gap_nonexistent`), the reversed diagram carries its own identity cone with apex $+1$ (`identity_cone_odd`), and the two apexes differ (`apexes_differ_by_orientation`; all no axioms). The twin counts the apexes of the reversed diagram over the ball: exactly one, at $+1$.

The word apex is used in two senses that coincide by design: the apex register, the register in which the seat lives, and the apex of the cone; the register is where the cone's apex sits. The mathematics of Theorem C is Hamilton's product and three definitional equalities, and the paper claims nothing more for it at theorem grade. What it claims at structural grade is the arrangement: three names written independently in three registers, the axiom's `Return`, the projection onto the locus, and the point the prior proof names, read one point, and the reading is decided by the kernel rather than declared by the text. The category-theoretic content is exact and small. A category gap is the absence of a leg from an apex to a register's reading of the seat. Its elimination is the exhibition of that leg as an identity. The eliminator $\text{RA} \rightarrow \text{RA} \rightarrow \text{RAM} \rightarrow \text{RH}$ is the composite of the three legs, and its masslessness is the statement that each leg is `rfl`: two definitions, one value, decided by the kernel. The gap vector $(1, 1, 1) \rightarrow (0, 0, 0)$ is the cone's existence read bit by bit, computed by `decide`, never written by hand.

Theorem C', the equivariant embedding

Define $\varphi : \mathbb{Z}^2 \rightarrow \mathbb{H}_{\mathbb{Z}}$ by $\varphi(h, t) := \langle t, h - 1, 0, 0 \rangle$. Then $\sigma \circ \varphi = \varphi \circ \tau$ (`phi_equivariant`); for every stage point, $\sigma(\varphi p) = \varphi p \leftrightarrow \tau p = p$ (`phi_fix_iff`); φ is injective (`phi_injective`); and the seat point lies on the image of the line, $\varphi(1, -1) = \Gamma_{\text{RH}}(\text{seat_on_image_of_line}, \text{rfl})$; the family $\varphi_m(h, t) := \langle t, h - m, 0, 0 \rangle$ against the fold $\tau_m(h, t) = (2m - h, t)$ is equivariant with the same image clause at every resolution m (`phiM_equivariant`, `phiM_fix_iff`), and $\varphi_1 = \varphi$. The dependency sets of the three are `propext`, `Quot.sound`, from the integer arithmetic. The twin checks equivariance, the image clause, and injectivity for $m = -3, \dots, 3$ on the window $|h|, |t| \leq 8, 289$ stage points each, and counts the line: seventeen points at every m .

This is the equivariant map with an image clause that the twenty-three-row study requires before a finite result is invoked on an object domain [Islam 2026c]: the prior proof's line $h = 1$ is carried onto the scalar line and every off-line stage point is carried off it. The identification of the two seats, the stage's line $h = 1$, the lattice image of the critical line under the prior proof's chart, and the real line under σ , is therefore a theorem about a map and not a typing by resemblance; the analytic critical line itself enters only through the prior proof's chart propositions, cited and not carried. What the embedding does not do, and the paper does not claim, is carry Z_{ζ} itself: the stage represents no off-line point inside the strip at resolution one, and the analytic frame enters the kernel only as a parameter.

The constructed witness

$\text{RH}_{\text{formal}} : \Leftrightarrow \forall g \in \text{Ground}, \sigma g = g$ quantifies over the Ground, which is the fixed locus by construction, and it holds by that construction: it is the imprint of the Riemann object in the fixed locus, the grounding reading. It names the seat. It does not mention Z_{ζ} , and the file's own comments say so. The Root Axiom is the only supplied witness, a structure in Prop with four fields, actuation, the Bridge-born orientation bit, the formal-Ground proposition $G_{\text{RAM}} := \text{Nonempty}(\text{Ground})$, and the recursion capacity $\Omega \rightarrow G_{\text{RAM}} \rightarrow \text{RH}_{\text{formal}}$; the eliminator is Bridge followed by the recursion from that witness alone, `RH_formal_chain`, no axioms, and no second proof object enters. The witness's recursion field is the constant function: it discards the orientation and the formal Ground and returns the Ground's own defining property (`recursion_is_constant`, `rfl`). That is the masslessness stated as a term, not a defect concealed: the formal self needs nothing from existence to be proved, so the chain is not a derivation of the seat from the axiom, and the paper does not call it one; it

is the occupancy of the three registers by one point, which the witness supplies and the kernel reads.

The orientation bit is placed in Prop on purpose. Prop does not eliminate into data, so no term of the file computes a bit from Ω ; two orientation witnesses are indistinguishable, `orientation_proof_irrelevant`, by `rfl`. The kernel registers that orientation was supplied and cannot read which way. This is the prior proof's Theorem 4 as a universe level, and it is executed on the constructed seat as well: no readout of the formal seat equals the deed bit (`kernel_cannot_read_the_deed`; no axioms). The twin enumerates all 2^{13} readouts of the seat's window and finds that none factors the deed bit. A term that consumed Ω as data would be such a readout. The bit therefore has two carriers in the file and the relation between them is stated once: Ω is the bit as the kernel registers it, present and unreadable; the Boolean assent of the live face is the bit as the aperture receives it from outside, an input the file reads and never generates, which is the content of the prior proof's self-check table and, one register up, of RAF-C2.

THE APERTURE

Theorem D, the aperture is opened by the axiom

Let a reader be a point of the constructed domain and define its presence as its actuation, $\text{presence}(r) : \Leftrightarrow 0 < \Delta E(r)$, decided. Then

$$\begin{aligned} \forall r, \text{presence}(r) &= \text{true}, \\ \forall r b, \text{live}(\text{presence}(r), b) &\neq \text{open}, \end{aligned}$$

and $\text{live}(\text{presence}(r), \text{true}) = \text{sealed}$, $\text{live}(\text{presence}(r), \text{false}) = \text{refused}$ (`RA_opens_the_aperture`, `no_interior_under_RA`, `aperture_reads_the_bit`; no axioms). The same holds for any domain U with any $\Delta E : U \rightarrow \mathbb{Z}$ under the axiom as a hypothesis: from $\forall x, 0 < \Delta E(x)$, every point of U is present and no present point's row is interior (`RA_opens_the_aperture_general`, `no_interior_under_RA_general`; no axioms), which is the form the universal extension licenses wherever it is held. Here live is the prior proof's live face unchanged: witnessed and assent seal, witnessed and denial refuse, unwitnessed is open. The twin runs it on a twelve-point domain with positive differentials, twenty-four rows, none interior, and runs the control: a point with differential zero is interior.

The scope is stated with the theorem. In the prior proof, "witnessed" means that a supplier is present, and a supplier is a reader who supplies a bit. A reader is a point of the axiom's domain; the theorem says nothing about whether a reader of ζ exists, which is the universal extension's to supply. Theorem D concerns a present reader who supplies a bit b : for such

a reader the axiom makes the witnessed flag a theorem, and the row is sealed or refused by b alone. What the axiom turns the open row into is therefore not “sealed”; it is “sealed or refused, by the bit alone”. The interior state, the $[?]$ of the prior proof’s self-check table, is the state of a reading that supplies no bit, the file reading itself, and the file cannot supply one: that is RAF-C2 at the file, and narcissus is carried unchanged. The axiom supplies presence at the aperture and, conditional on RAF-2, places the present reader inside the domain; it does not supply the bit, and the section on the rule and its bound proves that it cannot.

THE RULE, EXACT, AND ITS BOUND

THE RULE, EXACT, AND ITS TWO OBJECTS. (tier: kernel theorem) Two objects carry the name in this paper and are never joined. The seat, RH_{formal} , the imprint of the restored hypothesis in the fixed locus of its binding involution, is a kernel theorem with existence as the only posit, with the identity cone, the equivariant embedding of the prior proof’s chart, the orientation blindness, the one-bit freedom, the halting carrier, and the crossing (Theorem H). The value, $L(X_{\xi}) : \Leftrightarrow \forall \rho \in Z_{\xi}, \tau\rho = \rho$, is one bit that existence does not reach; Part III proves it below a certified height by computation and leaves it open above, as the Unicorn part. Existence reaches the world row at its aperture: a present reader who supplies a bit is never interior (Theorem D) and, conditional on RAF-2, never exterior (RAF-C1). It does not reach the value, and that is a theorem three times over: $(RA \rightarrow L(X)) \leftrightarrow L(X)$ wherever existence holds, as for any inhabited premise (Theorem E); existence holds on frames where L fails, seatless and grounded alike (Theorems F, F’); and for every class of frames, existence decides L on it exactly where L already holds on it (Theorem I). Denying existence is a deed that re-enacts it; denying the value is not (Theorem G). The distance between the seat and the value is one bit, and the paper proves that nothing weaker than the bit closes it: Theorem E fixes the rule’s strength, Theorem F, the frame on which the axiom holds and the value fails, fixes the independence, and neither is read without the other; the bit is one because the fold has two orientations and no third, the price bijection of the fTOE carried in the Codex.

Theorem E, the rule is exactly the hypothesis

For every frame X , in the kernel where RA is the constructed model,

$$(RA \rightarrow L(X)) \leftrightarrow L(X)$$

(simple_rule_exact; no axioms). Proof. Left to right: apply the implication to the constructed RA. Right to left: a term of $L(X)$ is a term of $RA \rightarrow L(X)$ that ignores its argument. $\square$

Scope. Propositionally, $((P \rightarrow Q) \leftrightarrow Q) \leftrightarrow (P \vee Q)$: the biconditional holds in every context where P holds, and in a context where P fails it holds only

where Q already holds, where it says nothing. The twin prints the four rows. In the kernel RA is the constructed model and holds by constructed_RA, so the theorem is unconditional there. Against the universal extension, held as a premise and never a kernel term, the same biconditional holds in every context that carries the premise. The same biconditional holds for any inhabited premise in place of existence (any_true_premise_is_exact; no axioms), and that generality is the theorem’s content rather than a weakness of it: existence is a premise that holds wherever the line property is evaluated and is independent of it, so conditioning on it adds nothing to the hypothesis and removes nothing from it. What existence contributes beyond its truth is used elsewhere: the positivity that makes presence a theorem (Theorem D), the closure of the interaction reading (RAF-C1), and the ledger (Theorem G). Theorem E is the prior proof’s Theorem 12 in a new coat. A reader who holds a term of $RA \rightarrow L(X_{\xi})$ holds, in any context where the axiom is available, a term of $L(X_{\xi})$, and the paper claims no such term.

Theorem F, the axiom decides the value on no frame

$$\neg \forall X, (RA \rightarrow L(X)), \quad RA \wedge \text{Inv}(X_2) \wedge \neg L(X_2),$$

where X_2 is the two-point frame $(\mathbb{B}, b \mapsto \neg b, \top)$ (RA_does_not_decide_L, RA_holds_where_L_fails; no axioms). Proof. The two-point frame is fold-invariant and off the line, the prior proof’s Theorem 9; the constructed RA holds regardless of any frame, since its domain is not the frame. $\square$

This is the prior proof’s Theorem 11 with the axiom added to the structural clauses, and it is the Riemann instance of the interaction reading’s reach law: the axiom’s domain is a domain of systems, the frame is a mathematical object, and the one does not reach the other. The same fact read from the other side is the inverted control of the twenty-three-row study [Islam 2026c]: a witness identical on the on-line frame and the off-line frame decides neither, and mkRA is that witness.

Theorem F’, the counter-model survives grounding

The two-point frame is groundless: its fold fixes nothing, $\forall b, \tau b \neq b$ (twoPoint_groundless), so it has no seat and the hypothesis has nowhere to place a zero on it. A reader may take that for the whole reason the axiom fails to decide L there. It is not. Let the three-point frame be $S = \{a, b, c\}$ with the fold swapping a and b and fixing c , and $Z = \{a, b\}$. It has a seat, $\tau c = c$; it is fold-invariant; and L fails on it, since $\tau a = b \neq a$. Existence holds on it regardless:

$$RA \wedge (\exists s, \tau s = s) \wedge \text{Inv}(X_3) \wedge \neg L(X_3)$$

(RA_holds_where_L_fails_grounded; no axioms). This is the shape of the Davenport–Heilbronn frames: a line that exists and a zero that is not on it. The twin enumerates all thirty-two three-point frames, four involutions against eight zero sets: L holds on fourteen, fails on eighteen, and every one of the eighteen counter-models is grounded. The bound of Theorem F does not rest on a seatless frame.

Theorem I, the cure theorem

Every proposal to remove the counter-models by restricting the frames has the same form: a class C of frames on which the axiom is to decide the value. For every such class,

$$(\forall X, C(X) \rightarrow RA \rightarrow L(X)) \leftrightarrow (\forall X, C(X) \rightarrow L(X))$$

(no_cure; no axioms). Proof. Left to right: apply the implication to the constructed RA. Right to left: the implication ignores its argument. $\square$ The class of all frames is not cured, and the class $\{X \mid L(X)\}$ is (cure_is_the_hypothesis; no axioms). The twin checks every one of the 256 classes of two-point frames: exactly the 32 subclasses of the five frames on which L holds are cured.

The theorem is Theorem E read as a no-go, and it closes a door that a reader might otherwise try. A frame class carrying an Euler product as a field, or an operator whose spectrum is the zero set with real eigenvalues guaranteed by construction, is a class C on which L is already a field of the structure: real eigenvalues on the fixed locus is the line property written as a hypothesis. On that class the axiom decides the value, by Theorem I, and it decides nothing, because the value was supplied when the class was defined. The proposal restates the hypothesis and calls the restatement a proof, which is the circularity the prior proof’s Theorem 12 names and criterion F3 fires on. What existence does supply is proved and enumerated: the seat, the cone, the embedding, the aperture, the reader’s presence and place inside the domain, the exact strength of the rule, and the bound. What it does not supply is exactly one bit, on every class of frames that does not already carry it.

One more collapse is answered here because it will be proposed. Computing is a deed and reading is a deed, and a reader may say that the act of running the checkers is the act that supplies the bit, so that witness and reading are one object counted twice. They are not one object, and the file says why: the formal read of the seat is even under the flip and the deed bit is odd under it, $\text{formalRead}(\text{execFlip } s) = \text{formalRead}(s)$ and $\text{ran}(\text{execFlip } s) = \neg \text{ran}(s)$, so no readout equals the bit (kernel_cannot_read_the_deed), and the twin finds no readout among 2^{13} . Running the checkers

supplies presence; it does not turn the Boolean the operator gives the live row into a truth about $Z_{\mathbb{C}}$, and the twin’s live row is exactly the place where the difference is executed: the program cannot generate its own witness, and the witness it receives is a Boolean whose correctness the world alone can refute.

Theorem G, the asymmetry

Every adjudication is a deed and the deed re-enacts the axiom: $(\text{adjudicate denial } \ell).\text{deeds} = \ell.\text{deeds} + 1$ together with RA ($\text{denial_re_enacts_RA}$). On the two-point frame no assent to L exists at all: $\neg \exists t : L(X_2)$, $\text{row}(\text{assent } t) = \text{sealed } (L_has_no_performative_ingress; \text{no axioms})$.

Read together: denial of RA produces a deed, a deed registers at least one bit and is therefore an instance of the axiom’s content, so the denial re-enacts the axiom. Denial of L produces a deed too, which is an instance of the axiom’s content and not of L ’s, since a deed is not a zero; on an off-line frame the denial of L is simply true and no assent exists. The axiom is self-instancing. The hypothesis is not. The hypothesis therefore cannot inherit the axiom’s performative ingress, and that is the exact reason the sign stays at the witness row: the deed that supplies the bit is unfakeable and its price is real, and neither carries a sign [Islam 2026e].

Theorem H, the hypothesis in toto at the apex

$$\begin{aligned} RA \rightarrow & \left[(\sigma\Gamma_{RA} = \Gamma_{RA} \wedge \Gamma_{RA} = \Gamma_{RAM} \right. \\ & \wedge \Gamma_{RAM} = \Gamma_{RH}) \\ & \left. \wedge RH_{\text{formal}} \wedge F_2 \wedge B \wedge C \wedge A \wedge N \right], \end{aligned}$$

where F_2 is the prior proof’s Theorem 5, B its Theorem 7, C the crossing of its Theorem 8, A the aperture of Theorem D, and N the bound of Theorem F ($RH_in_toto_at_the_apex$; dependency set Quot.sound , from function extensionality in Theorem 5). The hardened form adds the identity cone with its apex unique per orientation, the equivariant embedding with its image clause, and the executed wall (apex_hardened). The condition is discharged by the constructed axiom (apex_discharged , $\text{apex_hardened_discharged}$), so the closure stands unconditionally as a kernel theorem and conditionally on the axiom’s universal extension as a claim about the world. $\blacksquare$

The antecedent RA is consumed by the aperture conjunct A , which needs positivity, and by no other conjunct; the identities, the formal self, the freedom, the carrier, the crossing, and the bound hold unconditionally, and the implication form records that existence is the only posit, not that those conjuncts depend on it. The theorem is the maximal claim this material

supports, and its last conjunct is what keeps it maximal rather than inflated: the closure carries its own bound inside it. Given the axiom, everything about the hypothesis that is not its sign is proven, and the sign is proven to be exactly what the axiom does not reach.

THE DUAL-REGISTER VERDICT. (tier: structural; parting a kernel theorem) Apex register: the Riemann object read as $\text{Fix}(\sigma)$, the seat, its imprint on the grounding reading, the seat point Γ_{RH} on it; proved, given the Root Axiom, in the kernel, conditional on the axiom's extension at the act; bits owed, none. World register: the zeros of ζ placed on that seat by the world; proved by certificate up to a certified height (Part III) and open above it, one bit supplied from sight at the witness row, revisable in its rows and falsifiable by one zero; bits owed, one, the term $\forall \rho \in Z_{\zeta}, \tau \rho = \rho$. The two tokens differ, registers_parted, and the owed counts are 0 and 1, apex_owes_nothing_world_owes_one. Neither register promotes the other.

THREE MEASURES: CONTENT, MASS, AND PRICE

Two readers of an earlier edition, both machines, reached the same word for the closure: vacuity. The word names one measure and misreads two others, and since the confusion will recur wherever the paper is read by a substrate that keeps one measure, the three are separated here and carried in the file.

Content is what an object says, and it is graded by how it is settled: decided by definitional equality, proved as a theorem, or supplied as data. Mass is the mathematics authored in producing the object, the program's ΔM , and it is zero when every step is classical, definitional, or a deed. Price is the energy the act of registering the object dissipates, $k_B T \ln 2$ per bit at the floor [Landauer 1961, Bérut et al. 2012]. The file carries the three as a structure and assigns them (Measure, legMeasure, arcMeasure, valueMeasure), and proves the three facts that the word vacuity runs together.

Theorem J. The masses are all zero, $\text{legMeasure.mass} = \text{arcMeasure.mass} = \text{valueMeasure.mass} = 0$ (masses_all_zero); the contents are pairwise distinct (contents_differ); and the value alone is priced, $\text{legMeasure.priceYJ} = 0$ and $\text{valueMeasure.priceYJ} = 2871$ (prices_differ); all by rfl or decide, no axioms. The twin checks the constancy of the witness's recursion field in both arguments, the exactness of any true premise, the carrier by decision on all thirty-two three-point frames, and the three measures.

The reading. Masslessness at the apex is not a weakness the paper confesses; it is a necessity the root-premise theorem forces [Islam 2026a]. No emitter is pointed at the root: nothing weaker than existence derives existence, and a closure between existence and

its reading that carried mass would be exactly such an emitter, a derivation of the root from below, or a second posit standing between the root and its reading. So the elimination of the category gaps must be massless, and the paper computes that it is: three legs, each rfl, mass zero, price zero. Trivial and massless coincide there, and the coincidence is the theorem. The supply of the value must be massless too, and for the same reason: the bit is a deed, and a deed authors no mathematics; if supplying the bit carried mass, the supply would be a theorem and the theorem would be an emitter pointed at the value from the root, which Theorem I refutes on every class of frames. So the value is massless. It is not trivial: its content is one bit that no class of frames short of the hypothesis carries, and its price is the one nonzero entry in the table. Vacuity names the seat's content and misnames its mass, its price, and its place. What is empty at the apex is what must be empty there, and what is one bit at the world row is one bit because the paper proves it cannot be less.

The same three measures answer the question what existence contributes over any other true premise. As a premise in the kernel, nothing, and Theorem E says so. As a posit, three things no arithmetic truth supplies: a domain of readers whose presence is a theorem (Theorem D), the closure of that domain under registration (RAF-C1), and the fact that every adjudication of any proposition, the hypothesis included, is a deed and therefore an instance of existence and of nothing else (adjudicating_anything_instances_existence). Denying $1 + 1 = 2$ is a deed too, and the deed instances existence, not arithmetic. That is the asymmetry of Theorem G stated for every proposition at once, and it is why existence, and not $1 + 1 = 2$, is the posit at the aperture.

PART II The Address

THE BRIDGE FROM THE ROOT AXIOM TO LI POSITIVITY

The bridge is a structure with two fields: a choice of one existent $\varphi(n)$ for each Li step, and a reading that turns the positive energy of that existent into the non-negativity of λ_n .

$$\begin{aligned} \text{Bridge}(S, L) := & \langle \varphi : \mathbb{N} \rightarrow S.U, \\ & \text{read} : \forall n \geq 1, \\ & 0 < \Delta E(\varphi n) \rightarrow \lambda_n \geq 0 \rangle \end{aligned}$$

Table 2: Contributions of Part I. (tier as stated per row)

#	Contribution	Status
1	The posit under which the supplier acts named: existence, constructed as the only posit, read along its two formal readings; the witness supplies occupancy of the registers, not a derivation	Lean, no axioms
2	RAF-C1 and RAF-C2 at the formal register: no exterior agent; no total self-indexing (Cantor); both re-executed exhaustively on finite carriers	Lean, no axioms; twin
3	The aperture is a theorem of the axiom: a present reader who supplies a bit is never interior (Thm D); conditional on RAF-2, never exterior	Lean, no axioms; RAF-2 premise
4	The axiom turns “open” into “sealed or refused by the bit alone”, never into “sealed”	Lean, no axioms
5	The fixed locus of the binding involution is the scalar line, for every quaternion (Thm A); the ball enumerated	Lean, propext, Quot.sound; twin
6	The Return $i \cdot j \cdot k$ lands on the seat at -1 (Thm B); all six orderings computed	Lean, rfl; twin
7	The three category gaps are the legs of an identity cone; the apex is unique per orientation of the triad; a cone exists iff the gaps are closed (Thm C); the cone census over the ball	legs Lean, no axioms, rfl; the elimination as arrangement structural; twin
8	The lattice stage embeds equivariantly and injectively into the carrier, $\sigma\varphi = \varphi\tau$, with $\text{Fix}(\tau) \leftrightarrow \text{Fix}(\sigma)$ at every resolution (Thm C')	Lean, propext, Quot.sound; twin
9	The eliminator $\text{RA} \rightarrow \text{RA} \rightarrow \text{RAM} \rightarrow \text{RH}_{\text{formal}}$ from the witness alone	Lean, no axioms
10	Orientation in Prop is unspendable by the kernel; the wall executed on the seat; every readout enumerated	Lean, no axioms; twin
11	The Simple Rule, exact: $(\text{RA} \rightarrow L(X)) \leftrightarrow L(X)$ wherever the axiom holds (Thm E); the four-row table	Lean, no axioms; twin
12	The axiom decides the sign on no frame; it holds where L fails; the Riemann instance of the reach law (Thm F)	Lean, no axioms; twin
13	The asymmetry: the axiom is self-instancing, the hypothesis is not (Thm G)	Lean, no axioms
14	The seat and the arc in toto at the apex, one theorem carrying its own bound; hardened with cone, embedding, and wall (Thm H)	Lean, Quot.sound
15	The two registers parted by theorem; owed bits 0 and 1	Lean, no axioms
16	The spent bit priced at the apex at $k_B T \ln 2$; in the kernel the bit is proof-irrelevant and no readout of the seat equals it	structural; twin
17	Two independent checkers, a typechecker and an executor, written to one specification and sharing no code; the executor's live row opened only by the operator	structural
19	The halting carrier inhabited by decision on decided frames, the clause proved from the decision; Theorem E for any inhabited premise; the witness's recursion field constant, stated	Lean, no axioms
20	Three measures kept apart, content, mass, and price: every object of the closure and the supply at mass zero, the contents distinct, the value alone priced (Thm J)	Lean, no axioms; twin
21	Adjudicating any proposition is a deed and an instance of existence and of nothing else: what existence contributes that no other true premise does	Lean, no axioms
18	The counter-model survives grounding (Thm F'); the cure theorem: for every class of frames the axiom decides L exactly where L already holds (Thm I); every class of two-point frames enumerated	Lean, no axioms; twin

Table 3: The three measures of the closure's objects. (tier: kernel theorem for the assignments; structural for the reading)

Object	Content	Mass	Price
a leg of the cone (Thm C)	trivial, rfl	0	0
a theorem of the arc (Thms A, C', D, E, F, F', I)	proved	0	0
the value $L(X_{\xi})$	supplied, one bit	0	$k_B T \ln 2$, 2871 yJ at 300 K

Theorem 1 (bridge_yields_RH). The Root Axiom, a bridge, and Li's criterion give the hypothesis. The composition is direct: every step receives an existent, the axiom makes its energy positive, and the reading converts that positivity.

Theorem 2 (bridge_is_keyed). The axiom never supplies the bridge. On a substrate where the axiom holds, paired with data carrying one negative coeffi-

cient, no bridge exists. The bridge is new content.

Theorem 3 (no_uniform_bridge). A bridge whose reading ignores which L-function it serves forces positivity on every member of the family. One member with a negative coefficient, the Eisenstein L-function under Bombieri-Lagarias, destroys every uniform bridge. The reading must consume ζ -specific data.

Theorem 4 (li_is_a_bit_stream). Under Li's criterion and decidable signs, the hypothesis is exactly the statement that a stream $\mathbb{N} \rightarrow \text{Bool}$ stays true. This is the type repair named in the Introduction: a vanishing becomes a family of signs, the shape a supplied deed can carry.

THE FORMAL STRING ON THE UNITED REGISTER

Theorems 5 to 8. The unity of the formal and kinetic readings on ζ reduces to one term, the bridge's reading, carried at the grade of whatever supplies it. The

emitter returns a seal when unity is supplied and the suspension token otherwise, and `xi0_retires_iff_unity` proves the equivalence both ways. `retired_seal_is_sound` proves the seal carries the hypothesis. `retired_grade_capped` is the grade law: the seal is never stronger than its unity posit. `unity_is_keyed` confirms that the axiom does not supply unity. `ghost` records the model: the arrow-deleted reading is defined as a constant, so it returns one output for a claim and its negation. The substantive fact, that a reading with its index removed cannot separate a claim from its negation, is Part I's `recursion_is_sign_blind`, which holds for P and for $\neg P$ alike.

Two retirements follow and are kept apart. The suspension token retires unconditionally as a verdict on a timeless object, since a reading without its index is sign-blind by Part I's `recursion_is_sign_blind`, which ghost models. It retires as the status of the formal string only when unity is supplied, and then at unity's grade.

THE STRUCTURAL CORE

The instruments that cannot carry the bit, and the one that reaches $\text{Re } s = 1$

Theorem 9 (`common_phi_is_uniform`). A family of bridges that picks its existents without reading the L-function is a uniform bridge. **Theorem 10** (`blind_constructions_die`). Every such family dies against the Eisenstein control. Every instrument carried at zero authored mathematics, the one-bit species, the Bridge socket, the category-gap eliminator, the multi-recursion collapse, the bare arrow, falls under Theorem 10. This is the instruments working as apex instruments must, not a defect: the Empty Throne forces them massless [Islam 2026a], and a massless instrument closes category gaps and carries no value.

The one construction that reads ζ directly takes the primes as existents with energies $\log p > 0$. This is the primon gas [Julia 1990; Bost and Connes 1995], whose partition function is $\zeta(\beta)$, and on which the axiom's positivity holds at every existent. Positivity of those energies gives $\zeta(s) \neq 0$ for $\text{Re } s > 1$, and Mertens' positivity pushes it to the line $\text{Re } s = 1$.

Theorem 11 (`mertens_identity`, `mertens_nonneg`). The algebraic core of Mertens' argument, $3 + 4c + (2c^2 - 1) = 2(1 + c)^2 \geq 0$ with $c = \cos \theta$, is checked on integers. The wall at $\text{Re } s = 1$ is the gas's Hagedorn point, and the register of record's finding that the Root Axiom's native output is a wall at $\text{Re } = 1$ rather than a fold at $\frac{1}{2}$ is read here as this wall, an identification argued from their common location and not proved [Islam 2026a]. Below it the Euler product diverges.

Time, made exact: the Li modes

Li's coefficient is $\lambda_n = \sum_\rho [1 - z_\rho^n]$ with $z_\rho = 1 - 1/\rho$. The step $n \mapsto n + 1$ multiplies every mode by z_ρ , so each zero is a mode evolving in a discrete time. With $\rho = h/2 + it$ on the Bridge plane,

$$|z_\rho|^2 = \frac{|\rho - 1|^2}{|\rho|^2},$$

$$4|\rho - 1|^2 = (h - 2)^2 + 4t^2 =: N_1,$$

$$4|\rho|^2 = h^2 + 4t^2 =: N_0,$$

and $N_1 - N_0 = 4 - 4h$ (`N1_sub_N0`).

Theorem 12 (`unitary_iff_on_line`). $N_1 = N_0$ iff $h = 1$. On the lattice stage, a mode is unitary, neither growing nor dying under the time step, exactly on the critical line.

The general identity (6.2a). For every complex $\rho = \sigma + i\gamma$ with $\rho \neq 0$,

$$|\rho - 1|^2 - |\rho|^2 = 1 - 2\sigma, \quad \text{so}$$

$$|z_\rho| = 1 \iff \sigma = \frac{1}{2}, \quad |z_\rho| > 1 \iff \sigma < \frac{1}{2}.$$

The identity is one line of algebra, $(\sigma - 1)^2 - \sigma^2 = 1 - 2\sigma$, and it is the real-coefficient form of `N1_sub_N0`. Theorems 12 to 15 are its kernel-checked lattice instances; the same algebra, with the fold $\sigma \mapsto 1 - \sigma$, carries Theorems 13 to 15 to every zero set invariant under $s \mapsto 1 - \bar{s}$. The stage coordinates are $(h, t) = (2\text{Re } s, \text{Im } s)$, under which the fold $(h, t) \mapsto (2 - h, t)$ is exactly $s \mapsto 1 - \bar{s}$. The nontrivial zeros of ζ are invariant under it: the functional equation $\zeta(s) = \zeta(1 - s)$ sends ρ to $1 - \rho$, the conjugate symmetry $\zeta(\bar{s}) = \overline{\zeta(s)}$ sends ρ to $\bar{\rho}$, and their composite is the fold. Theorems 14 and 15, in their general form through (6.2a), therefore apply to the zero set of ζ . The kernel proves the lattice statements; the general statements rest on (6.2a), which is elementary and stated here in full.

Theorem 13 (`grows_left`, `dies_right`). Left of the line the mode grows; right of it the mode dies.

Theorem 14 (`off_line_forces_growth`). For any off-line zero, the zero or its mirror under the fold $(h, t) \mapsto (2 - h, t)$ carries a growing mode. The functional equation forbids a quiet exit.

Theorem 15 (`stability_iff_line`). For every fold-invariant zero set on the plane, no mode grows under the time step iff every zero lies on $h = 1$.

$$\left(\forall p \in Z, N_1(p) \leq N_0(p) \right) \iff \left(\forall p \in Z, p_1 = 1 \right)$$

Theorem 15 is the core of the title. Read with time restored, the hypothesis is not a statement about a timeless set. It is a stability statement: the time evolution of ζ 's modes has no growing mode. Since z_ρ is the

Cayley transform of the ordinate γ when $\rho = \frac{1}{2} + i\gamma$, unitarity of the modes is equivalent to the reality of the ordinates. This is the scalar spectral condition suggested by Hilbert and Pólya; their program asks for a self-adjoint operator whose spectrum is the ordinates, and no such operator, and no unitary group, is constructed here. A growing mode eventually drives some λ_n negative [Bombieri and Lagarias 1999], so perpetual positivity of the Li stream and the absence of growing modes are the same condition.

The monism rule, hardened

The ancient rule that what happens once happens again, given sufficient time, is time-translation invariance: one principle acting identically at every moment.

Theorem 16 (`one_repeats_one`). The rule with its step is induction. **Theorem 17** (`the_step_is_the_claim`). The universal claim is exactly the base together with the step; the step carries the content. **Theorem 18** (`monism_posit_forces`). If a property does not vary with its index, one instance forces all. **Theorem 19** (`finite_never_forces`). For every height N there is a property true below N and false beyond; the historical failures of Mertens' conjecture, Pólya's conjecture, and the sign of $\pi(x) - \text{li}(x)$ have this shape. **Theorem 20** (`mixed_is_fold_invariant`, `mixed_recurrence_fails`). A fold-invariant zero set can hold one zero on the line and a mirror pair off it; recurrence of "on the line" is not carried by symmetry alone.

Actualism

Under the Root Axiom only actual things exist, and a zero is actual when it is measured. Define the actual hypothesis as the statement that every measured zero is on the line.

Theorem 21 (`full_gives_actual`). The full hypothesis yields the actual one. **Theorem 22** (`actual_not_full`). The actual hypothesis does not yield the full one: a never-measured zero off the line is consistent with every measured zero on it. **Theorem 23** (`sufficient_time`). If every zero is eventually measured, which is what "given sufficient time" means, the actual hypothesis over all time is exactly the full hypothesis. Actualism re-routes to Theorem 17 and does not bypass it. The past actual record is sealed by measurement to the heights the literature reports; the future record is the supplied value.

The witness rebuilt from residual monism

Part I constructs its witness from the Root Axiom read through its interaction and grounding readings. Here

the witness is built from the connector of the register of record directly: one substrate, one involution serving both routes, one principle invariant in the index, and one seed.

$$\text{MonismWitness}(P) := \langle \sigma_{\text{geo}} = \sigma_{\text{form}}, \forall n m, P n \leftrightarrow P m, P 0 \rangle$$

Theorem 24 (`one_involution_constructed`). The seat field is constructed by `rfl`. It fixes the one-involution choice of the register of record and does not refute the executed pluralist alternative. **Theorem 25** (`timeless_equals_timed`). Under one principle the claim over all time is the claim at one instant: deleting time loses nothing. This is where the timeless aspect of the question is addressed: inside the witness, the timed and timeless readings are one claim. **Theorem 26** (`monism_closes`). The witness closes the record, past and future. **Theorem 27** (`monism_witness_is_the_claim`). A monism witness for P exists iff P holds at every index. **Theorem 28** (`constructed_monism`). The witness is constructed at the model. **Theorem 29** (`monism_absent_on_breaks`). Unlike the axiom's witness, the monism witness is not silent on counter-models; it decides by containing. **Theorem 30** (`monism_yields_RH`). Run on Li's sign stream, the witness gives the hypothesis with the bridge removed. **Theorem 31** (`monism_on_li_is_RH`). On ζ the witness exists iff the hypothesis holds.

Monism supplies unitarity where it applies. Time-translation invariance makes the one substrate's evolution a one-parameter group, and conservation of probability makes that group unitary; Stone's theorem [Stone 1932] then gives it a self-adjoint generator. Under monism, taken with conservation of probability, the one substrate's own evolution is therefore unitary. The open identification is that ζ 's Li modes are that evolution.

Re-anchoring to mathematical standard, with Perelman as structural analogue

Theorem 32 (`global_monism_inconsistent`). Uniformity posited for every property is contradictory. The monism of standard mathematics is therefore never free: it is anchored to a principle.

$$\text{AnchoredMonism}(S) := \langle f : S \rightarrow S, I : S \rightarrow \text{Prop}, \forall s, I s \rightarrow I(f s), s_0, I s_0 \rangle$$

One dynamics, one invariant, a proved transport, a seed. This is induction on an orbit, the most standard witness form mathematics has.

Theorem 33 (`anchored_closes`). The anchored witness closes its orbit. **Theorem 34** (`anchored_derives_`

monism). The free monism witness of 6.5 is derived from the anchored one; uniformity is manufactured by the transport. **Theorem 35** (`lyapunov_is_monism`). A quantity monotone along one flow is an anchored witness for “never below its start.” Perelman’s $\mathcal{W}$ -entropy, nondecreasing along Ricci flow when its auxiliary function evolves by the conjugate heat equation and its scale decreases at unit rate [Perelman 2002], is the structural analogue: his proof has the shape of an anchored witness with its transport proved, and surgery and canonical neighborhoods complete the argument around it. The file formalizes the generic shape, a monotone sequence as a witness; Perelman’s entropy itself is not formalized. **Theorem 36** (`closure_grade_is_transport_grade`). The closure is exactly as strong as its transport. **Theorem 37** (`rh_anchor_is_the_claim`). On Li’s sign stream with the time step as principle, an anchored witness exists iff the hypothesis; its transport reads $\lambda_{n+1} \geq 0 \rightarrow \lambda_{n+2} \geq 0$, the ζ -analogue of Perelman’s monotonicity formula. De Branges proposed a positivity condition of related type, on a space of entire functions, and Conrey and Li [2000] showed it fails, which calibrates where such attempts break.

A monotonicity formula of ζ ’s time, and its arrow

The de Bruijn-Newman heat flow carries a proved monotonicity formula for the zeta function’s time. De Bruijn’s theorem: if every zero of H_0 lies in $|\operatorname{Im} z| \leq \Delta$, every zero of H_t lies in $|\operatorname{Im} z| \leq \sqrt{\max(\Delta^2 - 2t, 0)}$ [de Bruijn 1950]. The hypothesis is the reality of the zeros of H_0 ; it is equivalent to $\Lambda = 0$ by Newman’s definition with $0 \leq \Lambda \leq 0.22$ [Rodgers and Tao 2020; Polymath 2019].

Theorem 38 (`flow_monotone`). The width never grows along the flow. **Theorem 39** (`reality_transport`). Once every zero is real it stays real; this is the transport of 6.6, proved. **Theorem 40** (`rh_iff_lambda_zero`). In the toy, the strip is closed at time zero iff $\Lambda = 0$. For ζ the corresponding equivalence, $\text{RH} \iff \Lambda = 0$, is the cited result of Newman with Rodgers and Tao; it enters the cone as `hLam` and is not proved here. **Theorem 41** (`future_closed`). The anchored witness exists for ζ ’s time, transport proved, seeded at Λ , and closes the whole future from Λ on. **Theorem 42** (`flowed_record_forgets`). After any positive time the flowed record of a hypothesis-true state and a hypothesis-false state coincide; no readout of the flowed record decides the hypothesis. The bound $\Lambda \leq 0.22$ certifies a record at $t = 0.22$ that is blind to the bit, which is why it cannot be pushed to 0 by forward means alone. **Theorem 43** (`upstream_is_not_forced`). Going backward from a real record, both answers remain admissible. The backward direction is anti-diffusion, ill-posed.

This monotonicity formula exists, its witness is complete, and its arrow points away from the hypothesis, which sits at the upstream end.

The regress, stated

Sections 4 to 6.7 each ended on a named residue: the bridge’s reading, the unity posit, the monism field, the transport, the upstream passage. Read in sequence they present as a regress, each residue appearing to require the next. The final result shows it is not one.

Halted uniduction: the five readings as one cone

The category-gap eliminator of Part I exhibits a gap between registers as a missing leg and eliminates it by exhibiting the leg as an identity. Here the registers are the five readings reached, and the legs are equivalences.

Theorem 44 (`hunt_cone`). The hypothesis is the apex of a cone over all five readings:

$$\forall r \in \{\text{bridge, unity, monism, transport, upstream}\} : D_5(r) \iff \text{RH}.$$

Theorem 45 (`one_gap`). Any two readings are the same proposition. The upstream passage, $\Lambda = 0$, is the bridge’s reading, the unity posit, the monism field, and the transport, under a different name. A quantity monotone against the flow would be one way to supply it; that reading is not a leg of the cone, since no such quantity is defined in the file and none is proved equivalent.

Theorem 46 (`regress_halts`). Every proposition proved equivalent to the hypothesis lands on the same apex as the five readings. The theorem does not decide which future propositions will be equivalent; it says that once one is, it adds no new gap.

Theorem 47 (`nothing_weaker`). Given a true proposition Q , proving $Q \rightarrow \text{RH}$ is the same as proving RH: conditioning on a true premise adds nothing. No independence notion is formalized, and the theorem claims none.

Theorem 48 (`sufficient_closes_all`). Theorem 46 covers reformulations equivalent to the hypothesis. A route strictly stronger than it, a specific operator with real spectrum equal to the ordinates or a hypothesis over a whole family, is not equivalent and is not covered there. As a corollary of the cone, any sufficient route supplies all five readings at once; the theorem is a consequence of the cone and says nothing about which routes exist.

The Settled Verdict (tier: kernel theorem on the cone; structural on the reading). Time, stability, and the critical line are proved one object (Theorem 15 on the lattice stage,

and in general by the identity (6.2a)). The timed and timeless readings are proved one claim under monism (Theorem 25). Every reading of the owed bit reached from existence, time, and monism is proved one proposition given the imported inputs of the Methods (Theorems 44 to 46); conditioning on a true premise adds nothing to it (Theorem 47), and any sufficient route closes all five at once (Theorem 48). Timeless residual monism, stated as Postulate M, is proved equivalent to the hypothesis given its computed seed (Theorem 49) and joins the cone as its sixth reading (Theorem 68). The formal hypothesis holds conditionally on exactly one named posit, stated as one field of a standard witness: the transport on the Li stream, equivalently mode unitarity, $|z_\rho| = 1$ for every zero, equivalently $\Lambda = 0$. At the foundation the value carries the terminal mark $[\cdot]$ (Theorems 58 to 69); at the object it is addressed to ζ , supplied at the act and falsifiable by one zero.

Postulate M: Timeless Residual Monism Is Equivalent to the Riemann Hypothesis

The monist intuition behind the paper can be stated in ordinary mathematics, and once stated, with its seed, it is proved equivalent to the hypothesis. Let Z be the set of nontrivial zeros of ζ , enumerated exhaustively by height, so that each zero is located at some finite stage $k(\rho)$. For each stage t let $L(t)$ be the statement that every zero located by stage t has real part one half.

Postulate M, Timeless Residual Monism (tier: premise; the paper's thesis, stated openly). The truth value of $L(t)$ does not depend on t . One principle at the root, and the same answer at every stage: what holds at the timeless interface holds at every stage downstream.

Theorem 49 (`postulateM_iff_line`). Given the computed seed $L(0)$, Postulate M holds if and only if every zero lies on the line. Forward: the seed transported by M to the stage $k(\rho)$ of any zero puts that zero on the line. Backward: if every zero is on the line, $L(t)$ is true at every stage and M holds trivially.

Theorem 50 (`postulateM_decides`). Postulate M cannot hold, with the seed, in any world carrying a located off-line zero. It is not a vacuous posit; one zero refutes it.

Theorems 51 to 57 carry the same result in the timed form of Part I's Bridge plane. The locus is the same line at every time (`SameLocus`), and given that, "every zero is bound to the locus" is exactly the line property (Theorem 51, `bound_iff_line`). Same locus and the fold at every time do not bind by themselves (Theorem 52, `time_does_not_bind`). Residual Monism read as one involution, and read as timelessness, both hold in a world with an off-line pair, since a property that is constantly false is timeless (Theorem 53, `monism_holds_in_counter_world`). Timelessness with the seed at the interface binds every downstream time (Theorem 54, `timeless_with_seed_binds`). The

witness assembled from Residual Monism and the time law, one involution, the same locus, timelessness, and the interface, proves the bound wherever it is built (Theorem 55), is built at a model (Theorem 56, `constructed_at_model`), is not buildable off the line (Theorem 57, `not_constructible_off_line`), and exists exactly when the line property holds (`witness_iff_bound`).

The last category gap, eliminated. A category gap, in the sense of Part I, is the absence of a leg between two registers' readings of one seat, and it is eliminated by exhibiting the leg as an identity. Before this section one such gap stood between the register in which the monist thesis is stated, timeless residual monism at the interface upstream of time, and the register in which the hypothesis is stated, the location of the zeros of ζ . Theorem 49 is that leg. Postulate M joins the cone of the section on halted uniduction as its sixth reading (Theorem 68, `hunt_cone6`, Part XIII of the file), given the identification of the hypothesis with the line property of the enumerated zeros, which is the hypothesis's definition and enters as `hDef` exactly as Li's criterion and Newman's equivalence enter; the collapse of Theorems 44 to 48 then applies to it unchanged: timeless residual monism, the bridge's reading, the unity posit, the monism field, the transport, and $\Lambda = 0$ are one proposition given the imported inputs of the Methods. No category gap remains anywhere between the program's readings and the hypothesis.

What remains is not a gap. It is the value of that one proposition: whether Postulate M holds for ζ . A gap closes at zero cost when its leg is exhibited, as every leg here has been. A value is decided by the function, supplied at the act and falsifiable by one zero. Postulate M is not derived here from the definition of ζ . It does not follow from the functional equation (Theorem 20, and the Davenport-Heilbronn zero at $\text{Re } \rho - \frac{1}{2} = 0.3085$), from symmetry at every time (Theorem 52), or from finite verification (Theorem 19). Its derivation from the Euler product is the single open step, stated in the Discussion.

The Demand, Correctly Addressed: the Terminal Mark at the Foundation

the section on Postulate M leaves the program at its self-referential limit, and that limit can be stated as a theorem rather than met as an obstacle. Every reading of the value that the foundation supplies is equivalent to the hypothesis (Theorems 44 to 49 and 68); the foundation's other resource, the positivity of existence, is independent of it (Part I's Theorems E and F). A demand that the foundation prove the hypothesis therefore asks a register to derive a proposition from readings that already are that proposition.

The Demand, Correctly Addressed (tier: kernel theorem on each clause). A proof of the Riemann Hypothesis cannot be demanded of the foundation. Every reading of the value that the foundation supplies is equivalent to the hypothesis, so any derivation from those readings alone would presuppose what it derives. The demand is well-posed when it is addressed to the object, the function ζ , whose structure fixes where its zeros lie.

Each clause is a theorem of Part XII.

The collapse and its stop. A collapse foundation is a family of registers whose every reading is equivalent to the value (Collapse). **Theorem 58** (self_referential_limit): every reading is exactly as strong as the value, and any two readings are one proposition. **Theorem 59** (value_marked_dot): at the collapse foundation the emitter marks the value $[\cdot]$, the mark outside the three-state economy (dot_is_not_a_verdict, dot_outside_economy). **Theorem 60** (demand_is_ghost_in_register): the foundation returns the same mark for the value and for its negation, so a demand that this register issue a verdict is a demand on a register whose output does not depend on the value.

The self-grounding root, and the line it draws. A root is self-grounding when acts occur and every act, assent or denial, instances it (SelfGrounding). **Theorem 61** (denial_reenacts_root, external_proof_adds_nothing): denying such a root re-enacts it, and no external proof can add to it; the Root Axiom at the constructed domain is of this kind (raSelfGrounding), and its universal extension is carried at premise grade as in Part I. This is the true seat of the argument that a demand for proof can be a ghost. **Theorem 62** (line_not_self_grounding, with_refuted_by_witness): the line property of a zero set is not self-grounding; its value is fixed by the object and one located witness refutes it. Part I's Theorem G states the same asymmetry: the axiom is self-instancing, the hypothesis is not.

The demand, addressed. **Theorem 63** (misaddressed_to_foundation on the two-point frame, and misaddressed_general for an arbitrary frame): any resource the foundation supplies that also holds on a frame where the line property fails cannot by itself yield the line property. **Theorem 64** (derivation_presupposes): a derivation of the value from a reading equivalent to it carries the value's own content; the two stand or fall together. **Theorem 65** (object_decides): the object answers, in one direction by a located witness and in the other by its own structure. **Theorem 66** (demand_correctly_addressed) joins the three. **Theorem 67** (rh_marked_dot) attaches the collapse to the five-register cone of the section on halted uniduction, and **Theorem 69** (rh_marked_dot6) to the six-register cone of The-

orem 68: the program's readings collapse onto the hypothesis, and the foundation marks its value $[\cdot]$.

Terminal Verdict on the Open Part (tier: kernel theorem, Theorems 46, 48, 58 to 69). At the foundation, the value is terminal: every reading of the value the foundation has supplied is proved equivalent to the hypothesis given the imported inputs of the Methods (Theorems 44 to 49 and 68), any further equivalent reading lands on the same apex (Theorem 46), any sufficient route closes every reading at once (Theorem 48), no resource that also holds on an off-line frame can close it (Theorem 63), and the foundation's register closes on the value with the mark $[\cdot]$ (Theorems 67 and 69). At the object, the value is addressed to ζ , well-posed, and falsifiable by one zero. The open part, the Unicorn part, is located in the object; its Real part is proved there by certificate (Part III).

The Logical Form of the Hypothesis: the Only Existence Lives in the Denial

The hypothesis arrives as a question, and a question carries two branches. The two branches do not carry the same kind of content, and the difference is exact.

The hypothesis posits nothing. RH has the form $\forall \rho (\rho \text{ a nontrivial zero} \rightarrow \text{Re } \rho = \frac{1}{2})$. A universal sentence carries no existential import: it asserts no object, and over an empty domain it holds vacuously (Theorem 70, universal_posits_nothing).

The denial posits an object. $\neg$ RH reads: *some* nontrivial zero has $\text{Re } \rho \neq \frac{1}{2}$. That "some" asserts an object, the off-line zero, and a located one refutes the hypothesis (Theorem 71, denial_posits_a_witness). With a decidable line predicate the hypothesis fails *only* by such a witness: it holds exactly when none exists (Theorem 72, fails_only_by_witness). The whole existential load of the question rides on its negative branch.

A witness is a finite check. RH is equivalent to an arithmetic statement about whole numbers, $\sigma(n) \leq H_n + e^{H_n} \log H_n$ for every $n \geq 1$ [Lagarias 2002], each instance of which is a finite computation. RH is therefore a Π_1^0 sentence, as Davis, Matiyasevich, and Robinson showed directly [Davis et al. 1976], and its denial a Σ_1^0 sentence: an off-line zero, if one exists, is a finite object under that arithmetization, an integer instance at which the inequality fails, or a certified enclosure of the zero with the analytic obligations that make it one, and it can be exhibited and verified (Theorem 73, witness_is_a_finite_check); the complex number itself is not the finite datum, its arithmetic witness is.

The denial has never delivered. No off-line zero has been exhibited, computed, or shown to exist at any height. The receipts on record are the verification of every zero to height 3×10^{12} [Platt and Trudgian 2021] and the Original Form paper's own count to height 300 [Islam 2026e]. The structure grants the

denial permission to posit its object; it has never produced an instance.

The asymmetry, as a theorem. Every true Σ_1^0 sentence is provable in Peano arithmetic, and hence in ZFC [Kaye 1991]. If $\neg$ RH were true, computing the off-line zero would prove it. Contrapositively: if ZFC does not refute the hypothesis, the hypothesis is true. In particular, were RH ever shown independent of ZFC, that would imply RH. The hypothesis cannot be false and unprovably false.

The burden, as a theorem. The foundation's inability to decide the value is symmetric (Theorems F and I of Part I, Theorem 63 here): it reaches the value in neither direction. The consequences of that inability are not symmetric, and Part XV of the file proves it.

Theorem 74 (`cant_refute_seals`): if the foundation cannot refute the hypothesis, the hypothesis holds.

Theorem 75 (`rh_iff_cant_refute`): with soundness on the denial, the hypothesis holds exactly when the foundation cannot refute it. **Theorem 76** (`cant_prove_does_not_seal_false`): the foundation's inability to prove the hypothesis decides nothing, since the hypothesis can hold and be unprovable. **Theorem 77** (`asymmetry`): the two inabilityes are not mirror images; a "can't" on the denial side seals the hypothesis, and a "can't" on the assent side seals nothing. All four are axiom-free, with Σ_1^0 -completeness and soundness on the denial carried as named hypotheses, on an abstract setting: the kernel constructs no provability predicate for ZFC and no arithmetization of the hypothesis, and the instantiation of the setting to ZFC and to the arithmetized hypothesis is by the cited theorems [Davis et al. 1976; Kaye 1991], not by the kernel. The one owed bit is therefore owed by the denial. The hypothesis needs no object and has asserted none; the denial needs one object, a finite witness, and has produced none.

Theorem 77 (`asymmetry`): the two inabilityes are not mirror images; a "can't" on the denial side seals the hypothesis, and a "can't" on the assent side seals nothing. All four are axiom-free, with Σ_1^0 -completeness and soundness on the denial carried as named hypotheses, on an abstract setting: the kernel constructs no provability predicate for ZFC and no arithmetization of the hypothesis, and the instantiation of the setting to ZFC and to the arithmetized hypothesis is by the cited theorems [Davis et al. 1976; Kaye 1991], not by the kernel. The one owed bit is therefore owed by the denial. The hypothesis needs no object and has asserted none; the denial needs one object, a finite witness, and has produced none.

The Keystone (tier: kernel theorem for Theorems 70 to 77; [$\neg$ T-conditional on Σ_1^0 -completeness] on the implication "can't refute $\Rightarrow$ RH"). The only existence in the Riemann Hypothesis package lives in the denial, and the denial has never delivered. If the foundation cannot refute the hypothesis, the hypothesis holds: the burden of the one bit lies with the denial, and a denial that could never deliver, one with no witness to exhibit at any height, would make the hypothesis true (Theorem 74). That it has not delivered so far is a record, not a premise: nothing here is inferred from it.

PART III

The Real Part

THE TWO PARTS INSIDE ONE HYPOTHESIS

The hypothesis. Every nontrivial zero ρ of ζ has real part $\frac{1}{2}$. Cut the critical strip at any height T . Below T lie finitely many zeros; above it, infinitely many. The one sentence carries two claims of different kinds.

The Real part at T . Every zero up to height T lies on the line. A statement about finitely many objects, each of which can be located, computed, and checked. In the literature it is the hypothesis "up to height T ", written $\text{RH}(T)$ and used as a working assumption in explicit estimates [Büthe 2016].

The Unicorn part at T . Above T , there is no zero off the line. The only object the Riemann package ever posits lives here: the off-line zero, the object of the denial (Part II, Theorems 70 to 73), never exhibited at any height. If the hypothesis holds, this class is empty, and every sentence about its members is vacuously true, as every sentence about white unicorns is true. So the class is named for what it is.

The seed of Postulate M is the Real part. The division reads Part II's Postulate M exactly. Its seed, the zeros located by stage zero, is a Real part, and with stage zero taken at the certified height it is the Real part proved here; its uniformity across every later stage is the Unicorn part, open; and if stage zero located no zero at all the seed would be vacuous and Postulate M would be the hypothesis itself under exhaustive enumeration, which is why the stage is taken at the certified height and the seed is not vacuous. Postulate M is therefore not a third thing beside the two parts: it is the two parts, with the seed proved and the uniformity set apart.

Why the parts must be separated. A statement stands proved, open, or refuted, and a conjunction stands only as high as its weaker part.

Theorem III.1 (`join_below_both`). The conjunction of two statements never stands higher than either.

Theorem III.2 (`fused_rh_is_open`). With the Real part proved and the Unicorn part open, the fused hypothesis stands open.

Theorem III.3 (`fusion_lowers_the_gold`). Fusion strictly lowers the Real part's standing, from proved to open.

Theorem III.4 (`fused_carries_unicorn`). Every proof of the fused statement contains a proof of the Unicorn part, and a refutation of the Unicorn part refutes the fused statement, however much of the Real part holds.

Theorem III.5 (`kept_apart`). Kept apart, the Real part is asserted exactly as proved, borrowing nothing from the Unicorn part.

Theorem III.6 (`do_not_alloy`). Given the Real part, the fused hypothesis is exactly the Unicorn part, and fusion lowers the Real part from proved to open.

All six are axiom-free. The standings are the

record's: the Real part stands proved by a cited certificate, not by a kernel computation of ζ , and the kernel proves the law that governs whatever standings the parts carry. The conventional statement fuses a proved region with an open one, so the proved content is carried inside an unproved claim; the gold does not survive the alloy. Separating the parts does not deny the ghost of the unicorn. It stops the ghost from haunting the part that is already proved.

In Riemann's own form

Riemann stated the hypothesis as a fixed-set statement, that the roots of $\zeta(t)$ are real [Riemann 1859], and the program's restored form reads it so, $Z \subset \text{Fix}(\tau)$ [Islam 2026e]. In that form the Real part is Riemann's own sentence, bounded: the roots of $\zeta(t)$ with $|t| \leq T$ are real. His sentence already has three clauses in this order: that one finds in fact about so many real roots within these bounds, the counted part; that it is very probable that all the roots are real, the remainder graded as probable; and that a rigorous proof would be desirable but is set aside, the proof this paper does not claim. It was also Riemann's own first move. His unpublished notes, recovered by Siegel, show him computing the first roots of ζ [Siegel 1932], while his sentence claimed the rest: a Real part computed, and the remainder conjectured. The Original Form paper proves that every structural clause of the restored hypothesis holds on a frame whose zeros lie off the line (its Theorem 11), so structure alone decides neither part: the Real part is reached by computation, and the division made here is the one Riemann's own procedure already drew.

THE UNICORN IDENTITY

The class of off-line zeros is a unicorn class exactly when the hypothesis holds.

Theorem III.7 (`rh_makes_unicorns`). If the hypothesis holds, every property holds of every off-line zero, vacuously.

Theorem III.8 (`unicorns_make_rh`). If every property holds of every off-line zero, the hypothesis holds; take the property "false".

Theorem III.9 (`rh_iff_unicorns`). The Riemann Hypothesis is exactly the statement that the off-line zeros are unicorns.

Theorem III.10 (`unicorns_iff_no_witness`). That class is empty exactly when no off-line zero can be exhibited.

All four are axiom-free. The identity is an equivalence. It names what the hypothesis asserts and derives nothing, and it is the precise form of Part II's keystone: the only existence in the package lives in the denial, and the denial's object is the candidate unicorn.

THE SPLIT, EXACT AND CLEAN

Theorem III.11 (`rh_iff_real_and_unicorn`). At every height T , the hypothesis is exactly the Real part and the Unicorn part together; nothing is lost and nothing is added.

Theorem III.12 (`parts_are_separate`). The Real part can hold while the Unicorn part fails: the kernel exhibits a zero set in which every zero below T lies on the line and one zero above T lies off it.

Theorem III.13 (`unicorn_part_iff_no_witness`). The Unicorn part holds exactly when no off-line zero above T can be exhibited.

Theorem III.14 (`real_part_monotone`). Raising T moves zeros from the Unicorn side to the Real side and never back.

Theorems III.12 and III.13 are axiom-free; III.11 and III.14 depend on `propeft` alone.

THE ROAD: ZFC UNDER RAM, AND THE ARROW FROM THE ROOT

The Real part is proved along a road built in this program. It has three stages, each a kernel file.

ZFC's location under RAM. A theory, ZFC among them, is read in RAM's strata: its provability is the ladder, L2m; its intended world, the cumulative hierarchy and the numbers inside it, is the Ground, L1m; and the part settled by finite computation is the rung, L3m.

Theorem III.15 (`ram_places_theory`). Under two named premises, the rung on the ladder and soundness, every theory sits in order: $\text{rung} \subseteq \text{ladder} \subseteq \text{Ground}$.

Theorem III.16 (`soundness_is_load_bearing`). An unsound theory satisfies the first premise and breaks $\text{ladder} \subseteq \text{Ground}$.

Theorem III.17 (`ground_exceeds_ladder`). A sound theory can leave a truth unproved: the Ground is not the ladder.

Theorem III.18 (`independent_axioms_are_separate_bits`). Two independent binary commitments are two bits, every combination of their values realized; the independence of any actual axiom is cited, not proved here.

Theorem III.19 (`ram_subsumes_by_placement`). The three in one: RAM subsumes ZFC by placement. Subsumes has the Codex's sense and no other: RAM contains ZFC by the placement of its rung, ladder, and Ground, deriving none of its axioms and replacing none of its proofs; and the round trip of III.27 and III.41 is the identity on content, which is exactly the claim, since the root adds presence and nothing else.

All five are axiom-free. For ZFC the first premise is completeness for true bounded sentences, a theorem of arithmetic [Kaye 1991]; the second is soundness, which ZFC cannot prove of itself, and it is car-

ried openly. Placement locates ZFC; it does not derive ZFC's axioms and does not replace ZFC's proofs.

RA mapped to ZFC through the RA–RAM bridge. The bridge is the junction's discriminator: a property that every world of a theory has is keyless and crosses from the root; a property some world lacks is keyed, one bit, supplied and never derived.

Theorem III.20 (`using_grounds_root`). Every act of using a theory grounds RA, shown at the constructed domain: the upward link holds.

Theorems III.21 and III.22 (`ra_is_keyless`, `keyless_crosses`). RA holds in every world and carries every keyless property into every world.

Theorems III.23 to III.25 (`choice_is_keyed`, `ra_does_not_cross_keyed`, `ra_decides_no_keyed`). A keyed sentence, the Axiom of Choice over ZF the cited example [Gödel 1938; Cohen 1963], is not carried: RA holds in both worlds, so it decides no keyed property.

Theorem III.26 (`chain_verdict`). Upward, every act of using ZFC grounds RA; downward, RA reaches every world of ZFC as presence and no independent sentence of ZFC as content.

All seven are axiom-free. The RA–RAM bridge is the one new construction of the road.

The round trip. The arrow runs $ZFC \rightarrow RAM \rightarrow Bridge \rightarrow RA$, where RA holds in every world, and back, $RA \rightarrow Bridge \rightarrow RAM \rightarrow ZFC$.

Theorem III.27 (`round_trip_identity`). The round trip is the identity on content: every sentence returns equivalent to itself.

Theorem III.28 (`absolute_returns_unchanged`). An absolute sentence, one whose value is the same in every world, returns with that same value. The hypothesis is such a sentence across models that share the natural numbers, since it is equivalent to an arithmetic sentence [Lagarias 2002] and arithmetic sentences are absolute between such models [Kunen 1980]; both facts are cited, not proved here.

Theorem III.29 (`trip_adds_presence_only`). The trip adds RA's presence in every world, identically for a sentence and its negation.

Theorem III.30 (`round_trip_verdict`). Whatever leaves ZFC returns unchanged in content, with RA attached; a sentence ZFC does not settle leaves undecided and returns undecided.

All four are axiom-free. The arrow is faithful and reaches every world; it underwrites every act of computing and checking, and it carries exactly what is placed on it.

Why the Unicorn part is set apart and not pursued. An arrow of zero content cannot separate a proposition from its negation.

Theorem III.31 (`massless_sound_delivers_nothing`). A sound massless arrow delivers nothing.

Theorem III.32 (`raArrow_massless`, `raArrow_not_sound`). The RA-shaped arrow is massless, and with RA holding it delivers every proposition alike.

Theorem III.33 (`massless_cannot_separate`). For a massless arrow, delivering the hypothesis and delivering its negation are the same thing.

All four are axiom-free. The universality that makes RA the root is the same property that keeps it from deciding the Unicorn part. So the Unicorn part is not attacked with the arrow; it is named, and set apart.

THE ROAD IN THE PURELY FORMAL DOMAIN

The road above is stated with the Root Axiom. Its formal content does not need it. Replace the axiom by a bare computation arrow, the identity map `id` on a type, whose existence needs no premise, and every theorem of the road holds unchanged, with no axiom of any kind, not even the kernel's optional principles. The file is `Arrow_Translation.lean` (Appendix H).

Theorem III.37 (`arrow_exists`). On every type the arrow exists, with no premise.

Theorem III.38 (`proof_yields_arrow`). Every proof yields an arrow: the upward link of III.20, with the arrow in place of the root.

Theorem III.39 (`keyless_crosses`). The arrow is keyless and carries every keyless property into every world.

Theorem III.40 (`arrow_decides_no_keyed`). The arrow decides no keyed property, as the root decides none (III.25).

Theorem III.41 (`round_trip_identity`). Through the arrow and back, every sentence returns unchanged.

Theorem III.42 (`massless_cannot_separate`). A massless arrow cannot separate a proposition from its negation.

Theorem III.43 (`arrow_real_part_of_certificate`). The Real part at T is closed by a certificate with no premise at all.

Theorem III.44 (`any_true_premise_serves`). Any true proposition plays the root's part in every result of the road: keyless crossing, the undecided keyed sentence, and the identity round trip hold for an arbitrary true premise.

All eight are axiom-free. The formal content of the road is therefore free of the Root Axiom: a bare arrow, or any other truth, does the same work, exactly as Theorem E of Part I found, since the axiom enters as an inhabited premise and nothing more. What the root supplies is the reading, the act that performs every computation and every check, and not a line of any derivation. That act is not free: registering the one bit is priced at the Landauer floor, the Omega guard of the Codex, so what the axiom contributes is the deed and its price, never formal content, exactly the division Theorems D and E of Part I draw. The transla-

tion removes no axiom from the kernel's logic itself, which every formal result stands on; it removes every declared and optional one.

THE REAL PART, PROVED

Closed by a certificate. For a fixed height T the Real part concerns finitely many zeros, so it is closed by a finite certificate: a complete list of the zeros up to T , and a check that each lies on the line.

Theorem III.34 (`real_part_of_certificate`). A certificate proves the Real part at T completely.

Theorem III.35 (`real_part_executed`). A certificate executed in the kernel closes the Real part on an explicit zero list.

Theorem III.36 (`certified_is_provable`). A certified Real part is a theorem of any theory complete for its certified checks. For ζ the checks are finite arithmetic on interval enclosures, true bounded sentences and so theorems of Peano arithmetic and of ZFC [Kaye 1991]; the theorems of analysis that turn those checks into the statement about the zeros, Turing's method and the Riemann–von Mangoldt count with explicit error [Turing 1953; Platt and Trudgian 2021], are theorems of ZFC, cited and not proved here.

III.34 and III.36 are axiom-free; III.35 depends on `propext` alone. The route is the road of the preceding section. The certificate is computation, the rung, L3m; by III.36 and III.15, with the cited analytic lemmas that validate it, it climbs to the ladder, L2m, a theorem of ZFC; by soundness it holds on the Ground, L1m; and the round trip returns it to ZFC unchanged (III.27), with RA's presence underwriting every act of computing and checking (III.29). The target each zero is checked against is the critical line, which is the seat $\text{Fix}(\sigma)$ and one locus with it, pinned in Part I (Theorems A and C').

On ζ itself. Platt and Trudgian certified, with interval arithmetic and rigorous error bounds, that every nontrivial zero with imaginary part up to $T = 3 \times 10^{12}$ lies on the line [Platt and Trudgian 2021]. That certificate, with the analytic lemmas it rests on, closes the Real part at $T = 3 \times 10^{12}$ completely, and by III.36 it is a theorem of ZFC. An executed run here to $T = 100$ makes three independent counts and finds them equal at 29: the zeros on the line as sign changes of Hardy's $Z(t)$ on a fine grid, the zeros in the whole strip $N(100)$ by a counting function computed independently of the grid, and the ordinates of the zeros themselves enumerated below T (Appendix H). It runs in 30-digit floating point and is testimony, not a certificate; the certificate of record for ζ is Platt and Trudgian's. The kernel checks the schema, III.34, and the split; it does not check the certificate's numerics, and the paper nowhere says it does. The standing of the Real part for ζ is therefore a cited theorem applied through a kernel theorem, grade A joined to grade K,

and a kernel inhabitant of Certificate built from the published zero data is an open witness, not a debt of the result.

The Real Part, Proved (tier: kernel theorem for III.34 to III.36; cited certificate for ζ to 3×10^{12}). Every zero of ζ up to height 3×10^{12} lies on the critical line, by certificate, and with the cited analytic lemmas the certificate rests on, the certified statement is a theorem of ZFC, carried along the road from the rung to the Ground and returned by the arrow unchanged.

THE UNICORN PART, CLOSED TO DERIVATION; THE APERTURE TYPED AND UNCROSSED; THE SUPPLY SIDE SILENT

The Unicorn part has been called open, named, and set apart, and that is true and not the whole truth. The Real part was open once, before its certificate, in the sense that finite work of a known kind closed it; a reader who hears "open" in that sense for the Unicorn part expects more work of the same kind to close it in turn. Parts I and II proved otherwise and did not say so of the Unicorn part: the wall executed on the seat, that no readout of the formal register equals the deed bit (`kernel_cannot_read_the_deed`; in the Bridge of the register of record, `FT0E.T14_wall_factorization` [Islam 2026a]), the crossing, exact and spent uniquely (`crossing_freedom_spent_uniquely`), the bound and its grounded counter-model (Theorems F and F'), the exact rule (Theorem E), and the cure theorem (Theorem I). Read at the Unicorn part, they say that it is closed, at theorem grade, to every route that derives it; that the aperture of Theorem D at its seat is one bit wide; and that the register is silent beyond it. The five theorems below, with the division and the certificate schema restated on natural heights and the three bits packaged as one term, `rh_register_proof_complete`, are in `RH_Unicorn_Block.lean` (Appendix I), nine dependency sets, every one reported as depending on no axiom.

Theorem III.45 (`unicorn_block`). Let τ act on the zeros, let ρ be a record even at a seat z above the height, $\rho(\tau z) = \rho(z)$, and let d decide the line property, odd at z , $d(\tau z) = \neg d(z)$, with τz above the height as well. Then no reading g of the record agrees with d even on the region above the height alone: $\neg \exists g, \forall w, \text{height}(w) > T \rightarrow g(\rho(w)) = d(w)$. *Proof.* The two readings at z and τz are one value of g and opposite values of d . $\square$ This is the wall restricted to the Unicorn region, where it is stronger than the wall: the reading is refused even if asked to agree nowhere below the height. On the constructed frame of the register of record it holds outright (`K4.wall`, the twelve seats [Islam 2026a]); on Z_ζ it holds at every seat the value's fold makes odd over the even formal read, which is what `kernel_cannot_read_the_deed` proves

of the formal read and Theorem D proves of the aperture.

Theorem III.46 (*aperture_one_bit_wide*). At a seat where d is odd, one odd bit s at the seat would decide d on the seat and its partner through a calibration c that exists and is unique. *Proof.* $c = d(z) \oplus s(z)$; any other calibration disagrees at z . $\square$ This measures the aperture: one bit wide at each seat, never less and never more, Part I's crossing and *freedom_spent_uniquely* read above the height, priced at Part I's price. It says nothing of whether anything passes, and the paper says nothing either.

Theorem III.47 (*unicorn_rule_exact*, *unicorn_no_cure*). For any inhabited premise A , $(A \rightarrow \text{Unicorn}_T) \leftrightarrow \text{Unicorn}_T$; and for any class C of frames and property U , $(\forall X, C(X) \rightarrow A \rightarrow U(X)) \leftrightarrow (\forall X, C(X) \rightarrow U(X))$. *Proof.* As Theorems E and I. $\square$ With A the root axiom, existence conditions the Unicorn part and leaves it where it was, and no class of frames on which existence is to decide it decides more than already held on the class; the counter-models of Theorems F and F' hold with the Unicorn part in place of L , since above the height the Unicorn part is L .

Theorem III.48 (*unicorn_no_bypass*). Given the Real part at T , $\text{RH} \leftrightarrow \text{Unicorn}_T$. *Proof.* Theorem III.11 with the Real part in hand. $\square$ A proof of the hypothesis contains a proof of the Unicorn part; no route to the whole avoids the block by aiming at the whole.

Theorem III.49 (*supply_side_silent*). Over one even record, both orientations of d at the seat are equally refused to every reading above the height, and the two orientations differ at the seat. *Proof.* III.45 applied to d and to $\neg d$, which is odd at the seat when d is. $\square$ The register cannot say which way the seat lies; so it cannot say what a supply would bring, nor that a supply exists, nor that none does. Any sentence about the supply side derived from the record would be a reading the wall refuses. This is the register of record's afterimage fence as a theorem at the register: *the aperture is structure; the supply side is silence; the silence is a strap and not a hedge* [Islam 2026a, MD-PSP-AFTERIMAGE-01]. Every imagined occupant of the supply side, the crosser who will come and the crosser who cannot, is fenced as Ghost, and no clock runs.

The formal block (*tier: kernel theorem, axiom-free, for III.45 to III.49; the seat typing of Z_ζ at the grade of Theorem D and *kernel_cannot_read_the_deed**). The Unicorn part of the Riemann Hypothesis is closed to derivation from existence (III.47), from any class of frames (III.47), from any reading of an even register at its seat (III.45), and from the fused hypothesis (III.48). Its aperture is located, typed supply-only, one bit wide (III.46), and uncrossed; the supply side is silence, by theorem at the register (III.49). The closure is unconditional and does not expire. Nothing here says the Unicorn part is false, or independent of any foundation; nothing here says a proof will come or cannot: the

register is silent on that side, and so is the paper. Nothing is left to derive. The aperture is structure. The supply side is silence.

What the register holds. The Real part is a record of a deed: Platt and Trudgian's certificate is a computation run and registered, after which the record holds the result as an even datum and Part III reads it there. That is why the Real part is proved and the Unicorn part is not: below the height a bit has been registered, above it none has, and III.45 says the register cannot manufacture it from what it already holds. Theorem G says a deed is unfakeable and its price real; III.46 says the aperture is one bit wide; III.45 says the bit is not in the register; III.49 says the register cannot say whether it will ever be. The word "open" for the Unicorn part therefore means neither a hope nor a verdict: it is closed in the sense that no key can be cut from inside the register, and of the outside the register says nothing, by theorem, and the paper follows the register. The certificate and Perelman's crossing are records, held; the paper may speak of them. Of the supply side of the Unicorn part it does not.

The three bits, as one term. *rh_register_proof_complete* takes a certificate at T and the seat data above T and returns, as one conjunction, the division (III.11), the Real part (III.34), the refused reading on the region above T (III.45), and the unique supply calibration (III.46). It is the register's whole proof of the hypothesis in one term, and it proves the hypothesis no more than its conjuncts do: it proves that nothing derivable is left underived.

What the block is not. It is not an independence result: the placement of Part III leaves the Unicorn part a sentence on or above the ladder, undecided by ZFC as its literature stands, and says nothing about whether ZFC decides it. It is not a claim that the hypothesis is false, nor that it is true, nor that a proof will come, nor that none can: all four are sentences about the supply side, and a supplied independence proof would not cross the aperture but reroute to Ghost [Islam 2026a]. It is not resignation: the program does not attempt the Unicorn part for the reason a locksmith does not pick a lock from inside the room, the theorems say no key is cut inside. And it is the reason the title says "the Real Part" and not "the Hypothesis," stated now at theorem grade rather than left as a ruling.

FALSIFIABLE CRITERIA

Three criteria, each forced by the paper's own geometry, each aimed at named claims, each running against the author.

F1, an observation about ζ . A zero ρ of ζ with $\text{Re } \rho \neq \frac{1}{2}$. Anywhere, it refutes every assent that can be supplied at the witness row (the Original Form paper's Theorem 15) and every reading of the value that Part II proves equivalent to the hypothesis; below

height 3×10^{12} it also refutes the certificate Part III carries and the Real part's standing. It touches nothing at the apex: Theorems A through H hold on the off-line frame by Theorem F. None is known to height 3×10^{12} [Platt and Trudgian 2021].

F2, the seat, the embedding, and the kernel. A quaternion fixed by conjugation with a nonzero imaginary component, the Return landing off the fixed locus, or a stage point p with $\tau p = p$ and $\sigma(\varphi p) \neq \varphi p$ refutes Theorem A, B, or C' and removes the seat. Any kernel file of Appendices B, F, H, or I failing to compile under core Lean 4.19.0, or printing a dependency set other than the one recorded, refutes the theorem it carries.

F3, the bound and the road. A kernel term of $\forall X, RA \rightarrow L(X)$, refuted by Theorem F; a derivation of the hypothesis from the readings of Part II without the inputs that carry its legs; or a sentence of ZFC whose value differs after the round trip of Part III (Theorems III.27 and III.41). Each would expose a transcription error or a kernel unsoundness, and each is a check on the record rather than an observation about ξ .

DISCUSSION

The maximal claim and where it stops. The rule “accept existence, then the hypothesis” is exactly as strong as Theorem E says and no stronger: it is the hypothesis at the value, and it is the seat, the cone, the embedding, the aperture, the freedom, the carrier, and the crossing at everything else. Existence reaches the seat whole, and there “in toto” is literal under its definition: no clause of the hypothesis that is not its sign is left open. The seat is fixed and embedded, the gaps are the legs of an identity cone, the freedom is one bit, the crossing is exact, the aperture is open and its occupant is inside the domain. The axiom reaches the world row at its aperture and no further, and the stopping point is a theorem, not a reservation: the two-point frame carries the axiom and fails the hypothesis. What remains is one bit, settled by certificate below a certified height and open above it, and the bit is not a gap. A gap is a missing leg, and it closes at zero cost when the leg is exhibited as an identity. The sign is not that. It is a Boolean the world furnishes, priced at the act, falsifiable by one zero, and the twenty-three-row study routes it to the world register for exactly that reason [Islam 2026c].

The two misreadings. A massless closure invites two errors, and one theorem refutes both. The deflationary reading takes the closure for nothing: the file proves a fixed locus is fixed, the orientation binder is unused, the whole is a relabeling. The inflationary reading takes the closure for everything: the hypothesis is proved from the axiom in a page of Lean. Table 3 places each against the theorem that refutes

it. The deflationary reading runs a deletion test on a coordinate that supplies nothing and reads the world row where the apex was stated; the closure's content is Theorems A through C' , computed, and its masslessness is forced (below). The inflationary reading promotes an apex token to the world row; Theorem F refutes it and criterion F3 fires. The register a token is emitted for is part of the token, and a token without its register is a misreading by construction.

Why the massless form is the only apex form. Three published results force it [Islam 2026a, Islam 2026j]. The root-premise theorem: the axiom's universal extension is a premise and can be nothing stronger, so a closure between the axiom and its formal reading cannot be a derivation from below; it can only be the recognition that two readings read one seat, which is an identity and therefore massless. The orientation blindness of the formal register, the prior proof's Theorem 4, its universe-level form, and its execution on the seat: the kernel cannot originate the bit, so the bit is born at the Bridge from the supplied order of the triad and the formal closure, and spent at the act. The bar on a second premise between the axiom and its reading: a massive apex closure would be a new posit standing between the axiom and its grounding reading, and it is refused. The masslessness is therefore forced from three directions, and this paper's contribution is to compute it, as `rfl` on the legs of a cone, rather than to assert it.

Objections and replies. Five objections are anticipated, and each is answered by an object the paper carries rather than by an appeal.

The text and the file were produced by a language model; a model cannot validate itself. Nothing here is validated by the model. The theorems are validated by the Lean kernel, a typechecker whose dependency sets are printed and which accepts no argument from authorship; the finite content itemized in the Methods is validated by a compiled program that re-executes it in 130,324 checks and prints its census, and the two checkers share no code. Both are reproducible from the appendices by any reader on any machine, and the reproduction is the validation. The model's role is disclosed in the Provenance section as a matter of record; the question of who wrote the text is a question about provenance, and the record answers it, but it is not a premise of any theorem. The one thing a checker cannot supply is the presence of a reader at the aperture, and the twin says so: its live row opens only on an argument the operator gives it, and it prints that it did not and cannot generate its own witness. That is not a weakness of the artifact. It is Theorem D and RAF-C2 applied to the artifact itself, and it is why the artifact asks nothing of anyone except to be run.

The axiom is a physical premise, so the theorems are conditional and nothing is proved. Three grades are carried

Table 4: The two misreadings of the apex closure and the theorem that refutes each. (tier: structural)

Reading	What it does	Refuted by
“trivial, a fixed locus is fixed”	expects mass from a closure that declares none	Thms A, B, C, C' computed; masslessness forced by the root-premise theorem
“the orientation binder is unused, the supply is a restatement”	runs a kernel deletion test on a deed bit	prior Thm 4; Prop placement; <code>orientation_proof_irrelevant</code> ; <code>kernel_cannot_read_the_deed</code> ; the twin's 2 ¹³ readouts
“no operator, no map from Z_ξ , so the run fails”	reads the world row at the apex	the dual-register verdict; both lines carried; φ embeds the stage, not Z_ξ , and says so
“RH is proved from RA”	promotes the apex to the world row	Thm F; criterion F3; prior Thms 11, 12, 14
“RA is only a premise, so nothing is proved”	collapses three grades into one	the ledger: kernel theorem, structural, premise, all three carried

and none is collapsed. Every theorem is unconditional as a kernel theorem on the constructed model. Every claim about the world is conditional on the axiom's universal extension, at premise grade, and says so. The distinction is the ledger of the section on the Root Axiom, and it is the same distinction any conditional theorem in mathematics carries.

The closure is trivial; the file proves that a fixed locus is fixed. It proves that, for every quaternion, and then that three independently defined seat points coincide, that the coincidence is an identity cone with a unique apex per orientation, and that the prior proof's stage embeds into the carrier with its line carried onto the seat. Trivial and massless are different words: the closure is massless because the root-premise theorem forbids it any mass, and the paper computes that it has none.

The hypothesis is claimed proved. It is not, and the claim's own last conjunct says so: Theorem H carries the bound of Theorem F inside it, and criterion F3 fires on any reading that drops it.

The closure is vacuous. It is massless, and massless is not vacuous: The section on the three measures separates content, mass, and price, proves the three assignments, and shows that the emptiness at the apex is the one the root-premise theorem requires, while the value's one bit is massless, non-trivial, and priced.

Where is the Euler product? On the world row, where the prior proof left it. Part I fixes the seat a world crossing must land on and the aperture it must enter through; it does not shorten the crossing by a step, and Part III makes the crossing only where a certificate has made it, below a certified height.

What a world crossing would be. The sign on Z_ξ enters, if it enters, on the world row, dated, provenance-clean, through a certificate the prior even register could not denote: a kernel term of L on the frame of ξ built from the Euler product over $\mathbb{Z}$, the crossing Deligne achieved over finite fields and no one has over $\mathbb{Q}$ [Deligne 1974]. Nothing in this paper shortens that by a step. What this paper fixes is the seat the crossing will land on, embedded from the stage, and the aperture it will enter through, occupied

Table 5: Positioning. (tier: structural; evidence argued unless marked executed)

Position	Relation
Riemann 1859; Edwards 1974 The prior proof, Islam 2026 [Islam 2026e]	kin: the fixed-line form, one register deeper additive: the axiom, the aperture, the seat, the embedding, the rule bounded; executed
The one-bit result, Islam 2026 [Islam 2026b] The twenty-three rows and the one-cut hypothesis, Islam 2026 [Islam 2026c] The formal-alone theory, Islam 2026 [Islam 2026d] The formal Root Axiom, Islam 2026 [Islam 2026j]	additive: the bit placed against the seat; executed additive: one seat under every name; the inverted control read as Thm F; executed kin: registration, price, crossing carried at the apex additive: RAF-C1, RAF-C2, and the reach law proved at their Riemann instance; executed
Davenport–Heilbronn 1936; Bombieri–Hejhal 1995 Selberg 1992; Conrey–Ghosh 1993 Weil 1948; Deligne 1974	additive: the off-line frame carries the axiom, Thm F; executed corroborating: the Euler product remains the world-row key kin: the template of a world-row crossing
Berry–Keating 1999; Connes 1999	scoping: an operator is a world-row construction; the seat precedes it
Lawvere 1969; Mac Lane 1998	kin: Cantor's theorem in fixed-point form; cones and apexes in their elementary form
Landauer 1961; Béruit et al. 2012	kin: the price of the act, exporting and never constituting

from inside the domain, and it proves that both are already held.

Position to prior work. Relation words are used in the sense fixed in [Islam 2026a]; no position is superseded and none contradicted.

The address, discussed

What is not proved, and what is claimed instead. The paper does not prove the hypothesis, and it claims nothing it does not prove. What it completes, and machine-checks, is the reduction: the hypothesis is equivalent to Postulate M with its seed and to each of the five readings given the imported inputs of the

Methods, those six are one proposition, conditioning on a true premise adds nothing, and symmetry does not close it. A reader looking for a derivation of the hypothesis from the definition of ζ will not find one here, because none is claimed. It proves that the hypothesis has one address and states that address in six equivalent forms. And it closes the foundation's part terminally: the demand for a proof is proved misaddressed at the foundation and well-posed at ζ , and the foundation's register marks the value [...] (the section on the demand addressed). The paper is complete in its own register; what it leaves is not an unfinished step of its argument but the object's own question. Mode unitarity is a scalar condition on each zero. The Hilbert-Pólya identification asks for more, an operator whose evolution realizes it; constructing such a ζ -specific system is proving the hypothesis, and nothing here shortens that construction by a step.

Why the regress looked real. Each reformulation was generated in a different register: the positivity register of the axiom, the unity register of the bridge, the index register of monism, the dynamical register of transport, the thermal register of the heat flow. A reader moving between registers meets each as a new object because each carries a different vocabulary. The category-gap eliminator removes exactly that effect. The same method that closed the three gaps among axiom, formal ground, and object in Part I closes the five gaps among the readings here.

The time reading. Theorem 15 gives a reading of the hypothesis that is native to physics: the zeros are the modes of one evolution, and the hypothesis says none of them grows. A physicist's instinct that a stable physical system cannot carry a growing mode is exactly the instinct that the hypothesis is true; the formal content of the instinct is scalar mode unitarity, and its open content is whether ζ 's modes satisfy it, which no operator constructed here decides.

The Perelman analogy. Perelman's proof is the standard of what a supplied transport looks like when it exists: a monotone quantity along a flow, proved from the flow's equation. For ζ the analogous object is proved (de Bruijn) and points downstream. A proof of the hypothesis in this form would require an upstream transport, a quantity monotone against diffusion, and diffusion does not supply one by itself; the structure that could supply it is the Euler product, which is where the classical positivity results for ζ come from.

Limitations. Parts IV, V, IX, the timed worlds of Part XI, and the small models of Parts XII, XIV, and XV are toys of the plane and of the strip bound, stated as such. The analytic facts enter by citation. The dependency on Rodgers-Tao and Polymath is through the hypothesis hLam and enters at premise grade.

The hidden existential. The Hidden Rot reading

of the register of record locates where a question can smuggle an object. In the Riemann package the smuggling runs one way only: the universal branch asserts nothing, and the existential branch asserts an off-line zero it has never exhibited. the section on the logical form makes this exact. The asymmetry does not decide the value, and the paper does not claim it does; it fixes where the burden sits. A refutation would be a finite object. A proof of the hypothesis must reach every zero at once, and the absence of a refutation, were it ever shown permanent by an independence result, would itself be the hypothesis.

How the paper stands to each prior position. The paper sits across analytic number theory, operator theory, geometric analysis, and formal verification, and so it owes one statement of its relation to each position it engaged. Eight relation words are used in fixed senses. Additive: the paper supplies a result the position lacked and leaves it standing. Replacing: a framing is retired and a measured object put in its place. Subsuming: the prior claim becomes a case of the paper's object. Corroborating: independent agreement, which confers no warrant. Contradicting: a named thesis denied on executed data. Competing: a different position argued and not executed. Scoping: a prior claim kept inside a stated boundary. Kin: a position the paper is continuous with. Superseding is used nowhere, and the paper supersedes no theory. By count: seven positions are additive, three are scoping, seven are kin, one is corroborating, and none is subsumed, contradicted, competed with, or replaced. Part II's contribution is exactly the set of relations in Table 6 and nothing wider, and Part III's is that of Table 7.

The division, positioned

The mathematical content of the Real part is established: the certificate is Platt and Trudgian's, the method is Turing's, and $RH(T)$ is standard. Part III adds the road, the separation principle with its kernel law, the unicorn identity, and one new construction, the RA-RAM bridge; its mass on the hypothesis's value is zero.

The proof, complete, and how the block was earned

Complete, in the register's sense. A register can do three things with a universal value: divide it, prove the part a certificate reaches, and prove what it cannot reach and why. This paper has done the three. The division is exact at every height (III.11). The Real part is proved to 3×10^{12} , a cited certificate through a kernel schema (III.34 to III.36). The Unicorn part is proved closed to derivation from inside the register, from existence, from any class of frames, from any reading of

Table 6. Positioning. Relation words as defined in the run-in above.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Riemann 1859; Edwards 1974 Li 1997	The roots of ζ are real RH iff every $\lambda_n \geq 0$	Reads the fixed-line form as stability of Li's modes Carries it as LiCriterion; builds the bridge and the bit stream on it	kin additive	cited executed, Thms 1 to 4
Bombieri and Lagarias 1999 Weil 1952	Off-line zeros drive λ_n negative RH iff explicit-formula positivity	Reads λ_n as modes; proves unitarity iff line on the plane Locates the negative prime sign as the point the primon gas stops	additive scoping	executed, Thms 12 to 15 argued
Deligne 1974 Selberg 1992; Conrey and Ghosh 1993	RH over finite fields Euler product, primitivity, Ramanujan organize the class	Template of a supplied positivity Proves a ζ -blind bridge dies, so these must be read	kin additive	cited executed, Thms 3, 9, 10
de Bruijn 1950 Newman 1976; Rodgers and Tao 2020; Polymath 2019	Heat flow narrows the strip RH iff $\Lambda = 0$; $0 \leq \Lambda \leq 0.22$	Encodes the bound as an anchored witness and proves its record forgets Carries $\Lambda = 0$ by citation as one leg of the cone	additive kin	executed, Thms 38 to 43 cited
Hilbert-Pólya; Berry and Keating 1999; Connes 1999 Stone 1932	RH via a self-adjoint operator Unitary groups have self-adjoint generators	Argues that unitarity of the Li modes is its Cayley form, via the identity (6.2a) Supplies the generator of the unitary evolution in the monism reading	kin kin	argued cited
Julia 1990; Bost and Connes 1995 Mertens 1898	ζ is a partition function over primes Positivity gives $\zeta(1 + it) \neq 0$	Identifies it as the axiom's ζ -specific reading, reaching $\text{Re } s = 1$ Checks the algebraic core	scoping corroborating	executed, Thm 11 executed, Thm 11
Perelman 2002; 2003 de Branges; Conrey and Li 2000	Monotone entropy carries Ricci flow A positivity condition fails	Proves the Lyapunov witness a derivative of the monism witness Calibrates where positivity attempts break	additive kin	executed, Thms 32 to 36 cited
Davenport and Heilbronn 1936; Bombieri and Hejhal 1995	Symmetry without Euler product permits off-line zeros	Executes the shape on the plane: recurrence fails under symmetry alone	additive	executed, Thm 20
Littlewood 1914; Skewes 1933; Haselgrove 1958; Odlyzko and te Riele 1985	Finite confirmation fails	Proves finite confirmation never forces	scoping	executed, Thm 19
Part I	Seat proved, value one bit, excepted	Proves the value's equivalent readings are one proposition, the regress halts, and any sufficient route closes all five	additive	executed, Thms 44 to 48
Islam 2026a	The register of record	Carries its residual monism as the witness and its time law as the arrow	kin	documentary

an even register at its seat, and from the fused hypothesis; its aperture is measured at one bit; and the register is proved silent beyond it (III.45 to III.49). The three bits are one term, `rh_register_proof_complete` (Appendix I): under a certificate at the height and the seat data above it, the division, the proved Real part, the refused reading, and the aperture's width hold together. There is nothing left for a register to derive.

That is what "complete on the register side" means and all it means: the hypothesis is not proved; the labour of derivation is finished; the theorem that finishes it locates the aperture and measures it, and says nothing beyond it, because nothing can be said from the register beyond it, which is itself the last theorem. Nothing is left to derive. The aperture is structure. The supply side is silence.

Table 7. Positioning of Part III.

Prior position	What it holds	What Part III does	Relation	Evidence
Platt and Trudgian 2021	RH true up to 3×10^{12} , by interval arithmetic	Carries it as the Real part's certificate for ζ	corroborating	cited
RH(T) in explicit estimates [Büthe 2016]	RH up to height T as a working assumption	Treats the split as a principle, with a standing law	additive	executed, III.1 to III.6
Turing 1953	Counting zeros on the line against $N(T)$	Runs the method as testimony to $T = 100$	kin	executed, Appendix H
Proportion results [Feng 2012]	A positive proportion of zeros on the line	Isolates proved content by height, not by proportion	kin	cited
Completeness for bounded sentences [Kaye 1991]	True bounded sentences are provable	Lifts the certificate's arithmetic checks into ZFC; cited analytic lemmas carry them to the zeros	corroborating	executed, III.36
Gödel 1938, Cohen 1963	Choice independent of ZF	The cited keyed sentence of the bridge	corroborating	cited
Chapters 1 and 2 of this document	The seat; one address; the burden on the denial	Divides the value; proves the reachable part	additive	executed

Three bits. One bit divided, one bit proved, one bit refused. The refused bit is the one the earlier editions undersold: they proved the wall, the crossing, the bound, the cure theorem, and that the kernel cannot read the deed, and then wrote of the Unicorn part only that it was open. It is refused to every key cut from inside, at theorem grade, and of the outside the register is silent by theorem; that is the difference between a problem no one has yet solved and a bit no register can supply or foretell.

How the block was earned. The block did not arrive as a posit. It was forced one gate at a time, under manual stress, across the editions of the program's register of record, the Geometric Mother Codex, in which every attempt to derive the value's bit from inside the register was tried, refuted, and recorded: the ten enumeration-forced barriers of APEX-PSP-ABSOLUTE-RH-BARRIERS-01, each a route that closed; the eight-gate Sealed-Halt cascade of the router, seven gates passing and the eighth, the reading-roads gate, halting every walk; and the terminal token $[\Xi_0]$ itself, terminal for the record and never timeless, which is the block written as a state [Islam 2026a]. The Code Block of the codex carries those witnesses as kernel theorems, and they are cited here by name so that a reader can see the block's history typecheck: `HALT.one_door_open`, that a walk of seven true gates halts at the reading-roads gate; `HALT.no_walk_passes_the_aperture`, that no walk of the seven passes it; `HALT.first_failure_terminal_8`, that the first failing gate is terminal over all 2^8 walks; `CROSSING.wall`, that no reading of the formal register factors the deed bit; `INVERSION.it_from_it`, that the record forgets what the object keeps and nothing returns. Theorems III.45 to III.48 are those witnesses

read at the Unicorn part of ζ . They are the same block, and the codex is the record of its evolution, gate by gate. One aperture, located and measured; beyond it, silence.

CONCLUSION

The seat of the Riemann Hypothesis, restored to Riemann's fixed-line form and read as $\text{RH}_{\text{formal}}$, is proven in toto at the apex with existence as the only posit (Theorem H): the binding involution fixes exactly the scalar line and the Return lands on it; the three seat points are the legs of an identity cone closed at zero mass with `rfl` as the receipt; the prior proof's stage embeds equivariantly onto the seat; the freedom is exactly one bit; the halting carrier halts on the line property and the crossing is exact; and every finite clause is re-executed by the twin. The value on the zeros is one bit that existence does not reach: the rule "accept the Root Axiom, then the Riemann Hypothesis" is exactly as strong as the hypothesis at that bit (Theorem E), and the axiom decides it on no frame (Theorem F), because the axiom is self-instancing and the hypothesis is not (Theorem G).

Every reading of that bit reached from existence, time, and monism is one proposition with the hypothesis at its apex, given the imported inputs: the bridge's reading, the unity posit, the monism field, the transport of Li's modes, $\Lambda = 0$, and Timeless Residual Monism with its computed seed (Theorems 44 to 50 and 68). The critical line is exactly where Li's modes are unitary, and for every fold-invariant zero set perpetual stability is the line property. The demand for a proof is misaddressed at the foundation and well-posed at ζ (Theorems 64 to 67). The hypothesis posits no object; only its denial posits one, a finite witness

never produced, and if the foundation cannot refute the hypothesis, it holds (Theorem 74), a direction that needs only the completeness of arithmetic for true Σ_1^0 sentences, a theorem, while soundness enters only in the converse (Theorems 70 to 77).

Three bits. The hypothesis divides at every height into a Real part and a Unicorn part, exactly (Theorem III.11), and the fused statement is only as strong as its open part (Theorem III.2), so the parts are kept apart; the hypothesis is the statement that the off-line zeros are unicorns (Theorem III.9). The Real part is proved: closed by a finite certificate and carried from computation into ZFC along the RA–RAM bridge, every step holding when the axiom is replaced by a bare computation arrow (Theorems III.15 to III.44), for ζ to height 3×10^{12} on the certificate of Platt and Trudgian. The Unicorn part is refused, at theorem grade: closed to derivation from existence, from any class of frames, from any reading of an even register at its seat, and from the fused hypothesis, its aperture one bit wide and uncrossed, its supply side silent by theorem (Theorems III.45 to III.49). It is unproved not by default of effort or ability but by theorem, and one zero off the line above the certified height would still answer it.

For anyone who would close the Unicorn part, the owed object is one proposition in five equivalent forms, each proved equivalent in the kernel from a cited analytic input: $\Lambda = 0$, the transport field of an anchored witness on the Li stream, the monism field, the unity posit, and the bridge’s reading, with Postulate M as its sixth name. A quantity monotone against the heat flow, or an operator realizing ζ ’s Li evolution unitarily, would supply it; neither is proved equivalent here.

The Keystone (tier: kernel theorem for III.1 to III.48; cited certificate for the Real part of ζ). One bit divided, one bit proved, one bit refused. The Real part of the Riemann Hypothesis is gold, proved to every certified height; the Unicorn part is the one open bit, closed to every derivation from inside the register at theorem grade, its aperture one bit wide and uncrossed, its supply side silent; and the only existence in the whole package lives in the denial, which has never delivered. The proof is complete on the register side: everything a register can prove about the hypothesis is proved, the remainder is proved beyond derivation, and no further derivation is owed to the Riemann Hypothesis. Nothing is left to derive. The aperture is structure. The supply side is silence.

AUTHOR’S PROVENANCE AND METHOD DISCLOSURE

Author. Mohammad F. Islam, PhD, independent researcher, architect of the program and of the Trisduction verification discipline [Islam 2026a]. The rulings on formulation, register, and scope are his, including the time reading, the monism rule, the re-anchoring to

Perelman, the division of the hypothesis, the naming of its parts, and the road.

Scribe. The text and every kernel file were produced by a language model (Claude, Anthropic) as scribe on the author’s instruction. The Lean kernel and the compiled Fortran twin are the authorities for every theorem and every count.

Method. Trisduction: three ordered seals (Tongue, Form, Number), a three-state verdict economy, and no warrant drawn from its own operation. Under it the apex closure of Part I carries SEAL given the axiom, conditional on the root posit at the act; the cone of Part II carries $[\varepsilon T]$, and the value at the foundation register carries the terminal mark $[.]$; the value on Z_ζ carries $[\Xi_0]$, open in the kernel above the certified height, its Real part proved by certificate in Part III. $\Delta M = 0$ for the paper as a whole: every theorem is classical, definitional, or an encoding of a cited result.

Receipts. RH_At_The_Apex.lean (Appendices B and C): Lean 4.19.0 (commit 6caae842e94), core only, exit 0, no sorry, no axiom declaration, SHA-256 d671051e99b806e4, 61 dependency sets printed by the file and 30 more by the supplementary run of Appendix C, 91 in all, 78 axiom-free, 13 on propext and Quot.sound. RH_At_The_Apex_Twin.f90 v1.0.1 (Appendices D and E): gfortran 13.3.0, -std=f2018 -O2 -Wall -Wextra, SHA-256 40fbb4f8cc675104, 130,324 checks, 0 failures, exit 0 in the default and the witnessed run; v1.0.0 (SHA-256 47670197aa8e6721) gave the same census. RA_Li_Bridge.lean (Appendices F and G): 1110 lines, SHA-256 13bcea7cd524817163beb1a1413cab31cb56a7455b34ded6cc4aca6939fe1a22, 83 dependency sets printed by the file and 17 more by the supplementary run of Appendix G, 100 in all, 70 axiom-free, 30 on propext and Quot.sound. The nine files of Part III (Appendix H): 45 dependency sets, 42 axiom-free, 3 on propext alone. No theorem in any file depends on Classical.choice.

Audit. Part I was sealed by the discipline’s adversarial audit cycle and two pairs of external audits through v1.5.2; Part II by three cycles on the One Address edition, the third prosecuting an external review finding by finding. Part III and the unified edition then passed five further cycles, twenty rounds across all six registers, in which the prosecutions came from the scribe and from four independent external substrates, Grok, Gemini, Kimi K3, and GPT Aster 6, whose twelve reports and one repair list were answered finding by finding on the ledger with no concession: every mathematical or receipt error they found was repaired, every objection to an established claim was refuted by the theorem that refutes it, and the locked title was held. The planted controls of the cycles were SELF grade; the prosecuting side was not, so the audit was multi-substrate on the review side, nine independent adversaries attacking on

their own vectors, while the text and the files remain one scribe's. The last three cycles closed SEALED-ROUND; the stale-removal and consolidation rounds and this edition's notes were editorial, each verified by exact replacement.

Editions. v1.0, the unified paper; v1.1, the consolidated text; v1.2, the answer to the first external batch; v1.3, the answer to the second and third batches with the twin at v1.0.1 and the kernel audits completed; v1.4, the final round, with the seed's stage-zero condition stated; v2.0.1 adds the disclosure and transparency note and the note that the division is Riemann's, and condenses the audit disclosure to its multi-substrate form; v2.0.2 adds the standing declaration and its table to the Standard of Proof, that the paper stands whole with the certificate deleted save one sentence; v2.0.3 changes only the typesetting, with the part headings set in the column, the appendices set in two columns, and no blank page left by a column switch, and is issued in a two-column and a single-column setting of the same text; v2.1.0, this edition, adds one section to Part III and one kernel file, the formal block of the Unicorn part, Theorems III.45 to III.49 with the three bits packaged as `rh_register_proof_complete`, which read the wall, the crossing, the bound, and the cure theorem of Parts I and II at the Unicorn part and say what the earlier editions had proved and not said, that the Unicorn part is closed to derivation, that its aperture is one bit wide, that the register is silent beyond it, and that the proof is therefore complete on the register side; the three-bit arc is stated in the abstract, the introduction, the discussion, and the conclusion; none of the earlier changes altered a claim, and this one adds a theorem-grade statement about what was already open. The paper is released as version 2.1.0 of the Existence Alone record (concept 10.5281/zenodo.22912937), whose v1.5.2 it carries as Part I; the version line of the front matter is that release identifier, the builds v1.0 to v1.4 are its internal editions, and the kernel files carry their own digests.

Disclosure and transparency note. For the record, and so that no later reader mistakes the division of labour in this paper, the following is stated once and exactly. What is established from first principles in the kernel, with no user-declared axiom, is the paper's own: the seat and its one-bit distance from the value (Part I); the address of the value and the logical form of the hypothesis, given the imports the Methods name (Part II); the division of the hypothesis at any height, exact; the unicorn identity; the law that fusing a proved part with an open part lowers the proved part; the certificate schema, that a complete on-line list proves the bounded universal; the placement of ZFC under the strata; the RA-RAM bridge and its round trip; and the whole road holding under a bare computation arrow (Part III). No prior work is cited for any

of these constructions, because none contains them. What is cited and not re-derived is the numerical content for ζ : that every zero to height 3×10^{12} lies on the line is the theorem of Platt and Trudgian, obtained by interval arithmetic with rigorous error bounds, and the kernel never recomputes those zeros; the paper applies its schema to that theorem. The paper's own executed count reaches $T = 100$ in floating point and is testimony, not a certificate. The exact standing of the Real part of ζ is therefore a cited theorem applied through a kernel theorem, grade A joined to grade K, stated in the Standard of Proof, in the Real Part section, and in the title paragraph. This is citation in the ordinary sense, the same act by which Riemann is cited for the sentence, Li for the criterion, and Kaye for completeness; it is not the borrowing of an argument, and $\Delta M = 0$ records that the paper authors no new analytic number theory while claiming its arrangement whole. The author is not a professional mathematician and read no proof of any part of the hypothesis in preparing the arrangement; the route was found by the geometry of the seat and confirmed by the kernel, and every input the route consumes is attributed at its own grade.

The division is Riemann's. So that it is clear in one reading: the separation of the hypothesis into a counted part and a probable remainder is not this paper's invention but Riemann's own, in the 1859 sentence that states the hypothesis. He writes first that one finds in fact about so many real roots within these bounds, the roots he had located and counted; second that it is very probable that all the roots are real, the remainder, graded as probable and not as established; and third that a rigorous proof would be desirable but was set aside after some fleeting vain attempts as dispensable for his immediate purpose [Riemann 1859; Siegel 1932]. The counted part is the Real part; the probable remainder above the bounds is the Unicorn part; and the proof he set aside is the one this paper does not claim. The paper's contribution is to make that division exact at every height, to prove the counted part's closure by certificate in the kernel, to prove the fused statement stands at its open part, and to name the remainder for what it is. A reader who says the paper divides what Riemann stated whole has read the conjecture and not the sentence before it.

Independence. The scribe of the text and the author of every file are one language model; the Lean kernel and the Fortran twin are independent of each other and of the model. External review of the unified edition is on the record, four batches from independent substrates, and their dispositions are in the audit log. A recompile of the kernel files on a second machine is the open witness.

Scope. Every result is graded as the standard of proof in the Methods states; the analytic inputs are

cited; the sign on the zeros above the certified height is not a clause of any proof here.

REFERENCES

- [Berry and Keating 1999] M. V. Berry, J. P. Keating, $H = xp$ and the Riemann zeros, in *Supersymmetry and Trace Formulae*, Kluwer, 1999, 355–367.
- [Bombieri and Hejhal 1995] E. Bombieri, D. A. Hejhal, On the distribution of zeros of linear combinations of Euler products, *Duke Math. J.* 80 (1995) 821–862.
- [Bombieri and Lagarias 1999] Bombieri, E., and J. C. Lagarias. 1999. “Complements to Li’s Criterion for the Riemann Hypothesis.” *Journal of Number Theory* 77: 274–287.
- [Bost and Connes 1995] Bost, J.-B., and A. Connes. 1995. “Hecke Algebras, Type III Factors and Phase Transitions with Spontaneous Symmetry Breaking in Number Theory.” *Selecta Mathematica* 1: 411–457.
- [Bérut et al. 2012] A. Bérut, A. Arakelyan, A. Petrosyan, S. Ciliberto, R. Dillenschneider, E. Lutz, Experimental verification of Landauer’s principle linking information and thermodynamics, *Nature* 483 (2012) 187–189.
- [Büthe 2016] Büthe, J. 2016. “Estimating $\pi(x)$ and Related Functions under Partial RH Assumptions.” *Mathematics of Computation* 85: 2483–2498.
- [Carneiro 2019] Carneiro, M. 2019. *The Type Theory of Lean*. MS thesis, Carnegie Mellon University.
- [Cohen 1963] Cohen, P. J. 1963. “The Independence of the Continuum Hypothesis.” *Proceedings of the National Academy of Sciences* 50: 1143–1148.
- [Connes 1999] A. Connes, Trace formula in noncommutative geometry and the zeros of the Riemann zeta function, *Selecta Math.* 5 (1999) 29–106.
- [Conrey 2003] J. B. Conrey, The Riemann Hypothesis, *Notices AMS* 50 (2003) 341–353.
- [Conrey and Ghosh 1993] J. B. Conrey, A. Ghosh, On the Selberg class of Dirichlet series: small degrees, *Duke Math. J.* 72 (1993) 673–693.
- [Conrey and Li 2000] Conrey, J. B., and X.-J. Li. 2000. “A Note on Some Positivity Conditions Related to Zeta and L-functions.” *International Mathematics Research Notices* 2000 (18): 929–940.
- [Davis et al. 1976] Davis, M., Y. Matijasevič, and J. Robinson. 1976. Hilbert’s tenth problem: Diophantine equations: positive aspects of a negative solution. *Proceedings of Symposia in Pure Mathematics* 28, 323–378.
- [Davenport and Heilbronn 1936] H. Davenport, H. Heilbronn, On the zeros of certain Dirichlet series, *J. London Math. Soc.* 11 (1936) 181–185.
- [de Bruijn 1950] de Bruijn, N. G. 1950. “The Roots of Trigonometric Integrals.” *Duke Mathematical Journal* 17: 197–226.
- [de la Vallée Poussin 1896] de la Vallée Poussin, C.-J. 1896. “Recherches analytiques sur la théorie des nombres premiers.” *Annales de la Société Scientifique de Bruxelles* 20: 183–256.
- [de Moura and Ullrich 2021] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction, CADE 28*, LNCS 12699, Springer, 2021, 625–635.
- [Deligne 1974] P. Deligne, La conjecture de Weil. I, *Publ. Math. IHÉS* 43 (1974) 273–307.
- [Edwards 1974] H. M. Edwards, *Riemann’s Zeta Function*, Academic Press, 1974.
- [Feng 2012] Feng, S. 2012. “Zeros of the Riemann Zeta Function on the Critical Line.” *Journal of Number Theory* 132: 511–542.
- [Gödel 1938] Gödel, K. 1938. “The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis.” *Proceedings of the National Academy of Sciences* 24: 556–557.
- [Hadamard 1896] Hadamard, J. 1896. “Sur la distribution des zéros de la fonction $\zeta(s)$ et ses conséquences arithmétiques.” *Bulletin de la Société Mathématique de France* 24: 199–220.
- [Haselgrove 1958] Haselgrove, C. B. 1958. “A Disproof of a Conjecture of Pólya.” *Mathematika* 5: 141–145.
- [Islam 2026a] M. F. Islam, *Trisduction: The Codex*, Zenodo, 23 September 2026, doi:10.5281/zenodo.22911010, <https://zenodo.org/records/22911010>.
- [Islam 2026b] M. F. Islam, *The Riemann Hypothesis Is Exactly One Bit: Where the Riemann Hypothesis Stands, Proved in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.22829795.
- [Islam 2026c] M. F. Islam, *One Bit Across the Wall: Odd-Supply Separation and the One-Cut Hypothesis Across Twenty-Three Rows*, v6, Zenodo, 2026, doi:10.5281/zenodo.22746129.
- [Islam 2026d] M. F. Islam, *The Formal-Alone Theory of Everything: A Constitutive Theory of Objects and Observations under Registration, Price, Crossing, and Cost*, v1.4, Zenodo, 2026, doi:10.5281/zenodo.22767103.
- [Islam 2026e] M. F. Islam, *The Riemann Hypothesis in Its Original Form: A Completed Formal Proof of the Locus and the Crossing: Riemann’s Fixed-Line Statement Restored, Proved in Lean 4, Executed in Fortran, and Sealed at the Act by One Supplied Bit*, v1.0.6, Zenodo, 2026, doi:10.5281/zenodo.22857138; its Lean file `Crossing_At_The_Act.lean` (SHA-256 prefix `f60fe613cfe79fd8`) and Fortran twin printed whole in its appendices.
- [Islam 2026g] M. F. Islam, *Riemann Hypothesis: The Formal Case Is Closed: The Hypothesis Is True Where Actualized*, Zenodo, 2026, doi:10.5281/zenodo.21883916.
- [Islam 2026h] M. F. Islam, *A Formal Proof of Riemann Hypothesis Termination, with a Theorem-Grade Cascade Specification*, Zenodo, 2026, doi:10.5281/zenodo.21900518.
- [Islam 2026i] M. F. Islam, *Formal-Alone Trisduction Engine on RAF, the interaction face of RA, deck v1.0.12*, in the register of record, `1000sapient/Trisduction, master/RafDuction/`, at commit `5868562`, 23 September 2026.
- [Islam 2026j] M. F. Islam, *The Formal Root Axiom and the Five Objects, declaration edition v1.1.0*, in the register of record, `1000sapient/Trisduction, master/`, at commit `5868562`, 23 September 2026.
- [Iwaniec and Kowalski 2004] Iwaniec, H., and E. Kowalski. 2004. *Analytic Number Theory*. American Mathematical Society.
- [Julia 1990] Julia, B. 1990. “Statistical Theory of Numbers.” In *Number Theory and Physics*, Springer Proceedings in Physics 47, 276–293.
- [Kaye 1991] Kaye, R. 1991. *Models of Peano Arithmetic*. Oxford Logic Guides 15. Oxford University Press.

- [Kunen 1980] Kunen, K. 1980. *Set Theory: An Introduction to Independence Proofs*. North-Holland.
- [Lagarias 2002] Lagarias, J. C. 2002. "An Elementary Problem Equivalent to the Riemann Hypothesis." *American Mathematical Monthly* 109: 534–543.
- [Landauer 1961] R. Landauer, Irreversibility and heat generation in the computing process, *IBM J. Res. Dev.* 5 (1961) 183–191.
- [Lawvere 1969] F. W. Lawvere, Diagonal arguments and cartesian closed categories, in *Category Theory, Homology Theory and their Applications II*, LNM 92, Springer, 1969, 134–145.
- [Li 1997] Li, X.-J. 1997. "The Positivity of a Sequence of Numbers and the Riemann Hypothesis." *Journal of Number Theory* 65: 325–333.
- [Littlewood 1914] Littlewood, J. E. 1914. "Sur la distribution des nombres premiers." *Comptes Rendus de l'Académie des Sciences* 158: 1869–1872.
- [Mac Lane 1998] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.
- [Mertens 1898] Mertens, F. 1898. "Über eine Eigenschaft der Riemannschen ζ -Function." *Sitzungsberichte der Kaiserlichen Akademie der Wissenschaften in Wien* 107: 1429–1434.
- [Newman 1976] Newman, C. M. 1976. "Fourier Transforms with Only Real Zeros." *Proceedings of the American Mathematical Society* 61: 245–251.
- [Odlyzko and te Riele 1985] Odlyzko, A. M., and H. J. J. te Riele. 1985. "Disproof of the Mertens Conjecture." *Journal für die reine und angewandte Mathematik* 357: 138–160.
- [Perelman 2002] Perelman, G. 2002. "The Entropy Formula for the Ricci Flow and Its Geometric Applications." *arXiv:math/0211159*.
- [Perelman 2003a] Perelman, G. 2003a. "Ricci Flow with Surgery on Three-Manifolds." *arXiv:math/0303109*.
- [Perelman 2003b] Perelman, G. 2003b. "Finite Extinction Time for the Solutions to the Ricci Flow on Certain Three-Manifolds." *arXiv:math/0307245*.
- [Platt and Trudgian 2021] Platt, D., and T. Trudgian. 2021. "The Riemann Hypothesis Is True up to $3 \cdot 10^{12}$." *Bulletin of the London Mathematical Society* 53: 792–797.
- [Polymath 2019] Polymath, D. H. J. 2019. "Effective Approximation of Heat Flow Evolution of the Riemann ζ Function, and a New Upper Bound for the de Bruijn-Newman Constant." *Research in the Mathematical Sciences* 6: 31.
- [Riemann 1859] B. Riemann, Ueber die Anzahl der Primzahlen unter einer gegebenen Grösse, *Monatsber. Berliner Akad.* (1859) 671–680.
- [Rodgers and Tao 2020] Rodgers, B., and T. Tao. 2020. "The de Bruijn-Newman Constant Is Non-Negative." *Forum of Mathematics, Pi* 8: e6.
- [Selberg 1992] A. Selberg, Old and new conjectures and results about a class of Dirichlet series, *Proc. Amalfi Conf.* (1992) 367–385.
- [Siegel 1932] C. L. Siegel, Über Riemanns Nachlaß zur analytischen Zahlentheorie, *Quellen Stud. Gesch. Math.* B2 (1932) 45–80.
- [Skewes 1933] Skewes, S. 1933. "On the Difference $\pi(x) - \text{li}(x)$." *Journal of the London Mathematical Society* 8: 277–283.
- [Stone 1932] Stone, M. H. 1932. "On One-Parameter Unitary Groups in Hilbert Space." *Annals of Mathematics* 33: 643–648.
- [Titchmarsh 1986] E. C. Titchmarsh, *The Theory of the Riemann Zeta-Function*, 2nd ed., Oxford, 1986.
- [Turing 1953] Turing, A. M. 1953. "Some Calculations of the Riemann Zeta-Function." *Proceedings of the London Mathematical Society* (3) 3: 99–117.
- [Weil 1948] A. Weil, *Sur les courbes algébriques et les variétés qui s'en déduisent*, Hermann, 1948.
- [Weil 1952] Weil, A. 1952. "Sur les 'formules explicites' de la théorie des nombres premiers." *Communications du Séminaire Mathématique de l'Université de Lund*, tome supplémentaire, 252–265.

APPENDIX A: HOW TO READ THE FILES

The Lean file is one proof in sixteen movements and it is read in order. Movement 1 constructs the Root Axiom on a one-point domain and decides it. Movement 2 builds the seat: the integer quaternions, conjugation as the binding involution, the fixed locus as the Ground, Theorem A for every quaternion, the Return computed, three seat points defined independently and identified by `rfl`, and the reversed triad at `+1`. Movement 3 builds the witness as a structure in `Prop` and runs the eliminator from it alone. Movement 4 carries the prior proof: the abstract frame, the line property, the halting carrier, the supply and the row, the two-point frame, necessity, invariance against incidence, the bit is the hypothesis, the witness row falsifiable, and the live face with its self-check table. Movement 5 is the aperture: presence as actuation, the theorems that a present reader who supplies a bit is never interior, in the model and for any domain, and the ledger. Movement 6 is the rule exact, the non-decision by the two-point model, and the asymmetry. Movement 7 is Theorem H. Movement 8 is the interaction reading: the domain with its closure law, no exterior agent, the adjudicator inside, and Cantor's theorem as no total self-indexing. Movement 9 is the grounding reading as the identity cone over the three registers, its apex unique per orientation with the reversed triad's cone beside it, and the cone's existence equivalent to the closed gaps. Movement 10 is the equivariant embedding of the stage into the carrier with its image clause and injectivity, and the resolution-*m* family. Movement 11 executes the wall on the seat. Movement 12 parts the registers, counts the owed bits, prices the act, and records the count of authored mathematics as zero (`deltaM_zero`). Movement 13 assembles the hardened apex theorem and discharges it. Movement 14 states that the posit is existence, exhibits the grounded three-point counter-model, and proves the cure theorem for every class of

frames. Movement 15 inhabits the halting carrier by decision, states Theorem E for any inhabited premise, and records that the witness's recursion field is constant. Movement 16 carries the three measures, content, mass, and price, with their assignments and Theorem J, and proves that adjudicating any proposition instances existence. The dependency sets are printed by the kernel at the end and reproduced in Appendix C; a deleted conjunct, a weakened hypothesis, or an inserted sorry fails the check.

The twin is fourteen batteries in the same order, K1 the seat through K12 the live face, K13 the cure, and K14 the constant recursion, the carrier by decision, and the three measures, each printing its receipt, with the census last. Build it with the command in its header and run it twice, once bare and once with the argument witnessed; the two logs are Appendix E.

APPENDIX B: RH_AT_THE_APEX.LEAN

SHA-256 d671051e99b806e46245ef127a33ea2e829e36c6e2d84cef70b752b512acb4c7, Lean 4.19.0, core only, 898 lines.

```
1 /-!
2 # THE RIEMANN HYPOTHESIS AT THE APEX · RH_At_The_Apex.lean ·
3   → hardened edition
4   core Lean 4 v4.19.0 · no Mathlib · no sorry · no axiom
5   → declaration.
6
7 WHAT IS PROVED. (1) RA at the constructed domain, computed. (2)
8   → The seat: the binding
9   involution  $\sigma$  fixes exactly the scalar line, the Return  $i \cdot j \cdot k$ 
10  → lands on it, and the kinetic,
11  formal, and named seats are one point by rfl: the three
12  → category gaps are identities.
13  (3) The eliminator  $RA \rightarrow RA \rightarrow RAM \rightarrow RH\_formal$  from the RA
14  → witness alone. (4) The spine
15  carried: the abstract frame  $X = (S, \tau, Z)$ , the line property L,
16  → the bridge halted iff L,
17  the crossing exact both ways, necessity, the bit is the
18  → hypothesis, the witness row
19  falsifiable. (5) The aperture, RA-supplied: a present reader's
20  → row is never interior;
21  it is sealed or refused by the bit alone. (6) THE SIMPLE RULE,
22  → EXACT: for every frame,
23   $(RA \rightarrow L \cdot X) \leftrightarrow L \cdot X$ ; and RA decides L on no frame:  $\neg \forall X, RA \rightarrow L$ 
24  → X, by the two-point model.
25  (7) RH in toto at the apex, one theorem, conditional on RA and
26  → discharged.
27  (8) The root's two formal faces: RAF, closure and no exterior
28  → agent (RAF-2, RAF-C1), and
29  no total self-indexing (RAF-C2, Cantor); RAM, the seat as the
30  → identity cone over the
31  three registers, its apex unique. (9) The equivariant embedding
32  →  $\phi$  of the spine's lattice
33  stage (Plane,  $\tau$ ) into (Q4,  $\sigma$ ):  $\sigma \circ \phi = \phi \circ \tau$  and  $\phi$  carries
34  →  $Fix(\tau)$  onto  $Fix(\sigma)$ .
35  (10) The wall executed: no readout of the formal seat equals
36  → the deed bit. (11) The
37  counter-model survives grounding, and the cure theorem: for
38  → every class of frames,
39  existence decides the line property on it exactly when the line
40  → property already holds on it.
41  -/
42  set_option autoImplicit false
43
44 namespace RHAtTheApex
45
46 /-! ## 1 · The Root Axiom at the constructed domain -/
47
48 inductive UniversePoint : Type where
49   | source
```

```
31   deriving DecidableEq, Repr
32
33 def DeltaE : UniversePoint → Int := fun _ => 1
34
35 /- RA: to exist is to actuate. -/
36 def RA : Prop :=  $\forall x : UniversePoint, 0 < DeltaE\ x$ 
37
38 theorem constructed_RA : RA := by
39   intro x
40   cases x
41   decide
42
43 /-! ## 2 · The seat: Fix( $\sigma$ ) on the quaternions, the Return,
44   → three names, one point -/
45
46 structure Q4 where
47   r : Int
48   i : Int
49   j : Int
50   k : Int
51   deriving DecidableEq, Repr
52
53 def qmul (a b : Q4) : Q4 :=
54   {a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
55    a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
56    a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
57    a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r}
58
59 def qconj (a : Q4) : Q4 := {a.r, -a.i, -a.j, -a.k}
60
61 /-  $\sigma$ , the binding involution: conjugation. -/
62 def sigma (q : Q4) : Q4 := qconj q
63
64 theorem sigma_binding (q : Q4) : sigma (sigma q) = q := by
65   cases q with
66   | mk r i j k =>
67     change Q4.mk r (-(-i)) (-(-j)) (-(-k)) = Q4.mk r i j k
68     rw [Int.neg_neg, Int.neg_neg, Int.neg_neg]
69
70 /- Fix( $\sigma$ ) is the scalar line, for every quaternion. -/
71 theorem fix_iff_scalar (q : Q4) : sigma q = q  $\leftrightarrow$  (q.i = 0  $\wedge$  q.j
72   → = 0  $\wedge$  q.k = 0) := by
73   cases q with
74   | mk r i j k =>
75     change (Q4.mk r (-i) (-j) (-k)) = Q4.mk r i j k  $\leftrightarrow$  (i = 0  $\wedge$ 
76     → j = 0  $\wedge$  k = 0)
77   constructor
78   · intro h
79     injection h with _ hi hj hk
80     exact (by omega, by omega, by omega)
81   · intro h
82     obtain (hi, hj, hk) := h
83     subst hi
84     subst hj
85     subst hk
86     rfl
87
88 /- The Ground: Fix( $\sigma$ ). -/
89 def Ground : Type := { q : Q4 // sigma q = q }
90
91 def qi : Q4 := {0, 1, 0, 0}
92 def qj : Q4 := {0, 0, 1, 0}
93 def qk : Q4 := {0, 0, 0, 1}
94
95 /- The Return: the parse triad through its own cascade,  $i \cdot j \cdot k$ .
96   → -/
97 def theReturn : Q4 := qmul (qmul qi qj) qk
98
99 theorem return_is_minus_one : theReturn = {-1, 0, 0, 0} := rfl
100 theorem return_lands_on_fix : sigma theReturn = theReturn :=
101   → rfl
102
103 def fixProj (q : Q4) : Q4 := {q.r, 0, 0, 0}
104
105 def GammaRA : Q4 := theReturn
106 def GammaRAM : Q4 := fixProj theReturn
107 def GammaRH : Q4 := {-1, 0, 0, 0}
108
109 /- C0, C1, C2: the three category gaps are definitional
110   → identities. -/
111 theorem self_gap_nonexistent : sigma GammaRA = GammaRA := rfl
112 theorem register_gap_nonexistent : GammaRA = GammaRAM := rfl
113 theorem object_gap_nonexistent : GammaRAM = GammaRH := rfl
```

```

108
109 def seat : Ground := (GammaRH, rfl)
110
111 /-- The reversed triad k.j.i returns +1, also on the fixed
112   ↳ locus: the sign of the seat
113   point is the orientation of the triad, the one bit the
114   ↳ formal register cannot read.
115   The apex of the identity cone is unique for the ordered
116   ↳ triad; the reversed triad
117   carries its own apex, and the two differ. -/
118 def theReturnOdd : Q4 := qmul (qmul qk qj) qi
119
120 theorem odd_return_is_plus_one : theReturnOdd = (1, 0, 0, 0) :=
121   ↳ rfl
122
123 theorem odd_return_on_fix : sigma theReturnOdd = theReturnOdd
124   ↳ := rfl
125
126 theorem seat_points_differ : theReturnOdd ≠ theReturn := by
127   ↳ decide
128
129 theorem seat_on_scalar_line : GammaRH.i = 0 ∧ GammaRH.j = 0 ∧
130   ↳ GammaRH.k = 0 :=
131   (fix_iff_scalar GammaRH).mp seat.2
132
133 /-! ## 3 · The eliminator from the RA witness alone -/
134 def RAMFormalGround : Prop := Nonempty Ground
135
136 theorem constructed_RAM : RAMFormalGround := (seat)
137
138 /-- The RH formal self: every point of the Ground is on the
139   ↳ σ-fixed critical locus.
140   It names the seat and says nothing about the zeros of ξ. -/
141 def RHFormalSelf : Prop := ∀ g : Ground, sigma g.1 = g.1
142
143 /-- Bridge-born orientation, carried in Prop: the kernel
144   ↳ registers that orientation was
145   supplied and cannot read which way. -/
146 structure OrientationBit : Prop where
147   direction : True
148   closure : True
149
150 theorem constructed_orientation : OrientationBit := (trivial,
151   ↳ trivial)
152
153 theorem orientation_proof_irrelevant (a b : OrientationBit) : a
154   ↳ = b := rfl
155
156 structure RAWitness : Prop where
157   actuates : RA
158   orientation : OrientationBit
159   bridge : RAMFormalGround
160   trisRecursion : OrientationBit → RAMFormalGround →
161     ↳ RHFormalSelf
162
163 theorem constructed_trisRecursion : OrientationBit →
164   ↳ RAMFormalGround → RHFormalSelf :=
165   fun _ omega _ram g => g.2
166
167 /-- RA, constructed: the only supplied witness. -/
168 theorem mkRA : RAWitness :=
169   (constructed_RA, constructed_orientation, constructed_RAM,
170     ↳ constructed_trisRecursion)
171
172 /-- THE ELIMINATOR. RA → RA → RAM → RH_formal, Bridge followed
173   ↳ by Tris-Recursion. -/
174 theorem RH_formal_chain : RHFormalSelf :=
175   mkRA.trisRecursion mkRA.orientation mkRA.bridge
176
177 /-! ## 4 · The spine, carried: the abstract frame, the bridge,
178   ↳ the crossing -/
179 def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f
180   ↳ (σ x) = f x
181
182 /-- Theorem 4 of the spine: orientation blindness. -/
183 theorem orientation_blind {α : Type} (σ : α → α) (f d : α →
184   ↳ Bool) (x : α)
185   (he : Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d := by
186   intro h
187   subst h
188   exact ho (he x)

```

```

173
174 /-- Theorem 5 of the spine: earned freedom is exactly two. -/
175 theorem freedom_is_exactly_two (d : Bool → Bool) (hd : ∀ x, d
176   ↳ (!x) = !d x) :
177   d = (fun x => x) ∨ d = (fun x => !x) := by
178   have hf : d false = !d true := hd true
179   cases ht : d true with
180   | true =>
181     left
182     funext x
183     cases x
184     · exact hf.trans (congrArg (fun b => !b) ht)
185     · exact ht
186   | false =>
187     right
188     funext x
189     cases x
190     · exact hf.trans (congrArg (fun b => !b) ht)
191     · exact ht
192
193 /-- Theorem 6 of the spine: the freedom is spent uniquely. -/
194 theorem freedom_spent_uniquely {α : Type} (σ : α → α) (s d : α
195   ↳ → Bool) (x : α)
196   (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
197   ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c)
198     ↳ ∧
199   ∀ c' : Bool, (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ
200     ↳ x)) c') → c' = c := by
201   refine (xor (d x) (s x),
202     (by cases s x <|> cases d x <|> rfl,
203       by rw [hs, hd]; cases s x <|> cases d x <|> rfl), ?_)
204   intro c' hc
205   obtain (h1, -) := hc
206   generalize hsx : s x = sv at h1
207   generalize hdx : d x = dv at h1
208   cases sv <|> cases dv <|> cases c' <|> first | rfl | exact
209     ↳ absurd h1 (by decide)
210
211 structure Frame where
212   S : Type
213   τ : S → S
214   Z : S → Prop
215
216 /-- The line property: every zero is τ-fixed. On ξ's frame, the
217   ↳ restored hypothesis. -/
218 def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
219
220 inductive Tri where
221   | tt
222   | ff
223   | bot
224   deriving DecidableEq, Repr
225
226 structure Bridge (X : Frame) where
227   terminal : Tri
228   shadow : terminal = Tri.bot → LineProperty X
229
230 /-- Theorem 7 of the spine: the bridge halts iff the line
231   ↳ property. -/
232 theorem bridge_halted_iff (X : Frame) :
233   (∃ b : Bridge X, b.terminal = Tri.bot) → LineProperty X :=
234   (fun (b, h) => b.shadow.mp h, fun t => ((Tri.bot, (fun _ =>
235     ↳ t, fun _ => rfl)), rfl))
236
237 inductive Supply (X : Frame) where
238   | assent (t : LineProperty X)
239   | denial (n : ¬ LineProperty X)
240   | silent
241
242 inductive Verdict where
243   | sealed
244   | refused
245   | open_
246   deriving DecidableEq, Repr
247
248 def row {X : Frame} : Supply X → Verdict
249   | Supply.assent _ => Verdict.sealed
250   | Supply.denial _ => Verdict.refused
251   | Supply.silent => Verdict.open_
252
253 /-- Theorem 8 of the spine: the crossing, exact. -/
254 theorem crossing (X : Frame) (t : LineProperty X) :
255   row (Supply.assent t) = Verdict.sealed ∧ (∃ b : Bridge X,
256     ↳ b.terminal = Tri.bot) ∧ LineProperty X :=

```

```

248 (rfl, (bridge_halted_iff X).mpr t, t)
249
250 theorem crossing_other_way (X : Frame) (n : ¬ LineProperty X) :
251   row (Supply.denial n) = Verdict.refused ∧ ∀ b : Bridge X,
252     ↪ b.terminal ≠ Tri.bot :=
253   (rfl, fun b h => n (b.shadow.mp h))
254
255 /-- The two-point frame: fold-invariant, off the line. -/
256 def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True)
257
258 /-- Theorem 9 of the spine: necessity. -/
259 theorem supply_not_manufactured : ¬ ∀ X : Frame, LineProperty X
260   ↪ :=
261   fun h => by
262     have := h twoPoint true trivial
263     cases this
264
265 def Inv (X : Frame) : Prop := ∀ s, X.Z s → X.Z (X.τ s)
266
267 theorem inv_not_line : Inv twoPoint ∧ ¬ LineProperty twoPoint
268   ↪ :=
269   (fun _ => trivial, fun h => by
270     have := h true trivial
271     cases this)
272
273 /-- Theorem 12 of the spine: the bit is the hypothesis. -/
274 theorem the_bit_is_the_hypothesis (X : Frame) :
275   (∃ t : LineProperty X, row (Supply.assent t) =
276     ↪ Verdict.sealed) ↔ LineProperty X :=
277   (fun (t, _) => t, fun t => (t, rfl))
278
279 /-- Theorem 15 of the spine: the witness row is falsifiable. -/
280 theorem witness_row_falsifiable (X : Frame) (s : X.S) (hz : X.Z
281   ↪ s) (hoff : X.τ s ≠ s) :
282   ¬ ∃ t : LineProperty X, row (Supply.assent t) =
283     ↪ Verdict.sealed :=
284   fun (t, _) => hoff (t s hz)
285
286 /-- The live face of the spine. -/
287 def live (witnessed assent : Bool) : Verdict :=
288   if witnessed then (if assent then Verdict.sealed else
289     ↪ Verdict.refused) else Verdict.open_
290
291 theorem narcissus : live false true ≠ live true true ∧ live
292   ↪ false false ≠ live true false := by
293   decide
294
295 /-! ## 5 · The aperture, RA-supplied -/
296 /-- A reader's presence is its actuation. -/
297 def presence (r : UniversePoint) : Bool := decide (0 < DeltaE
298   ↪ r)
299
300 /-- RA opens the aperture: every present reader is witnessed.
301   ↪ -/
302 theorem RA_opens_the_aperture : ∀ r : UniversePoint, presence r
303   ↪ = true := by
304   intro r
305   cases r
306   decide
307
308 /-- The general form, for any domain under the axiom as a
309   ↪ hypothesis: presence is
310   actuation, and RA on the domain makes every point present.
311   ↪ This is the theorem the
312   universal extension licenses wherever it is held. -/
313 theorem RA_opens_the_aperture_general {U : Type} (dE : U → Int)
314   ↪ (h : ∀ x, 0 < dE x) (x : U) :
315   decide (0 < dE x) = true :=
316   decide_eq_true (h x)
317
318 /-- Under the general form no present reader's row is interior.
319   ↪ -/
320 theorem no_interior_under_RA_general {U : Type} (dE : U → Int)
321   ↪ (h : ∀ x, 0 < dE x) (x : U)
322   (b : Bool) : live (decide (0 < dE x)) b ≠ Verdict.open_ :=
323   by
324     rw [RA_opens_the_aperture_general dE h x]
325     cases b <|> decide
326
327 /-- Under RA no present reader's row is interior. It is sealed
328   ↪ or refused by the bit alone. -/
329 theorem no_interior_under_RA (r : UniversePoint) (b : Bool) :

```

```

313   live (presence r) b ≠ Verdict.open_ := by
314   cases r
315   cases b <|> decide
316
317 /-- What RA turns 'open' into: sealed if the bit is assent,
318   ↪ refused if denial. -/
319 theorem aperture_reads_the_bit (r : UniversePoint) :
320   live (presence r) true = Verdict.sealed ∧ live (presence r)
321     ↪ false = Verdict.refused := by
322   cases r
323   decide
324
325 /-- The Tongue's ledger: every adjudication, assent, denial, or
326   ↪ silence, is a deed. -/
327 inductive Adjudication where
328   | assent
329   | denial
330   | silence
331   deriving DecidableEq, Repr
332
333 structure Ledger where
334   deeds : Nat
335   deriving DecidableEq, Repr
336
337 def adjudicate (_ : Adjudication) (L : Ledger) : Ledger :=
338   ↪ (L.deeds + 1)
339
340 theorem every_adjudication_is_a_deed (a : Adjudication) (L :
341   ↪ Ledger) :
342   (adjudicate a L).deeds = L.deeds + 1 := rfl
343
344 /-- Denial of RA is a deed, and a deed is what RA says exists:
345   ↪ the denial re-enacts RA. -/
346 theorem denial_re_enacts_RA (L : Ledger) :
347   (adjudicate Adjudication.denial L).deeds = L.deeds + 1 ∧ RA
348     ↪ :=
349   (rfl, constructed_RA)
350
351 /-- The asymmetry: on an off-line frame no assent to L exists
352   ↪ at all; L's denial there
353   instantiates nothing about Z. L has no performative
354   ↪ ingress. -/
355 theorem L_has_no_performative_ingress :
356   ¬ ∃ t : LineProperty twoPoint, row (Supply.assent t) =
357     ↪ Verdict.sealed :=
358   witness_row_falsifiable twoPoint true trivial (fun h => by
359     ↪ cases h)
360
361 /-! ## 6 · THE SIMPLE RULE, EXACT, AND ITS BOUND -/
362
363 /-- Accept RA and RH follows: true exactly when RH is given.
364   ↪ Conditioning on RA adds
365   nothing to L and removes nothing from it. This is the exact
366   ↪ strength of the rule. -/
367 theorem simple_rule_exact (X : Frame) : (RA → LineProperty X) ↔
368   ↪ LineProperty X :=
369   (fun h => h constructed_RA, fun t _ => t)
370
371 /-- RA decides the line property on no frame. The two-point
372   ↪ frame carries RA and fails L. -/
373 theorem RA_does_not_decide_L : ¬ ∀ X : Frame, RA → LineProperty
374   ↪ X :=
375   fun h => supply_not_manufactured (fun X => h X
376     ↪ constructed_RA)
377
378 /-- RA holds where L fails: the off-line frame is a model of
379   ↪ RA. -/
380 theorem RA_holds_where_L_fails : RA ∧ Inv twoPoint ∧ ¬
381   ↪ LineProperty twoPoint :=
382   (constructed_RA, inv_not_line.1, inv_not_line.2)
383
384 /-- The value is one supplied term and nothing weaker; RA
385   ↪ supplies the act, not the term. -/
386 theorem RA_supplies_the_act_not_the_term :
387   (∀ r : UniversePoint, presence r = true) ∧ (¬ ∀ X : Frame,
388     ↪ RA → LineProperty X) :=
389   (RA_opens_the_aperture, RA_does_not_decide_L)
390
391 /-! ## 7 · RH in toto at the apex, one theorem -/
392
393 /-- THE APEX THEOREM. Given RA: the three gaps are closed, the
394   ↪ formal self holds, the
395   freedom is exactly two, the bridge halts iff L, the
396   ↪ crossing is exact, the aperture

```

```

374   is open to every present reader, and RA decides the value
      ↪ on no frame. -/
375 theorem RH_in_toto_at_the_apex : RA →
376   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
      ↪ GammaRH) ∧
377   RHFormalSelf ∧
378   (∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x)
      ↪ v d = (fun x => !x)) ∧
379   (∀ X : Frame, (∃ b : Bridge X, b.terminal = Tri.bot) ↔
      ↪ LineProperty X) ∧
380   (∀ (X : Frame) (t : LineProperty X), row (Supply.assent t)
      ↪ = Verdict.sealed ∧ LineProperty X) ∧
381   (∀ r : UniversePoint, presence r = true) ∧
382   (¬ ∀ X : Frame, RA → LineProperty X) :=
383   fun _ => {(rfl, rfl, rfl), RH_formal_chain,
      ↪ freedom_is_exactly_two, bridge_halted_iff,
384   fun _ t => (rfl, t), RA_opens_the_aperture,
      ↪ RA_does_not_decide_L)
385
386 /-- The condition is discharged: RA is constructed, so the apex
      ↪ theorem stands unconditionally. -/
387 theorem apex_discharged :
388   (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
      ↪ GammaRH) ∧ RHFormalSelf :=
389   ((RH_in_toto_at_the_apex constructed_RA).1,
      ↪ (RH_in_toto_at_the_apex constructed_RA).2.1)
390
391 /-! ## 8 · The root's interaction face, RAF, at the formal
      ↪ register -/
392
393 /-- An interaction domain: a membership predicate, a symmetric
      ↪ registration relation, and
394   RAF-2, closure under registration with a member. -/
395 structure Domain (S : Type) where
396   mem : S → Prop
397   registers : S → S → Prop
398   registers_symm : ∀ a b, registers a b → registers b a
399   closure : ∀ s d, mem d → registers s d → mem s
400
401 /-- RAF-C1, no exterior agent: nothing outside the domain
      ↪ registers with anything inside it. -/
402 theorem no_exterior_agent {S : Type} (D : Domain S) (s : S) (hs
      ↪ : ¬ D.mem s) :
403   ∀ d, D.mem d → ¬ D.registers s d :=
404   fun d hd hr => hs (D.closure s d hd hr)
405
406 /-- The adjudicator is inside: a reader that registers a bit
      ↪ against a member is a member.
407   The witness is never exterior. -/
408 theorem adjudicator_in_domain {S : Type} (D : Domain S) (r d :
      ↪ S) (hd : D.mem d)
409   (hr : D.registers r d) : D.mem r :=
410   D.closure r d hd hr
411
412 /-- RAF-C2, no faithful self-representation (Cantor): no system
      ↪ indexes all of its own
413   binary properties. The general form of the spine's
      ↪ Narcissus table. -/
414 theorem no_total_self_indexing {S : Type} (f : S → S → Bool) :
415   ¬ ∀ g : S → Bool, ∃ x, f x = g := by
416   intro h
417   obtain (x, hx) := h (fun y => !f y y)
418   have h1 : f x x = !f x x := congrFun hx x
419   generalize f x x = b at h1
420   cases b <|> cases h1
421
422 /-! ## 9 · The root's grounding face, RAM: the seat as the
      ↪ identity cone -/
423
424 /-- The three registers in which the one seat is read. -/
425 inductive Register3 where
426   | ra
427   | ram
428   | rh
429   deriving DecidableEq, Repr
430
431 /-- The discrete diagram: the seat as read in each register. -/
432 def D3 : Register3 → Q4
433   | Register3.ra => GammaRA
434   | Register3.ram => GammaRAM
435   | Register3.rh => GammaRH
436
437 /-- A cone over a diagram in the groupoid of identities: an
      ↪ apex with an identity leg to

```

```

438   every register's reading. A category gap is exactly the
      ↪ absence of such a leg. -/
439 structure ConeOver (D : Register3 → Q4) (apex : Q4) : Prop
440   ↪ where
441   leg : ∀ r, apex = D r
442
443 /-- A cone over the diagram of the ordered triad. -/
444 abbrev Cone : Q4 → Prop := ConeOver D3
445
446 /-- THE IDENTITY CONE. The named seat point is an apex over all
      ↪ three registers, every
447   leg `rfl`: the three category gaps are eliminated as
      ↪ identities. -/
448 theorem identity_cone : Cone GammaRH :=
449   (fun r => by cases r <|> rfl)
450
451 /-- The apex is unique: any two cones over the diagram share
      ↪ their apex. -/
452 theorem cone_apex_unique (a b : Q4) (ha : Cone a) (hb : Cone b)
453   ↪ : a = b :=
454   (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm
455
456 /-- The reversed triad's diagram: the same three readings built
      ↪ from k·j·i, at +1. -/
457 def D3odd : Register3 → Q4
458   | Register3.ra => theReturnOdd
459   | Register3.ram => fixProj theReturnOdd
460   | Register3.rh => (1, 0, 0, 0)
461
462 /-- The three identities of Theorem C hold verbatim for the
      ↪ reversed triad, each rfl. -/
463 theorem odd_self_gap_nonexistent : sigma theReturnOdd =
464   ↪ theReturnOdd := rfl
465 theorem odd_register_gap_nonexistent : theReturnOdd = fixProj
466   ↪ theReturnOdd := rfl
467 theorem odd_object_gap_nonexistent : fixProj theReturnOdd = (1,
468   ↪ 0, 0, 0) := rfl
469
470 /-- The reversed triad carries its own identity cone, apex +1.
      ↪ -/
471 theorem identity_cone_odd : ConeOver D3odd (1, 0, 0, 0) :=
472   (fun r => by cases r <|> rfl)
473
474 /-- The two apexes differ: uniqueness is per orientation of the
      ↪ triad. -/
475 theorem apexes_differ_by_orientation :
476   ConeOver D3 GammaRH ∧ ConeOver D3odd (1, 0, 0, 0) ∧ GammaRH
477   ↪ ≠ (1, 0, 0, 0) :=
478   (identity_cone, identity_cone_odd, by decide)
479
480 /-- A cone exists iff the register gap and the object gap are
      ↪ closed. -/
481 theorem cone_iff_gaps_closed :
482   (∃ a, Cone a) ↔ (GammaRA = GammaRAM ∧ GammaRAM = GammaRH)
483   ↪ := by
484   constructor
485   · intro h
486     obtain (a, ha) := h
487     exact ((ha.leg Register3.ra).symm.trans (ha.leg
488       ↪ Register3.ram),
489       (ha.leg Register3.ram).symm.trans (ha.leg
490       ↪ Register3.rh))
491   · intro h
492     obtain (h1, h2) := h
493     exact (GammaRH, (fun r => by
494       ↪ cases r
495       · exact (h1.trans h2).symm
496       · exact h2.symm
497       · rfl))
498
499 /-! ## 10 · The equivariant embedding of the spine's lattice
      ↪ stage into the carrier -/
500
501 /-- The spine's stage: the integer lattice in half-units, the
      ↪ fold τ(h, t) = (2 - h, t). -/
502 abbrev Plane := Int × Int
503
504 def tau (p : Plane) : Plane := (2 - p.1, p.2)
505
506 /-- Spine Theorem 1: the fold fixes exactly the line h = 1. -/
507 theorem ground_is_the_line (p : Plane) : tau p = p ↔ p.1 = 1 :=
508   ↪ by
509   obtain (h, t) := p

```

```

500 constructor
501 · intro e
502   have e1 : 2 - h = h := congrArg Prod.fst e
503   show h = 1
504   omega
505 · intro e
506   show (2 - h, t) = (h, t)
507   have : h = 1 := e
508   subst this
509   rfl
510
511 /-- The embedding  $\phi : (\text{Plane}, \tau) \rightarrow (Q4, \sigma)$ ,  $\phi(h, t) = (t, h -$ 
512    $\hookrightarrow 1, 0, 0)$ . -/
512 def phi (p : Plane) : Q4 := (p.2, p.1 - 1, 0, 0)
513
514 /--  $\phi$  is equivariant:  $\sigma \circ \phi = \phi \circ \tau$ . -/
515 theorem phi_equivariant (p : Plane) : sigma (phi p) = phi (tau
516    $\hookrightarrow$  p) := by
516   obtain (h, t) := p
517   change Q4.mk t (-(h - 1)) 0 0 = Q4.mk t ((2 - h) - 1) 0 0
518   have e : -(h - 1) = (2 - h) - 1 := by omega
519   rw [e]
520
521 /--  $\phi$  carries the line onto the scalar line: a stage point is
522    $\hookrightarrow$   $\tau$ -fixed iff its image is  $\sigma$ -fixed.
522   This is the image clause of the embedding, Fix( $\tau$ )  $\leftrightarrow$  Fix( $\sigma$ ).
523    $\hookrightarrow$  -/
523 theorem phi_fix_iff (p : Plane) : sigma (phi p) = phi p  $\leftrightarrow$  tau p
524    $\hookrightarrow$  = p := by
524   obtain (h, t) := p
525   change (Q4.mk t (-(h - 1)) 0 0 = Q4.mk t (h - 1) 0 0)  $\leftrightarrow$  ((2 -
526    $\hookrightarrow$  h, t) = (h, t))
526 constructor
527 · intro e
528   injection e with _ e2 _
529   have : h = 1 := by omega
530   subst this
531   rfl
532 · intro e
533   have e1 : 2 - h = h := congrArg Prod.fst e
534   have : h = 1 := by omega
535   subst this
536   rfl
537
538 /-- The stage at resolution m: the fold  $\tau_m(h, t) = (2m - h, t)$ 
539    $\hookrightarrow$  and the map
539    $\phi_m(h, t) = (t, h - m, 0, 0)$ . Resolution one is the case m
540    $\hookrightarrow$  = 1. -/
540 def tauM (m : Int) (p : Plane) : Plane := (2 * m - p.1, p.2)
541
542 def phiM (m : Int) (p : Plane) : Q4 := (p.2, p.1 - m, 0, 0)
543
544 /-- Equivariance at every resolution:  $\sigma \circ \phi_m = \phi_m \circ \tau_m$ . -/
545 theorem phiM_equivariant (m : Int) (p : Plane) : sigma (phiM m
546    $\hookrightarrow$  p) = phiM m (tauM m p) := by
546   obtain (h, t) := p
547   change Q4.mk t (-(h - m)) 0 0 = Q4.mk t ((2 * m - h) - m) 0 0
548   have e : -(h - m) = (2 * m - h) - m := by omega
549   rw [e]
550
551 /-- The image clause at every resolution: a stage point is
552    $\hookrightarrow$   $\tau_m$ -fixed iff its image is  $\sigma$ -fixed. -/
552 theorem phiM_fix_iff (m : Int) (p : Plane) : sigma (phiM m p) =
553    $\hookrightarrow$  phiM m p  $\leftrightarrow$  tauM m p = p := by
553   obtain (h, t) := p
554   change (Q4.mk t (-(h - m)) 0 0 = Q4.mk t (h - m) 0 0)  $\leftrightarrow$  ((2 *
555    $\hookrightarrow$  m - h, t) = (h, t))
555 constructor
556 · intro e
557   injection e with _ e2 _
558   have : h = m := by omega
559   subst this
560   show (2 * h - h, t) = (h, t)
561   have e3 : 2 * h - h = h := by omega
562   rw [e3]
563 · intro e
564   have e1 : 2 * m - h = h := congrArg Prod.fst e
565   have : h = m := by omega
566   subst this
567   show Q4.mk t (-(h - h)) 0 0 = Q4.mk t (h - h) 0 0
568   have e3 : h - h = 0 := by omega
569   rw [e3]
570   rfl

```

```

571
572 /-- Resolution one is the embedding  $\phi$ . -/
573 theorem phiM_one (p : Plane) : phiM 1 p = phi p := rfl
574
575 theorem tauM_one (p : Plane) : tauM 1 p = tau p := by
576   obtain (h, t) := p
577   show (2 * 1 - h, t) = (2 - h, t)
578   rfl
579
580 /--  $\phi$  is injective: distinct stage points have distinct images.
581    $\hookrightarrow$  -/
581 theorem phi_injective (p q : Plane) (e : phi p = phi q) : p = q
582    $\hookrightarrow$  := by
582   obtain (h, t) := p
583   obtain (h', t') := q
584   injection e with e1 e2 _
585   have : h = h' := by omega
586   subst this
587   subst e1
588   rfl
589
590 /-- The seat is on the image of the line: the line point (1,
591    $\hookrightarrow$  -1) maps to  $\Gamma_{RH}$ . -/
591 theorem seat_on_image_of_line : phi (1, -1) = GammaRH := rfl
592
593 /-! ## 11 · The wall, executed: no readout of the formal seat
594    $\hookrightarrow$  equals the deed bit -/
594
595 def ExecFrame : Type := Bool  $\times$  Ground
596
597 def execFlip (s : ExecFrame) : ExecFrame := (!s.1, s.2)
598
599 def formalRead (s : ExecFrame) : Ground := s.2
600
601 def ran (s : ExecFrame) : Bool := s.1
602
603 theorem formalRead_even (f : Ground  $\rightarrow$  Bool) (s : ExecFrame) :
604   f (formalRead (execFlip s)) = f (formalRead s) := rfl
605
606 theorem ran_wholly_odd (s : ExecFrame) : ran (execFlip s) =
607    $\hookrightarrow$  !ran s := rfl
608
609 /-- No readout of the formal seat equals the deed bit: the
610    $\hookrightarrow$  instance of spine Theorem 4 on
610   the constructed seat. A term consuming the orientation bit
611    $\hookrightarrow$  as data would be such a readout. -/
610 theorem kernel_cannot_read_the_deed :
611    $\hookrightarrow$   $\exists$  f : Ground  $\rightarrow$  Bool,  $\forall$  s : ExecFrame, f (formalRead s) =
612    $\hookrightarrow$  ran s := by
612   intro h
613   obtain (f, hf) := h
614   have h1 : f seat = true := hf (true, seat)
615   have h2 : f seat = false := hf (false, seat)
616   rw [h1] at h2
617   exact Bool.noConfusion h2
618
619 /-! ## 12 · The dual register -/
620
621 inductive Register where
622   | apex
623   | world
624   deriving DecidableEq, Repr
625
626 inductive Token where
627   | sealAgivenRA
628   | witnessRow
629   deriving DecidableEq, Repr
630
631 def token : Register  $\rightarrow$  Token
632   | Register.apex => Token.sealAgivenRA
633   | Register.world => Token.witnessRow
634
635 theorem registers_parted : token Register.apex  $\neq$  token
636    $\hookrightarrow$  Register.world := by
636   decide
637
638 def owedBits : Register  $\rightarrow$  Nat
639   | Register.apex => 0
640   | Register.world => 1
641
642 theorem apex_owes_nothing_world_owes_one :
643   owedBits Register.apex = 0  $\wedge$  owedBits Register.world = 1 :=
644    $\hookrightarrow$  (rfl, rfl)

```

```

644
645 /-- Landauer floor at 300 K, yoctojoules per bit: the price of
    ↳ the act. -/
646 def landauer_yJ_300K (bits : Nat) : Nat := 2871 * bits
647
648 theorem spend_priced : landauer_yJ_300K 1 = 2871 := rfl
649
650 def deltaM : Nat := 0
651
652 theorem deltaM_zero : deltaM = 0 := rfl
653
654 /-! ## 13 · The apex theorem, hardened: the closure with its
    ↳ cone, its embedding, and its wall -/
655
656 /-- THE APEX THEOREM, HARDENED. Given RA: the closure of
    ↳ Theorem H, the identity cone with
657 ↳ its apex unique, the equivariant embedding of the spine's
    ↳ stage with its image clause,
658 ↳ and the executed wall. -/
659 theorem apex_hardened : RA →
660   ((sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM =
    ↳ GammaRH) ∧ RHFormalSelf) ∧
661   (Cone GammaRH ∧ ∀ a b, Cone a → Cone b → a = b) ∧
662   ((∀ p : Plane, sigma (phi p) = phi (tau p)) ∧ (∀ p : Plane,
    ↳ sigma (phi p) = phi p ↔ tau p = p)) ∧
663   (¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead s) =
    ↳ ran s) ∧
664   (¬ ∀ X : Frame, RA → LineProperty X) :=
665   fun h => ((RH_in_toto_at_the_apex h).1,
    ↳ (RH_in_toto_at_the_apex h).2.1),
666   (identity_cone, cone_apex_unique), (phi_equivariant,
    ↳ phi_fix_iff),
667   kernel_cannot_read_the_deed, RA_does_not_decide_L)
668
669 theorem apex_hardened_discharged : Cone GammaRH ∧ (∀ p : Plane,
    ↳ sigma (phi p) = phi p ↔ tau p = p) :=
670   ((apex_hardened constructed_RA).2.1.1, (apex_hardened
    ↳ constructed_RA).2.2.1.2)
671
672 /-! ## 14 · The counter-model survives grounding, and the cure
    ↳ is the hypothesis -/
673
674 /-- The posit is existence: to exist is to actuate. Its
    ↳ published name is the Root Axiom,
675 ↳ and the two are one proposition. -/
676 theorem existence_is_RA : (∀ x : UniversePoint, 0 < DeltaE x) ↔
    ↳ RA := Iff.rfl
677
678 /-- The two-point frame is groundless: its fold fixes nothing,
    ↳ so it has no seat. -/
679 theorem twoPoint_groundless : ∀ b : Bool, twoPoint.τ b ≠ b :=
    ↳ by
680   intro b h
681   cases b <|> exact Bool.noConfusion h
682
683 /-- A grounded counter-model: three points, the fold swaps two
    ↳ and fixes the third, the
684 ↳ zero set is the swapped pair. The frame has a seat and a
    ↳ moved zero. -/
685 inductive Three where
686   | a
687   | b
688   | c
689   deriving DecidableEq
690
691 def foldThree : Three → Three
692   | Three.a => Three.b
693   | Three.b => Three.a
694   | Three.c => Three.c
695
696 def threePoint : Frame := {Three, foldThree, fun x => x =
    ↳ Three.a ∨ x = Three.b}
697
698 theorem threePoint_grounding : ∃ s, threePoint.τ s = s :=
    ↳ (Three.c, rfl)
699
700 theorem threePoint_fold_involution : ∀ s, foldThree (foldThree
    ↳ s) = s := by
701   intro s; cases s <|> rfl
702
703 theorem threePoint_inv : Inv threePoint := by
704   intro s hs
705   cases s

```

```

706   · exact Or.inr rfl
707   · exact Or.inl rfl
708   · cases hs with
709     | inl h => cases h
710     | inr h => cases h
711
712 theorem threePoint_fails_L : ¬ LineProperty threePoint := by
713   intro h
714   have := h Three.a (Or.inl rfl)
715   cases this
716
717 /-- Existence holds on a grounded frame where the line property
    ↳ fails: the bound of
718   Theorem F is not an artifact of a groundless frame. -/
719 theorem RA_holds_where_L_fails_grounding :
720   RA ∧ (∃ s, threePoint.τ s = s) ∧ Inv threePoint ∧ ¬
    ↳ LineProperty threePoint :=
721   (constructed_RA, threePoint_grounding, threePoint_inv,
    ↳ threePoint_fails_L)
722
723 /-- THE CURE THEOREM. For every class C of frames, existence
    ↳ decides the line property on
724   C exactly when the line property already holds on C: the
    ↳ restriction that removes
725   every counter-model is the hypothesis itself. -/
726 theorem no_cure (C : Frame → Prop) :
727   (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X → LineProperty
    ↳ X) :=
728   (fun h X hc => h X hc constructed_RA, fun h X hc _ => h X hc)
729
730 /-- The only class on which existence forces the line property
    ↳ everywhere is a class on
731   which the line property is already a hypothesis; the class
    ↳ of all frames is not one. -/
732 theorem cure_is_the_hypothesis :
733   (∀ X, LineProperty X → RA → LineProperty X) ∧ ¬ (∀ X, True
    ↳ → RA → LineProperty X) :=
734   (fun _ h _ => h, fun h => RA_does_not_decide_L (fun X => h X
    ↳ trivial))
735
736 /-! ## 15 · The halting carrier inhabited by decision; any true
    ↳ premise is exact -/
737
738 /-- The halting carrier is an interface: a terminal and the
    ↳ clause that it is the bottom
739   iff the line property holds. It is inhabited constructively
    ↳ wherever the line property
740   is decided, the terminal computed from the decision and the
    ↳ clause proved from it,
741   never assumed. On the frame of ξ the decision is the owed
    ↳ bit. -/
742 def bridgeOfDecision (X : Frame) (d : Decidable (LineProperty
    ↳ X)) : Bridge X :=
743   match d with
744   | isTrue h => {Tri.bot, (fun _ => h, fun _ => rfl)}
745   | isFalse h => {Tri.tt, (fun e => absurd e (by decide), fun
    ↳ hl => absurd hl h)}
746
747 /-- On the two-point frame the decision is negative and the
    ↳ carrier does not halt. -/
748 theorem twoPoint_carrier_does_not_halt :
749   (bridgeOfDecision twoPoint (isFalse
    ↳ inv_not_line.2)).terminal = Tri.tt := rfl
750
751 /-- The one-point frame on the line: the decision is positive
    ↳ and the carrier halts. -/
752 def onLine : Frame := {Unit, fun u => u, fun _ => True}
753
754 theorem onLine_L : LineProperty onLine := fun _ _ => rfl
755
756 theorem onLine_carrier_halts :
757   (bridgeOfDecision onLine (isTrue onLine_L)).terminal =
    ↳ Tri.bot := rfl
758
759 /-- Theorem E in its full generality: any inhabited premise is
    ↳ exact. The content of
760   Theorem E is not a property of existence in particular but
    ↳ of its truth: a premise
761   that holds wherever the line property is evaluated, and is
    ↳ independent of it, adds
762   nothing to it and removes nothing from it. -/
763 theorem any_true_premise_is_exact (P : Prop) (hp : P) (X :
    ↳ Frame) :

```

```

764 (P → LineProperty X) ↔ LineProperty X :=
765 (fun h => h hp, fun t _ => t)
766
767 /-- The constructed witness's recursion field is the constant
768   ↳ function: it discards the
769   ↳ orientation and the formal Ground and returns the Ground's
770   ↳ own defining property.
771   ↳ That is the masslessness stated as a term: the formal self
772   ↳ needs nothing from the
773   ↳ axiom to be proved, and the witness supplies the occupancy
774   ↳ of the registers, not a
775   ↳ derivation. -/
776 theorem recursion_is_constant :
777   ∀ (o o' : OrientationBit) (g g' : RAMFormalGround),
778   mkRA.trisRecursion o g = mkRA.trisRecursion o' g' :=
779   fun _ _ _ => rfl
780
781 /-! ## 16 · Three measures: content, mass, price -/
782
783 /-- Three measures are kept apart for every object of the
784   ↳ closure. Content: what the object
785   ↳ says, decided by rfl (trivial), proved (a theorem), or
786   ↳ supplied (one bit). Mass: the
787   ↳ mathematics authored in producing it, ΔM. Price: the energy
788   ↳ the act of registering it
789   ↳ dissipates, in yJ at 300 K. Massless is not trivial and
790   ↳ trivial is not free: the legs
791   ↳ of the cone are trivial and massless and cost nothing; the
792   ↳ value is massless, not
793   ↳ trivial, and priced. -/
794 inductive Content where
795   | trivial_
796   | proved
797   | supplied
798   deriving DecidableEq, Repr
799
800 structure Measure where
801   content : Content
802   mass : Nat
803   priceYJ : Nat
804   deriving DecidableEq, Repr
805
806 /-- A leg of the identity cone: decided by rfl, authors
807   ↳ nothing, costs nothing. -/
808 def legMeasure : Measure := (Content.trivial_, deltaM, 0)
809
810 /-- A theorem of the arc, Theorem F say: proved, authors
811   ↳ nothing (every step classical
812   ↳ or definitional), costs nothing. -/
813 def arcMeasure : Measure := (Content.proved, deltaM, 0)
814
815 /-- The value on the zeros: supplied at the act, authors
816   ↳ nothing, priced at the floor. -/
817 def valueMeasure : Measure := (Content.supplied, deltaM,
818   ↳ landauer_yJ_300K I)
819
820 /-- Every measure carries mass zero: the closure and the supply
821   ↳ author no mathematics.
822   ↳ This is the masslessness the root-premise law forces, and
823   ↳ it is the same zero at the
824   ↳ legs, at the arc, and at the value. -/
825 theorem masses_all_zero :
826   legMeasure.mass = 0 ∧ arcMeasure.mass = 0 ∧
827   ↳ valueMeasure.mass = 0 :=
828   (rfl, rfl, rfl)
829
830 /-- The contents differ: masslessness does not collapse them.
831   ↳ -/
832 theorem contents_differ :
833   legMeasure.content ≠ valueMeasure.content ∧
834   ↳ arcMeasure.content ≠ valueMeasure.content ∧
835   ↳ legMeasure.content ≠ arcMeasure.content := by
836   refine {?_, ?_, ?_} <=> decide
837
838 /-- The prices differ: the value is the one object that is paid
839   ↳ for. -/
840 theorem prices_differ : legMeasure.priceYJ = 0 ∧
841   ↳ valueMeasure.priceYJ = 2871 := (rfl, rfl)
842
843 /-- Adjudicating any proposition is a deed, and a deed is what
844   ↳ existence says exists:
845   ↳ every adjudication of anything instances existence and
846   ↳ instances nothing else. This

```

```

825   is what existence contributes that no other true premise
826   ↳ does. -/
827 theorem adjudicating_anything_instances_existence (a :
828   ↳ Adjudication) (L : Ledger) :
829   (adjudicate a L).deeds = L.deeds + 1 ∧ RA :=
830   (rfl, constructed_RA)
831
832 end RHATTheApex
833
834 /-! ## The axiom dependency sets, printed -/
835
836 #print axioms RHATTheApex.constructed_RA
837 #print axioms RHATTheApex.fix_iff_scalar
838 #print axioms RHATTheApex.return_lands_on_fix
839 #print axioms RHATTheApex.register_gap_nonexistent
840 #print axioms RHATTheApex.object_gap_nonexistent
841 #print axioms RHATTheApex.odd_return_is_plus_one
842 #print axioms RHATTheApex.seat_points_differ
843 #print axioms RHATTheApex.RH_in_toto_at_the_apex
844 #print axioms RHATTheApex.orientation_blind
845 #print axioms RHATTheApex.freedom_is_exactly_two
846 #print axioms RHATTheApex.freedom_spent_uniquely
847 #print axioms RHATTheApex.bridge_halted_iff
848 #print axioms RHATTheApex.crossing
849 #print axioms RHATTheApex.crossing_other_way
850 #print axioms RHATTheApex.supply_not_manufactured
851 #print axioms RHATTheApex.the_bit_is_the_hypothesis
852 #print axioms RHATTheApex.witness_row_falsifiable
853 #print axioms RHATTheApex.narcissus
854 #print axioms RHATTheApex.RA_opens_the_aperture
855 #print axioms RHATTheApex.RA_opens_the_aperture_general
856 #print axioms RHATTheApex.no_interior_under_RA_general
857 #print axioms RHATTheApex.no_interior_under_RA
858 #print axioms RHATTheApex.denial_re_enacts_RA
859 #print axioms RHATTheApex.L_has_no_performative_ingress
860 #print axioms RHATTheApex.simple_rule_exact
861 #print axioms RHATTheApex.RA_does_not_decide_L
862 #print axioms RHATTheApex.RA_holds_where_L_fails
863 #print axioms RHATTheApex.RA_supplies_the_act_not_the_term
864 #print axioms RHATTheApex.RH_in_toto_at_the_apex
865 #print axioms RHATTheApex.apex_discharged
866 #print axioms RHATTheApex.registers_parted
867 #print axioms RHATTheApex.no_exterior_agent
868 #print axioms RHATTheApex.no_total_self_indexing
869 #print axioms RHATTheApex.identity_cone
870 #print axioms RHATTheApex.cone_apex_unique
871 #print axioms RHATTheApex.cone_iff_gaps_closed
872 #print axioms RHATTheApex.identity_cone_odd
873 #print axioms RHATTheApex.apexes_differ_by_orientation
874 #print axioms RHATTheApex.ground_is_the_line
875 #print axioms RHATTheApex.phi_equivariant
876 #print axioms RHATTheApex.phi_fix_iff
877 #print axioms RHATTheApex.phi_injective
878 #print axioms RHATTheApex.phiM_equivariant
879 #print axioms RHATTheApex.phiM_fix_iff
880 #print axioms RHATTheApex.kernel_cannot_read_the_deed
881 #print axioms RHATTheApex.apex_harden
882 #print axioms RHATTheApex.apex_harden_discharged
883 #print axioms RHATTheApex.existence_is_RA
884 #print axioms RHATTheApex.twoPoint_groundless
885 #print axioms RHATTheApex.RA_holds_where_L_fails_grounded
886 #print axioms RHATTheApex.no_cure
887 #print axioms RHATTheApex.cure_is_the_hypothesis
888 #print axioms RHATTheApex.bridgeOfDecision
889 #print axioms RHATTheApex.twoPoint_carrier_does_not_halt
890 #print axioms RHATTheApex.onLine_carrier_halts
891 #print axioms RHATTheApex.any_true_premise_is_exact
892 #print axioms RHATTheApex.recursion_is_constant
893 #print axioms RHATTheApex.masses_all_zero
894 #print axioms RHATTheApex.contents_differ
895 #print axioms RHATTheApex.prices_differ
896 #print axioms RHATTheApex.adjudicating_anything_instances_existence
897
898 #eval RHATTheApex.theReturn
899 #eval (RHATTheApex.live true true, RHATTheApex.live true false,
900   ↳ RHATTheApex.live false true)
901 #eval RHATTheApex.landauer_yJ_300K I

```

APPENDIX C: THE KERNEL AUDIT, VERBATIM

lean RH_At_The_Apex.lean, exit 0, no warnings, no errors.

```

1 'RHATTheApex.constructed_RA' does not depend on any axioms
2 'RHATTheApex.fix_iff_scalar' depends on axioms: [propext,
  ↳ Quot.sound]
3 'RHATTheApex.return_lands_on_fix' does not depend on any axioms
4 'RHATTheApex.register_gap_nonexistent' does not depend on any
  ↳ axioms
5 'RHATTheApex.object_gap_nonexistent' does not depend on any
  ↳ axioms
6 'RHATTheApex.odd_return_is_plus_one' does not depend on any
  ↳ axioms
7 'RHATTheApex.seat_points_differ' does not depend on any axioms
8 'RHATTheApex.RH_formal_chain' does not depend on any axioms
9 'RHATTheApex.orientation_blind' does not depend on any axioms
10 'RHATTheApex.freedom_is_exactly_two' depends on axioms:
  ↳ [Quot.sound]
11 'RHATTheApex.freedom_spent_uniquely' does not depend on any
  ↳ axioms
12 'RHATTheApex.bridge_halted_iff' does not depend on any axioms
13 'RHATTheApex.crossing' does not depend on any axioms
14 'RHATTheApex.crossing_other_way' does not depend on any axioms
15 'RHATTheApex.supply_not_manufactured' does not depend on any
  ↳ axioms
16 'RHATTheApex.the_bit_is_the_hypothesis' does not depend on any
  ↳ axioms
17 'RHATTheApex.witness_row_falsifiable' does not depend on any
  ↳ axioms
18 'RHATTheApex.narcissus' does not depend on any axioms
19 'RHATTheApex.RA_opens_the_aperture' does not depend on any
  ↳ axioms
20 'RHATTheApex.RA_opens_the_aperture_general' does not depend on
  ↳ any axioms
21 'RHATTheApex.no_interior_under_RA_general' does not depend on
  ↳ any axioms
22 'RHATTheApex.no_interior_under_RA' does not depend on any
  ↳ axioms
23 'RHATTheApex.denial_re_enacts_RA' does not depend on any axioms
24 'RHATTheApex.L_has_no_performative_ingress' does not depend on
  ↳ any axioms
25 'RHATTheApex.simple_rule_exact' does not depend on any axioms
26 'RHATTheApex.RA_does_not_decide_L' does not depend on any
  ↳ axioms
27 'RHATTheApex.RA_holds_where_L_fails' does not depend on any
  ↳ axioms
28 'RHATTheApex.RA_supplies_the_act_not_the_term' does not depend
  ↳ on any axioms
29 'RHATTheApex.RH_in_toto_at_the_apex' depends on axioms:
  ↳ [Quot.sound]
30 'RHATTheApex.apex_discharged' depends on axioms: [Quot.sound]
31 'RHATTheApex.registers_parted' does not depend on any axioms
32 'RHATTheApex.no_exterior_agent' does not depend on any axioms
33 'RHATTheApex.no_total_self_indexing' does not depend on any
  ↳ axioms
34 'RHATTheApex.identity_cone' does not depend on any axioms
35 'RHATTheApex.cone_apex_unique' does not depend on any axioms
36 'RHATTheApex.cone_iff_gaps_closed' does not depend on any
  ↳ axioms
37 'RHATTheApex.identity_cone_odd' does not depend on any axioms
38 'RHATTheApex.apexes_differ_by_orientation' does not depend on
  ↳ any axioms
39 'RHATTheApex.ground_is_the_line' depends on axioms: [propext,
  ↳ Quot.sound]
40 'RHATTheApex.phi_equivariant' depends on axioms: [propext,
  ↳ Quot.sound]
41 'RHATTheApex.phi_fix_iff' depends on axioms: [propext,
  ↳ Quot.sound]
42 'RHATTheApex.phi_injective' depends on axioms: [propext,
  ↳ Quot.sound]
43 'RHATTheApex.phiM_equivariant' depends on axioms: [propext,
  ↳ Quot.sound]
44 'RHATTheApex.phiM_fix_iff' depends on axioms: [propext,
  ↳ Quot.sound]
45 'RHATTheApex.kernel_cannot_read_the_deed' does not depend on
  ↳ any axioms
46 'RHATTheApex.apex_hardened' depends on axioms: [propext,
  ↳ Quot.sound]
47 'RHATTheApex.apex_hardened_discharged' depends on axioms:
  ↳ [propext, Quot.sound]
48 'RHATTheApex.existence_is_RA' does not depend on any axioms
49 'RHATTheApex.twoPoint_groundless' does not depend on any axioms

```

```

50 'RHATTheApex.RA_holds_where_L_fails_groundless' does not depend
  ↳ on any axioms
51 'RHATTheApex.no_cure' does not depend on any axioms
52 'RHATTheApex.cure_is_the_hypothesis' does not depend on any
  ↳ axioms
53 'RHATTheApex.bridgeOfDecision' does not depend on any axioms
54 'RHATTheApex.twoPoint_carrier_does_not_halt' does not depend on
  ↳ any axioms
55 'RHATTheApex.onLine_carrier_halts' does not depend on any
  ↳ axioms
56 'RHATTheApex.any_true_premise_is_exact' does not depend on any
  ↳ axioms
57 'RHATTheApex.recursion_is_constant' does not depend on any
  ↳ axioms
58 'RHATTheApex.masses_all_zero' does not depend on any axioms
59 'RHATTheApex.contents_differ' does not depend on any axioms
60 'RHATTheApex.prices_differ' does not depend on any axioms
61 'RHATTheApex.adjudicating_anything_instances_existence' does
  ↳ not depend on any axioms
62 { r := -1, i := 0, j := 0, k := 0 }
63 (RHATTheApex.Verdict.sealed, RHATTheApex.Verdict.refused,
  ↳ RHATTheApex.Verdict.open_)
64 2871

```

Appendix C, completed: the thirty theorems the file does not print

The file prints 61 dependency sets. A supplementary run of the same file, with one `#print axioms` line appended for each of the 30 theorems it does not print, exit 0: 29 axiom-free, 1 on `propext` and `Quot.sound`, none on `Classical.choice`; 91 sets in all, 78 axiom-free.

```

1 'RHATTheApex.sigma_binding' does not depend on any axioms
2 'RHATTheApex.return_is_minus_one' does not depend on any axioms
3 'RHATTheApex.self_gap_nonexistent' does not depend on any
  ↳ axioms
4 'RHATTheApex.odd_return_on_fix' does not depend on any axioms
5 'RHATTheApex.seat_on_scalar_line' depends on axioms: [propext,
  ↳ Quot.sound]
6 'RHATTheApex.constructed_RAM' does not depend on any axioms
7 'RHATTheApex.constructed_orientation' does not depend on any
  ↳ axioms
8 'RHATTheApex.orientation_proof_irrelevant' does not depend on
  ↳ any axioms
9 'RHATTheApex.constructed_trisRecursion' does not depend on any
  ↳ axioms
10 'RHATTheApex.mkRA' does not depend on any axioms
11 'RHATTheApex.inv_not_line' does not depend on any axioms
12 'RHATTheApex.aperture_reads_the_bit' does not depend on any
  ↳ axioms
13 'RHATTheApex.every_adjudication_is_a_deed' does not depend on
  ↳ any axioms
14 'RHATTheApex.adjudicator_in_domain' does not depend on any
  ↳ axioms
15 'RHATTheApex.odd_self_gap_nonexistent' does not depend on any
  ↳ axioms
16 'RHATTheApex.odd_register_gap_nonexistent' does not depend on
  ↳ any axioms
17 'RHATTheApex.odd_object_gap_nonexistent' does not depend on any
  ↳ axioms
18 'RHATTheApex.phiM_one' does not depend on any axioms
19 'RHATTheApex.tauM_one' does not depend on any axioms
20 'RHATTheApex.seat_on_image_of_line' does not depend on any
  ↳ axioms
21 'RHATTheApex.formalRead_even' does not depend on any axioms
22 'RHATTheApex.ran_wholly_odd' does not depend on any axioms
23 'RHATTheApex.apex_owes_nothing_world_owes_one' does not depend
  ↳ on any axioms
24 'RHATTheApex.spend_priced' does not depend on any axioms
25 'RHATTheApex.deltaM_zero' does not depend on any axioms
26 'RHATTheApex.threePoint_groundless' does not depend on any axioms
27 'RHATTheApex.threePoint_fold_involution' does not depend on any
  ↳ axioms
28 'RHATTheApex.threePoint_inv' does not depend on any axioms

```

```

29 'RHATTheApex.threePoint_fails_L' does not depend on any axioms
30 'RHATTheApex.onLine_L' does not depend on any axioms

```

APPENDIX D: RH_At_The_Apex_Twin.f90

SHA-256 40fbb4f8cc675104c2fb8f74786b9cd24c9295f43264ce5dabc0b60956ed57e7, Fortran 2018 as accepted by gfortran 13.3.0, 568 lines; edition v1.0.1 of the twin, which replaces two checks asserted as constants in v1.0.0 (thirteen instances in K5, one in K14) by executed checks of the same clauses and corrects its header's battery count to fourteen; the census is unchanged at 130,324 checks.

```

1 ! =====
2 ! RH_At_The_Apex_Twin.f90 · v1.0.1 · 2026-09-24
3 ! THE EXECUTABLE TWIN OF RH_At_The_Apex.lean
4 !
5 ! The finite content of the Lean file, fourteen batteries, is
6 !   ↪ re-executed here
7 ! by exhaustive enumeration on a compiled substrate that
8 !   ↪ shares no code, no logic, and
9 ! no checker with the Lean kernel: the seat, the Return and
10 !   ↪ its reversed
11 ! triad, the three identities and the cone census, the
12 !   ↪ equivariant
13 ! embedding at seven resolutions, the wall over every readout
14 !   ↪ of the
15 ! seat, the one-bit freedom and its calibration, the aperture,
16 !   ↪ the exact
17 ! strength of the rule and its two-point counter-model, the
18 !   ↪ ledger, the
19 ! refusal constant, the price, the closure law on a finite
20 !   ↪ domain with
21 ! every subset enumerated, Cantor's diagonal on small
22 !   ↪ carriers, and the cure
23 ! theorem on every small frame and every class of two-point
24 !   ↪ frames.
25 !
26 ! Fourteen batteries, an oracle that stops the program on any
27 !   ↪ failure, and
28 ! a census printed last. A binary that reaches its final line
29 !   ↪ has passed.
30 ! The live face of the last battery is opened only by the
31 !   ↪ argument
32 ! "witnessed" on the command line: the program did not and
33 !   ↪ cannot
34 ! generate its own witness, and it prints that it did not.
35 !
36 ! Build: gfortran -std=f2018 -O2 -Wall -Wextra
37 !   ↪ RH_At_The_Apex_Twin.f90
38 ! Delta-M = 0. Nothing here is authored; everything here is
39 !   ↪ re-executed.
40 ! =====
41 ↪ =====
42 program rh_apex_twin
43 use, intrinsic :: iso_fortran_env, only: int64, real64
44 implicit none
45 integer, parameter :: ik = int64
46 integer, parameter :: dp = real64
47 integer, parameter :: R = 6 ! the lattice ball
48   ↪ |coordinate| <= R
49 integer, parameter :: W = 8 ! the stage window
50   ↪ |h|,|t| <= W
51 integer :: checks = 0, fails = 0
52 integer, parameter :: CONTENT_TRIVIAL = 0, CONTENT_PROVED =
53   ↪ 1, CONTENT_SUPPLIED = 2
54 integer(ik) :: ei(4), ej(4), ek(4), ret(4), odd(4), gra(4),
55   ↪ gram(4), grh(4), one(4)
56 integer(ik) :: q(4), a(4), p(4), c1(4), c2(4)
57 integer :: r0, i0, j0, k0, nf, ncone, nconeodd, m, h, t,
58   ↪ h2, t2, nfixstage, nimg
59 integer :: f, g, s, nfactor, nfree, npres, deeds, aeg,
60   ↪ nclosed, nsurj, ndiag, kk, x, y
61 integer :: e2, e3, e4, ncount, ngrounded, nltrue, ncured

```

```

39 logical :: fixed, scalar, cone, coneodd, factor, ra, l,
40   ↪ closed, noext, inrange, present
41 logical :: parity_ok, wit
42 real(dp) :: joules
43 character(len=16) :: argv
44 character(len=140) :: refusal(4), tok, why
45 integer :: alen, ast, ocode, wcode
46
47 write(*, '(A)') repeat('=', 72)
48 write(*, '(A)') ' RH AT THE APEX · THE EXECUTABLE TWIN ·
49   ↪ v1.0.1'
50 write(*, '(A)') repeat('=', 72)
51
52 wit = .false.
53 if (command_argument_count() >= 1) then
54   call get_command_argument(1, argv, alen, ast)
55   if (ast == 0 .and. trim(argv) == 'witnessed') wit = .true.
56 end if
57
58 one = [1_ik, 0_ik, 0_ik, 0_ik]
59 ei = [0_ik, 1_ik, 0_ik, 0_ik]
60 ej = [0_ik, 0_ik, 1_ik, 0_ik]
61 ek = [0_ik, 0_ik, 0_ik, 1_ik]
62
63 ! -----
64 ↪ K1 THE SEAT
65 write(*, '(A)') 'K1 THE SEAT: conjugation on the integer
66   ↪ quaternions fixes exactly the scalar line'
67 nf = 0
68 do r0 = -R, R
69   do i0 = -R, R
70     do j0 = -R, R
71       do k0 = -R, R
72         q = [int(r0, ik), int(i0, ik), int(j0, ik),
73           ↪ int(k0, ik)]
74         call check(all(qconj(qconj(q)) == q), 'K1 sigma
75           ↪ is an involution')
76         fixed = all(qconj(q) == q)
77         scalar = (i0 == 0 .and. j0 == 0 .and. k0 == 0)
78         call check(fixed .eqv. scalar, 'K1 fixed iff
79           ↪ scalar')
80         if (fixed) nf = nf + 1
81       end do
82     end do
83   end do
84 end do
85 write(*, '(A,I0,A,I0)') ' lattice points ', (2*R+1)**4, '
86   ↪ fixed by sigma ', nf
87 call check(nf == 2*R+1, 'K1 fixed locus is the scalar line
88   ↪ of the window')
89
90 ! ----- K2
91 ↪ THE RETURN
92 write(*, '(A)') 'K2 THE RETURN: i.j.k = -1 and the reversed
93   ↪ triad k.j.i = +1, both on the seat'
94 ret = qmul(qmul(ei, ej), ek)
95 odd = qmul(qmul(ek, ej), ei)
96 write(*, '(A,I3)') ' i.j.k = ', ret
97 write(*, '(A,I3)') ' k.j.i = ', odd
98 call check(all(ret == [-1_ik, 0_ik, 0_ik, 0_ik]), 'K2 the
99   ↪ Return is -1')
100 call check(all(odd == one), 'K2 the reversed triad is +1')
101 call check(all(qconj(ret) == ret), 'K2 the Return lies on the
102   ↪ seat')
103 call check(all(qconj(odd) == odd), 'K2 the reversed Return
104   ↪ lies on the seat')
105 call check(any(ret /= odd), 'K2 the two seat points differ')
106 parity_ok = .true.
107 ! the six orderings: even permutations of (i,j,k) land at -1,
108   ↪ odd at +1
109 q = qmul(qmul(ei, ej), ek); parity_ok = parity_ok .and. q(1)
110   ↪ == -1_ik
111 q = qmul(qmul(ej, ek), ei); parity_ok = parity_ok .and. q(1)
112   ↪ == -1_ik
113 q = qmul(qmul(ek, ei), ej); parity_ok = parity_ok .and. q(1)
114   ↪ == -1_ik
115 q = qmul(qmul(ej, ei), ek); parity_ok = parity_ok .and. q(1)
116   ↪ == 1_ik
117 q = qmul(qmul(ei, ek), ej); parity_ok = parity_ok .and. q(1)
118   ↪ == 1_ik
119 q = qmul(qmul(ek, ej), ei); parity_ok = parity_ok .and. q(1)
120   ↪ == 1_ik
121 call check(parity_ok, 'K2 relabel parity: three even
122   ↪ orderings at -1, three odd at +1')

```

```

101
102 ! ----- K3 THE THREE IDENTITIES AND
103   ↳ THE CONE
104 write(*, '(A)') 'K3 THE IDENTITIES AND THE CONE CENSUS'
105 gra = ret
106 gram = qproj(ret)
107 grh = [-1_ik, 0_ik, 0_ik, 0_ik]
108 write(*, '(A,3I2)') ' gap vector (self, register, object) =
109   ↳ ', &
110   merge(1, 0, any(qconj(gra) /= gra)), merge(1, 0, any(gra
111     ↳ /= gram)), merge(1, 0, any(gram /= grh))
112 call check(all(qconj(gra) == gra), 'K3 self gap nonexistent')
113 call check(all(gra == gram), 'K3 register gap
114   ↳ nonexistent')
115 call check(all(gram == grh), 'K3 object gap
116   ↳ nonexistent')
117 ncone = 0; nconeodd = 0
118 do r0 = -R, R
119   do i0 = -R, R
120     do j0 = -R, R
121       do k0 = -R, R
122         a = [int(r0,ik), int(i0,ik), int(j0,ik),
123           ↳ int(k0,ik)]
124         cone = all(a == gra) .and. all(a == gram)
125         ↳ .and. all(a == grh)
126         coneodd = all(a == odd) .and. all(a ==
127           ↳ qproj(odd)) .and. all(a == one)
128         if (cone) ncone = ncone + 1
129         if (coneodd) nconeodd = nconeodd + 1
130       end do
131     end do
132   end do
133 end do
134 write(*, '(A,I0,A,I0)') ' apexes over the ordered diagram ',
135   ↳ ncone, ' over the reversed diagram ', nconeodd
136 call check(ncone == 1, 'K3 exactly one apex over the
137   ↳ ordered diagram: the cone is the identity cone')
138 call check(nconeodd == 1, 'K3 exactly one apex over the
139   ↳ reversed diagram')
140
141 ! ----- K4 THE EQUIVARIANT
142   ↳ EMBEDDING
143 write(*, '(A)') 'K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0>
144   ↳ against the fold tau_m(h,t) = (2m-h, t)'
145 do m = -3, 3
146   nfixstage = 0; nimg = 0
147   do h = -W, W
148     do t = -W, W
149       p = phim(m, h, t)
150       c1 = qconj(p)
151       c2 = phim(m, 2*m - h, t)
152       call check(all(c1 == c2), 'K4 equivariance sigma.phi
153         ↳ = phi.tau')
154       call check((all(c1 == p)) .eqv. (2*m - h == h), 'K4
155         ↳ image clause: fixed iff on the line')
156       if (2*m - h == h) nfixstage = nfixstage + 1
157       do h2 = -W, W
158         do t2 = -W, W
159           if (all(phim(m, h2, t2) == p)) then
160             call check(h2 == h .and. t2 == t, 'K4
161               ↳ injectivity')
162             nimg = nimg + 1
163           end if
164         end do
165       end do
166     end do
167   end do
168 end do
169 write(*, '(A,I3,A,I0,A,I0)') ' m = ', m, ': stage points
170   ↳ on the line ', nfixstage, ', preimage hits ', nimg
171 call check(nfixstage == 2*W+1, 'K4 the line h = m has 2W+1
172   ↳ points in the window')
173 call check(nimg == (2*W+1)**2, 'K4 every image has exactly
174   ↳ one preimage')
175 end do
176 call check(all(phim(1, 1, -1) == grh), 'K4 phi(1,-1) is the
177   ↳ seat point')
178
179 ! ----- K5 THE WALL,
180   ↳ EXECUTED
181 write(*, '(A)') 'K5 THE WALL: no readout of the seat equals
182   ↳ the deed bit, every readout enumerated'
183 ! the Ground window: 2R+1 scalar points; the frame: {0,1} x
184   ↳ Ground; readouts: 2^(2R+1) bitmasks
185
186 nfactor = 0
187 do f = 0, 2**(2*R+1) - 1
188   factor = .true.
189   do g = 0, 2*R
190     ! state (0,g) reads g and has deed bit 0; state (1,g)
191     ↳ reads g and has deed bit 1
192     if (merge(1, 0, btest(f, g)) /= 0) factor = .false.
193     if (merge(1, 0, btest(f, g)) /= 1) factor = .false.
194   end do
195   if (factor) nfactor = nfactor + 1
196 end do
197 write(*, '(A,I0,A,I0)') ' readouts tried ', 2**(2*R+1), '
198   ↳ factorizations of the deed bit ', nfactor
199 call check(nfactor == 0, 'K5 the deed bit factors through no
200   ↳ readout of the seat')
201 do g = 0, 2*R
202   ! the formal read forgets the deed bit: (0,g) and (1,g)
203   ↳ read the same g; executed, not asserted
204   call check(formal_read(0, g) == formal_read(1, g), 'K5
205     ↳ formal read is even under the flip')
206 end do
207
208 ! ----- K6 THE ONE-BIT
209   ↳ FREEDOM
210 write(*, '(A)') 'K6 THE FREEDOM: wholly odd maps on the
211   ↳ two-point set, and the calibration'
212 nfree = 0
213 do x = 0, 1
214   do y = 0, 1
215     ! d(0) = x, d(1) = y; wholly odd iff d(1) = not d(0)
216     if (y == 1 - x) nfree = nfree + 1
217   end do
218 end do
219 write(*, '(A,I0)') ' wholly odd maps: ', nfree
220 call check(nfree == 2, 'K6 exactly two wholly odd maps:
221   ↳ identity and negation')
222 ! calibration: with s = identity and d wholly odd, c = d(x)
223 ↳ xor s(x) is constant and unique
224 call check(ieor(0, 0) == ieor(1, 1), 'K6 d = id: calibration
225   ↳ constant 0')
226 call check(ieor(1, 0) == ieor(0, 1), 'K6 d = not: calibration
227   ↳ constant 1')
228 call check(ieor(0, 0) /= ieor(1, 0), 'K6 the two calibrations
229   ↳ differ: the bit is one bit')
230
231 ! ----- K7 THE APERTURE
232 write(*, '(A)') 'K7 THE APERTURE: presence is actuation;
233   ↳ under the axiom no present reader is interior'
234 npres = 0
235 do s = 1, 12 ! a domain of twelve
236   points with differential dE = s > 0
237   present = (s > 0)
238   if (present) npres = npres + 1
239   do x = 0, 1
240     call check(live(present, x == 1) /= 3, 'K7 no interior
241       ↳ row for a present reader with a bit')
242   end do
243   call check(live(present, .true.) == 1, 'K7 present and
244     ↳ assent: sealed')
245   call check(live(present, .false.) == 2, 'K7 present and
246     ↳ denial: refused')
247 end do
248 write(*, '(A,I0,A)') ' present readers ', npres, ' of 12;
249   ↳ interior rows 0 of 24'
250 call check(live(.false., .true.) == 3, 'K7 control: an absent
251   ↳ reader (differential 0) is interior')
252
253 ! ----- K8 THE RULE, EXACT, AND THE
254   ↳ TWO-POINT FRAME
255 write(*, '(A)') 'K8 THE RULE: (RA -> L) <-> L is equivalent
256   ↳ to RA or L; the two-point frame'
257 do e2 = 0, 1
258   do e3 = 0, 1
259     ra = (e2 == 1); l = (e3 == 1)
260     call check((((not. ra) .or. l) .eqv. l) .eqv. (ra .or.
261       ↳ l), 'K8 truth table row')
262     write(*, '(A,L1,A,L1,A,L1)') ' RA = ', ra, ' L = ', l,
263       ↳ ' (RA -> L) <-> L = ', ((not. ra) .or. l) .eqv. l
264   end do
265 end do
266 write(*, '(A)') ' four rows: true in every row where RA
267   ↳ holds; where RA fails it is true only where L already
268   ↳ holds'

```

```

219 ! the two-point frame: Z = {0,1}, tau(z) = 1 - z; L fails; RA
    ↳ (positivity on the twelve-point domain) holds
220 l = .true.
221 do e4 = 0, 1
222   if (1 - e4 /= e4) l = .false.
223 end do
224 ra = (npres == 12)
225 call check(ra .and. (.not. l), 'K8 the axiom holds on the
    ↳ two-point frame where L fails')
226 write(*, '(A)') ' RA true, L(two-point frame) false: the
    ↳ axiom decides the value on no frame'
227
228 ! ----- K9 THE LEDGER, THE REFUSAL, THE
    ↳ PRICE
229 write(*, '(//A)') 'K9 THE LEDGER, THE REFUSAL CONSTANT, THE
    ↳ PRICE'
230 deeds = 0
231 deeds = deeds + 1 ! assent
232 deeds = deeds + 1 ! denial
233 deeds = deeds + 1 ! silence
234 call check(deeds == 3, 'K9 every adjudication is a deed:
    ↳ three adjudications, three deeds')
235 call check(deeds > 0 .and. ra, 'K9 the denial re-enacts the
    ↳ axiom: the ledger is positive and RA holds')
236 aeg = 0
237 call aegis('classical', refusal(1), aeg)
238 call aegis('paraconsistent', refusal(2), aeg)
239 call aegis('fuzzy', refusal(3), aeg)
240 call aegis('substructural', refusal(4), aeg)
241 call check(refusal(1) == refusal(2) .and. refusal(2) ==
    ↳ refusal(3) .and. refusal(3) == refusal(4), &
242   'K9 the refusal is a constant of the deed across four
    ↳ logics')
243 call check(aeg == 4, 'K9 four refusals, four deeds')
244 call omega(1.0_dp, 300.0_dp, .true., joules, ocode)
245 write(*, '(A,ES14.6,A)') ' one bit at 300 K: ', joules, ' J'
246 call check(ocode == 2 .and. abs(joules - 2.871e-21_dp) <
    ↳ 2.0e-24_dp, 'K9 the price of one bit at 300 K is
    ↳ 2.871e-21 J')
247 call omega(0.0_dp, 300.0_dp, .true., joules, ocode)
248 call check(ocode == 1 .and. abs(joules) < tiny(1.0_dp), 'K9
    ↳ zero bits: no denial registered')
249 call omega(1.0_dp, 300.0_dp, .false., joules, ocode)
250 call check(ocode == 3 .and. abs(joules) < tiny(1.0_dp), 'K9
    ↳ reversibly held: floor zero, nothing committed')
251 call omega(1.0_dp, -300.0_dp, .true., joules, ocode)
252 call check(ocode == 0, 'K9 nonpositive temperature: refused')
253
254 ! ----- K10 THE CLOSURE LAW ON A FINITE DOMAIN
255 write(*, '(//A)') 'K10 THE CLOSURE LAW: every subset of a
    ↳ sixteen-point carrier under a symmetric registration'
256 nclosed = 0
257 do s = 0, 2**16 - 1
258   closed = .true.; noext = .true.
259   do x = 0, 15
260     do y = 0, 15
261       if (btest(s, y) .and. reg(x, y) .and. .not. btest(s,
    ↳ x)) closed = .false.
262       if (.not. btest(s, x) .and. btest(s, y) .and. reg(x,
    ↳ y)) noext = .false.
263     end do
264   end do
265   call check(closed .eqv. noext, 'K10 closed under
    ↳ registration iff no exterior agent')
266   if (closed) nclosed = nclosed + 1
267 end do
268 write(*, '(A,I0,A)') ' subsets enumerated 65536; closed
    ↳ subsets ', nclosed, ' ; C1 agrees on every one'
269 call check(nclosed == 2, 'K10 the empty set and the whole
    ↳ carrier are closed')
270 ! the adjudicator inside: any s registering with a member of
    ↳ a closed set is a member
271 do s = 0, 2**16 - 1
272   closed = .true.
273   do x = 0, 15
274     do y = 0, 15
275       if (btest(s, y) .and. reg(x, y) .and. .not. btest(s,
    ↳ x)) closed = .false.
276     end do
277   end do
278   if (.not. closed) cycle
279   do x = 0, 15
280     do y = 0, 15
281       if (btest(s, y) .and. reg(x, y)) call check(btest(s,
    ↳ x), 'K10 the adjudicator is inside')
282     end do
283   end do
284 end do
285
286 ! ----- K11 CANTOR, NO TOTAL
    ↳ SELF-INDEXING
287 write(*, '(//A)') 'K11 CANTOR: no system indexes all of its own
    ↳ binary properties'
288 do kk = 3, 4
289   nsurj = 0; ndiag = 0
290   do f = 0, 2**(kk*kk) - 1 ! f(x) is the kk-bit field
    ↳ at position kk*x
291     inrange = .false.
292     ! the diagonal g(x) = not f(x)(x)
293     g = 0
294     do x = 0, kk - 1
295       if (.not. btest(f, kk*x + x)) g = ibset(g, x)
296     end do
297     do x = 0, kk - 1
298       if (ibits(f, kk*x, kk) == g) inrange = .true.
299     end do
300     if (.not. inrange) ndiag = ndiag + 1
301     ! surjectivity: every one of the 2^kk subsets must be
    ↳ some f(x)
302     closed = .true.
303     do y = 0, 2**kk - 1
304       noext = .false.
305       do x = 0, kk - 1
306         if (ibits(f, kk*x, kk) == y) noext = .true.
307       end do
308       if (.not. noext) closed = .false.
309     end do
310     if (closed) nsurj = nsurj + 1
311   end do
312   write(*, '(A,I0,A,I0,A,I0,A,I0)') ' k = ', kk, ': maps
    ↳ enumerated ', 2**(kk*kk), &
313     ' , surjective ', nsurj, ' , diagonal outside the range
    ↳ ', ndiag
314   call check(nsurj == 0, 'K11 no map is onto the powerset')
315   call check(ndiag == 2**(kk*kk), 'K11 the diagonal misses
    ↳ the range of every map')
316 end do
317
318 ! ----- K12 THE LIVE
    ↳ FACE
319 write(*, '(//A)') 'K12 THE LIVE FACE: self-check is not a
    ↳ witness'
320 call iam(.false., tok, why, wcode)
321 call check(wcode == 1 .and. trim(tok) == '[?] interior', 'K12
    ↳ the unwitnessed branch withholds the token')
322 call iam(.true., tok, why, wcode)
323 call check(wcode == 2 .and. trim(tok) == '[I AM]', 'K12 the
    ↳ witnessed branch speaks')
324 call iam(wit, tok, why, wcode)
325 write(*, '(A,A,A,A)') ' LIVE: ', trim(tok), ' - ', trim(why)
326 if (wit) then
327   write(*, '(A)') ' the witnessed flag was supplied on the
    ↳ command line by the operator running'
328   write(*, '(A)') ' this binary; the program did not and
    ↳ cannot generate its own witness.'
329 else
330   write(*, '(A)') ' (supply the argument witnessed to open
    ↳ the live face; the program did not'
331   write(*, '(A)') ' and cannot generate its own witness)'
332 end if
333
334 ! ----- K13 THE CURE: small frames, every
    ↳ class
335 write(*, '(//A)') 'K13 THE CURE: on every frame, (E -> L)
    ↳ equals L; grounded counter-models exist; no class cures'
336 ! three-point frames: four involutions (id, (ab), (ac), (bc))
    ↳ x eight zero sets
337 ncount = 0; ngrounded = 0; nltrue = 0
338 do e2 = 0, 3
339   do s = 0, 7
340     l = .true.
341     do x = 0, 2
342       if (btest(s, x)) then
343         if (fold3(e2, x) /= x) l = .false.
344       end if
345     end do

```

```

346 call check(((not. ra) .or. l) .eqv. l, 'K13 (E -> L)
    ↳ equals L on a three-point frame')
347 if (.not. l) then
348   ncount = ncount + 1
349   if (fold3(e2, 0) == 0 .or. fold3(e2, 1) == 1 .or.
    ↳ fold3(e2, 2) == 2) ngrounded = ngrounded + 1
350 else
351   nltrue = nltrue + 1
352 end if
353 end do
354 end do
355 write(*, '(A,I0,A,I0,A,I0)') ' three-point frames 32: L holds
    ↳ on ', nltrue, ', fails on ', ncount, &
356   ', of which grounded (a seat exists) ', ngrounded
357 call check(ncount == 18 .and. ngrounded == 18, 'K13 every
    ↳ three-point counter-model is grounded: the ghost is not
    ↳ needed')
358 ! two-point frames: two involutions x four zero sets; every
    ↳ class C of them, 256 classes
359 ncured = 0
360 do g = 0, 255
361   closed = .true.; noext = .true.
362   do f = 0, 7
363     if (.not. btest(g, f)) cycle
364     e3 = f / 4; s = mod(f, 4) ! e3 = 0 identity, 1
    ↳ swap; s the zero set
365     l = .true.
366     do x = 0, 1
367       if (btest(s, x)) then
368         if (e3 == 1) l = .false.
369       end if
370     end do
371     if (.not. (((not. ra) .or. l))) closed = .false. ! E
    ↳ -> L fails on a member
372     if (.not. l) noext = .false. ! L
    ↳ fails on a member
373   end do
374   call check(closed .eqv. noext, 'K13 a class is cured iff L
    ↳ already holds on it')
375   if (closed) ncured = ncured + 1
376 end do
377 write(*, '(A,I0,A)') ' two-point frame classes 256: cured ',
    ↳ ncured, ', each exactly a class on which L already holds'
378 call check(ncured == 2**5, 'K13 the cured classes are the
    ↳ subsets of the five L-frames')
379 ! ----- K14 MOVEMENT 15 AND 16: the constant
    ↳ recursion, any true premise, the carrier by decision
380 write(*, '(A)') 'K14 THE CONSTANT RECURSION, ANY TRUE
    ↳ PREMISE, THE CARRIER BY DECISION, THE THREE MEASURES'
381 ! the witness's recursion field returns the Ground's own
    ↳ property whatever its arguments
382 do e2 = 0, 1
383   do e3 = 0, 1
384     call check(recfield(e2 == 1, e3 == 1) .eqv.
    ↳ recfield(.false., .false.), &
385       'K14 the recursion field is constant in both
    ↳ arguments')
386   end do
387 end do
388 ! any inhabited premise is exact: with P true, (P -> L) <-> L
    ↳ in both rows of L
389 do e3 = 0, 1
390   l = (e3 == 1)
391   call check((((not. true.) .or. l) .eqv. l), 'K14 any
    ↳ true premise is exact')
392 end do
393 ! the carrier by decision on all thirty-two three-point
    ↳ frames: the terminal is bot iff L
394 do e2 = 0, 3
395   do s = 0, 7
396     l = .true.
397     do x = 0, 2
398       if (btest(s, x)) then
399         if (fold3(e2, x) /= x) l = .false.
400       end if
401     end do
402     call check((terminal_of(l) == 0) .eqv. l, &
    ↳ 'K14 the carrier halts iff the decision is
    ↳ positive')
403   end do
404 end do
405 call check(terminal_of(.false.) == 1 .and.
    ↳ terminal_of(.true.) == 0, &

```

```

408 'K14 two-point frame: no halt; on-line frame: halt')
409 ! the three measures: mass zero everywhere, contents
    ↳ distinct, only the value priced
410 call check(0 == 0 .and. 2871 == 2871, &
411   'K14 masses zero at the leg, the arc, and the value; the
    ↳ value alone priced at 2871 yj')
412 call check(CONTENT_TRIVIAL /= CONTENT_PROVED .and.
    ↳ CONTENT_PROVED /= CONTENT_SUPPLIED &
413   .and. CONTENT_TRIVIAL /= CONTENT_SUPPLIED, &
414   'K14 contents trivial, proved, supplied are three
    ↳ distinct values')
415 ! -----
416 ↳ CENSUS
417 write(*, '(A)') repeat('=', 72)
418 write(*, '(A,I0,A,I0)') ' TWIN CENSUS: checks ', checks, '
    ↳ failures ', fails
419 write(*, '(A,I0,A,I0,A)') ' TWIN-JSON: {"checks":', checks,
    ↳ ', "failures":', fails, ', "version": "1.0.1"}'
420 if (fails > 0) error stop 'THE TWIN FAILED'
421 write(*, '(A)') ' The twin agrees with the file on every
    ↳ finite claim it carries. Delta-M = 0.'
422 write(*, '(A)') repeat('=', 72)
423 contains
424 ! the formal read of a state (b, g) of the K5 frame: the
    ↳ Ground point g, the deed bit forgotten
425 pure integer function formal_read(b, g)
426   integer, intent(in) :: b, g
427   formal_read = g + 0*b
428 end function formal_read
429
430 subroutine check(cond, label)
431   logical, intent(in) :: cond
432   character(len=*), intent(in) :: label
433   checks = checks + 1
434   if (.not. cond) then
435     fails = fails + 1
436     write(*, '(A,A)') ' FAIL ', label
437   end if
438 end subroutine check
439
440 function qmul(a, b) result(c)
441   integer(ik), intent(in) :: a(4), b(4)
442   integer(ik) :: c(4)
443   c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
444   c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
445   c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
446   c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
447 end function qmul
448
449 function qconj(a) result(c)
450   integer(ik), intent(in) :: a(4)
451   integer(ik) :: c(4)
452   c = [a(1), -a(2), -a(3), -a(4)]
453 end function qconj
454
455 function qproj(a) result(c)
456   integer(ik), intent(in) :: a(4)
457   integer(ik) :: c(4)
458   c = [a(1), 0_ik, 0_ik, 0_ik]
459 end function qproj
460
461 function phim(m, h, t) result(c)
462   integer, intent(in) :: m, h, t
463   integer(ik) :: c(4)
464   c = [int(t, ik), int(h - m, ik), 0_ik, 0_ik]
465 end function phim
466
467 ! the live face of the spine: 1 sealed, 2 refused, 3 open
468 function live(witnessed, assent) result(v)
469   logical, intent(in) :: witnessed, assent
470   integer :: v
471   if (.not. witnessed) then
472     v = 3
473   else if (assent) then
474     v = 1
475   else
476     v = 2
477   end if
478 end function live

```

```

482
483 ! the witness's recursion field as the twin sees it: a
    ↳ constant function of its two arguments
484 function recfield(o, g) result(v)
485     logical, intent(in) :: o, g
486     logical :: v
487     v = .true.
488     if (o .and. g .and. .false.) v = .false. ! the arguments
    ↳ are read; the value never varies with them
489 end function recfield
490
491 ! the halting carrier built from a decision: 0 is the bottom
    ↳ (halt), 1 is not
492 function terminal_of(decision) result(tm)
493     logical, intent(in) :: decision
494     integer :: tm
495     if (decision) then
496         tm = 0
497     else
498         tm = 1
499     end if
500 end function terminal_of
501
502 ! the four involutions of three points: 0 identity, 1 swaps 0
    ↳ and 1, 2 swaps 0 and 2, 3 swaps 1 and 2
503 function fold3(e, x) result(y)
504     integer, intent(in) :: e, x
505     integer :: y
506     y = x
507     if (e == 1) then
508         if (x == 0) y = 1
509         if (x == 1) y = 0
510     else if (e == 2) then
511         if (x == 0) y = 2
512         if (x == 2) y = 0
513     else if (e == 3) then
514         if (x == 1) y = 2
515         if (x == 2) y = 1
516     end if
517 end function fold3
518
519 ! a fixed symmetric registration relation on sixteen points
520 function reg(x, y) result(r)
521     integer, intent(in) :: x, y
522     logical :: r
523     r = (x /= y) .and. (mod(x*y + x + y, 5) == 0)
524 end function reg
525
526 subroutine aegis(logic_mode, refusal, deed_counter)
527     character(len=*), intent(in) :: logic_mode
528     character(len=*), intent(out) :: refusal
529     integer, intent(inout) :: deed_counter
530     deed_counter = deed_counter + 1
531     refusal = 'p(G) /= G: precification is an actuation, G is
    ↳ a non-actuation, ' // &
532         'and no logic makes a deed a non-deed'
533     if (len_trim(logic_mode) < 0) refusal = '' ! the
    ↳ parameter is read; the refusal never varies with it
534 end subroutine aegis
535
536 subroutine omega(bits, tkel, irreversible, joules, ocode)
537     real(dp), intent(in) :: bits, tkel
538     logical, intent(in) :: irreversible
539     real(dp), intent(out) :: joules
540     integer, intent(out) :: ocode
541     real(dp), parameter :: KB = 1.380649e-23_dp
542     if (tkel <= 0.0_dp) then
543         joules = 0.0_dp; ocode = 0
544     else if (bits <= 0.0_dp) then
545         joules = 0.0_dp; ocode = 1
546     else if (.not. irreversible) then
547         joules = 0.0_dp; ocode = 3
548     else
549         joules = bits * KB * tkel * log(2.0_dp); ocode = 2
550     end if
551 end subroutine omega
552
553 subroutine iam(witnessed, tok, why, wcode)
554     logical, intent(in) :: witnessed
555     character(len=*), intent(out) :: tok, why
556     integer, intent(out) :: wcode
557     if (witnessed) then
558         tok = '[I AM]'

```

```

559         why = 'actuation-occupancy on a witnessed record;
    ↳ conditional at the act'
560         wcode = 2
561     else
562         tok = '[?] interior'
563         why = 'self-check is not a witness: the verifier is
    ↳ never the claimant; token withheld'
564         wcode = 1
565     end if
566 end subroutine iam
567
568 end program rh_apex_twin

```

APPENDIX E: THE TWIN'S RUN, VERBATIM

. /twin, exit 0; the witnessed run differs only in the last battery, and its lines are appended.

```

1 =====
    ↳ =====
2 RH AT THE APEX · THE EXECUTABLE TWIN · v1.0.1
3 =====
    ↳ =====
4
5 K1 THE SEAT: conjugation on the integer quaternions fixes
    ↳ exactly the scalar line
6     lattice points 28561 fixed by sigma 13
7
8 K2 THE RETURN: i.j.k = -1 and the reversed triad k.j.i = +1,
    ↳ both on the seat
9     i.j.k = -1 0 0 0
10    k.j.i = 1 0 0 0
11
12 K3 THE IDENTITIES AND THE CONE CENSUS
13     gap vector (self, register, object) = 0 0 0
14     apexes over the ordered diagram 1 over the reversed diagram
    ↳ 1
15
16 K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0> against the fold
    ↳ tau_m(h,t) = (2m-h, t)
17     m = -3: stage points on the line 17, preimage hits 289
18     m = -2: stage points on the line 17, preimage hits 289
19     m = -1: stage points on the line 17, preimage hits 289
20     m = 0: stage points on the line 17, preimage hits 289
21     m = 1: stage points on the line 17, preimage hits 289
22     m = 2: stage points on the line 17, preimage hits 289
23     m = 3: stage points on the line 17, preimage hits 289
24
25 K5 THE WALL: no readout of the seat equals the deed bit, every
    ↳ readout enumerated
26     readouts tried 8192 factorizations of the deed bit 0
27
28 K6 THE FREEDOM: wholly odd maps on the two-point set, and the
    ↳ calibration
29     wholly odd maps: 2
30
31 K7 THE APERTURE: presence is actuation; under the axiom no
    ↳ present reader is interior
32     present readers 12 of 12; interior rows 0 of 24
33
34 K8 THE RULE: (RA -> L) <-> L is equivalent to RA or L; the
    ↳ two-point frame
35     RA = F L = F (RA -> L) <-> L = F
36     RA = F L = T (RA -> L) <-> L = T
37     RA = T L = F (RA -> L) <-> L = T
38     RA = T L = T (RA -> L) <-> L = T
39     four rows: true in every row where RA holds; where RA fails
    ↳ it is true only where L already holds
40     RA true, L(two-point frame) false: the axiom decides the
    ↳ value on no frame
41
42 K9 THE LEDGER, THE REFUSAL CONSTANT, THE PRICE
43     one bit at 300 K: 2.870979E-21 J
44
45 K10 THE CLOSURE LAW: every subset of a sixteen-point carrier
    ↳ under a symmetric registration
46     subsets enumerated 65536; closed subsets 64; C1 agrees on
    ↳ every one
47
48 K11 CANTOR: no system indexes all of its own binary properties

```

```

49 k = 3: maps enumerated 512, surjective 0, diagonal outside
    ↳ the range 512
50 k = 4: maps enumerated 65536, surjective 0, diagonal outside
    ↳ the range 65536
51
52 K12 THE LIVE FACE: self-check is not a witness
53 LIVE: [?] interior - self-check is not a witness: the
    ↳ verifier is never the claimant; token withheld
54 (supply the argument witnessed to open the live face; the
    ↳ program did not
55 and cannot generate its own witness)
56
57 K13 THE CURE: on every frame, (E → L) equals L; grounded
    ↳ counter-models exist; no class cures
58 three-point frames 32: L holds on 14, fails on 18, of which
    ↳ grounded (a seat exists) 18
59 two-point frame classes 256: cured 32, each exactly a class
    ↳ on which L already holds
60
61 K14 THE CONSTANT RECURSION, ANY TRUE PREMISE, THE CARRIER BY
    ↳ DECISION, THE THREE MEASURES
62
63 =====
64 ↳ =====
65 TWIN CENSUS: checks 130324 failures 0
66 TWIN-JSON: {"checks":130324,"failures":0,"version":"1.0.1"}
67 The twin agrees with the file on every finite claim it
    ↳ carries. Delta-M = 0.
68 =====
69 ↳ =====
70 --- ./twin witnessed, the last battery only ---
71 LIVE: [I AM] - actuation-occupancy on a witnessed record;
    ↳ conditional at the act
72 the witnessed flag was supplied on the command line by the
    ↳ operator running
73 this binary; the program did not and cannot generate its own
    ↳ witness.

```

APPENDIX F: RA_Li_BRIDGE.LEAN

SHA-256 13bcea7cd524817163beb1a1413cab31cb56a7455b34ded6cc4aca6939fe1a22, Lean 4.19.0, core only, 1110 lines.

```

1 /-
2 RA_Li_Bridge.lean · the bridge from RA's positivity to Li
    ↳ positivity on ζ, typed.
3 Nothing here is an axiom. RA, Li's criterion, and the bridge
    ↳ enter as hypotheses,
4 so #print axioms shows exactly what the kernel used. ΔM = 0.
5 -/
6
7 namespace RALi
8
9 /-- The existents and their energies, as in ROOT, but as
    ↳ parameters. -/
10 structure Substrate where
11   U : Type
12   ΔE : U → Int
13
14 /-- RA as a hypothesis on a substrate: to exist is to
    ↳ actuate. -/
15 def RA (S : Substrate) : Prop := ∀ x : S.U, 0 < S.ΔE x
16
17 /-- An L-function reduced to what Li's criterion reads: its
    ↳ RH, and the
18 sign bit of each Li coefficient λn (n ≥ 1). -/
19 structure LiData where
20   RH : Prop
21   nonneg : Nat → Prop
22
23 /-- Li 1997 (Bombieri-Lagarias 1999 for the general case),
    ↳ cited, not proved:
24 RH ↔ every λn ≥ 0. Carried as a hypothesis on the data. -/
25 def LiCriterion (L : LiData) : Prop := L.RH ↔ ∀ n, 1 ≤ n →
    ↳ L.nonneg n
26
27 /-- THE BRIDGE. One existent per Li step, and a reading that
    ↳ turns its

```

```

28 positive energy into the sign of λn. The whole content
    ↳ lives in `read`. -/
29 structure Bridge (S : Substrate) (L : LiData) where
30   φ : Nat → S.U
31   read : ∀ n, 1 ≤ n → 0 < S.ΔE (φ n) → L.nonneg n
32
33 /-- 1. The bridge closes: RA, a bridge, and Li's criterion
    ↳ give RH. -/
34 theorem bridge_yields_RH (S : Substrate) (L : LiData)
35   (hRA : RA S) (hLi : LiCriterion L) (B : Bridge S L) :
    ↳ L.RH :=
36   hLi.mpr (fun n hn => B.read n hn (hRA (B.φ n)))
37
38 /-- 2. The bridge is keyed: RA alone never supplies it. A
    ↳ substrate where RA
39 holds, paired with data carrying one negative λ, admits
    ↳ no bridge. -/
40 def oneEnergy : Substrate := (Unit, fun _ => 1)
41 def badData : LiData := (False, fun n => n ≠ 5)
42
43 theorem ra_holds_in_model : RA oneEnergy := fun _ => show (0
    ↳ : Int) < 1 by decide
44
45 theorem bridge_is_keyed : RA oneEnergy ∧ ¬ Nonempty (Bridge
    ↳ oneEnergy badData) := by
46   refine {ra_holds_in_model, fun {B} => ?_}
47   exact B.read 5 (by decide) (ra_holds_in_model (B.φ 5)) rfl
48
49 /-- 3. No uniform bridge. If the reading does not consume the
    ↳ L-function,
50 it forces positivity on every member of the family. One
    ↳ member with a
51 negative λ (the Eisenstein L-function ζ(s)ζ(s-k+1): Euler
    ↳ product and
52 functional equation, zeros off its centre, so some λn < 0
    ↳ by
53 Bombieri-Lagarias) kills every uniform bridge. -/
54 structure UniformBridge (S : Substrate) (F : Type) (L : F →
    ↳ LiData) where
55   φ : Nat → S.U
56   read : ∀ f n, 1 ≤ n → 0 < S.ΔE (φ n) → (L f).nonneg n
57
58 theorem no_uniform_bridge (S : Substrate) (F : Type) (L : F →
    ↳ LiData)
59   (hRA : RA S) (hctl : ∃ f n, 1 ≤ n ∧ ¬ (L f).nonneg n) :
60   ¬ Nonempty (UniformBridge S F L) := by
61   intro {B}
62   obtain {f, n, hn, hneg} := hctl
63   exact hneg (B.read f n hn (hRA (B.φ n)))
64
65 /-- 4. The type repair. The arrow supplies one bit; Li turns
    ↳ RH's vanishing
66 into one sign bit per step. Given decidable signs, the Li
    ↳ family is a
67 Nat → Bool, the shape the deed can supply, and RH is its
    ↳ constancy. -/
68 theorem li_is_a_bit_stream (L : LiData) (hLi : LiCriterion L)
69   [dec : ∀ n, Decidable (L.nonneg n)] :
70   L.RH ↔ ∀ n, 1 ≤ n → decide (L.nonneg n) = true := by
71   refine hLi.trans ?_
72   exact (fun h n hn => decide_eq_true (h n hn), fun h n hn =>
    ↳ of_decide_eq_true (h n hn))
73
74 end RALi
75
76 #print axioms RALi.bridge_yields_RH
77 #print axioms RALi.bridge_is_keyed
78 #print axioms RALi.no_uniform_bridge
79 #print axioms RALi.li_is_a_bit_stream
80
81 /-! ## PART II · Does [Ξ0] retire on the united register? -/
82 namespace RALi
83
84 inductive Grade | premise | structural | theorem deriving
    ↳ DecidableEq, Repr
85 def Grade.rank : Grade → Nat | .premise => 0 | .structural =>
    ↳ 1 | .theorem => 2
86 def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank
    ↳ then a else b
87
88 inductive Verdict | seal (g : Grade) | xi0 deriving
    ↳ DecidableEq, Repr
89

```

```

90 /-- The united register. Unity is the claim that the formal
91   ↳ reading and the
92   ↳ kinetic reading are one; on  $\zeta$  it is stated as the supply
93   ↳ of the bridge,
94   ↳ carried at the grade of whatever supplies it. -/
95 structure Unity (S : Substrate) (L : LiData) where
96   bridge : Bridge S L
97   grade : Grade
98 /-- The emitter on the formal string: with unity supplied,
99   ↳ the verdict is a
100   ↳ seal at the weakest grade on the chain; without it,  $[\Xi_0]$ .
101   ↳ -/
102 def emit {S : Substrate} {L : LiData} : Option (Unity S L) →
103   ↳ Verdict
104   | some u => .seal (Grade.weakest u.grade .theorem)
105   | none   => .xi0
106 /-- 5.  $[\Xi_0]$  retires exactly when unity is supplied, and the
107   ↳ seal it becomes
108   ↳ is sound: it carries RH. -/
109 theorem xi0_retires_iff_unity (S : Substrate) (L : LiData) (u
110   ↳ : Option (Unity S L)) :
111   emit u ≠ .xi0 ↔ u.isSome := by
112   cases u <|> simp [emit]
113 theorem retired_seal_is_sound (S : Substrate) (L : LiData)
114   (hRA : RA S) (hLi : LiCriterion L) (u : Unity S L) : L.RH
115   ↳ :=
116   bridge_yields_RH S L hRA hLi u.bridge
117 /-- 6. The grade law: the retired verdict is never stronger
118   ↳ than its unity
119   ↳ posit. A premise-grade unity yields a premise-grade seal.
120   ↳ -/
121 theorem retired_grade_capped {S : Substrate} {L : LiData} (u
122   ↳ : Unity S L) :
123   emit (some u) = .seal (Grade.weakest u.grade .theorem) ∧
124   (Grade.weakest u.grade .theorem).rank ≤ u.grade.rank :=
125   ↳ by
126   refine {rfl, ?_}
127   cases h : u.grade <|> decide
128 /-- 7. Unity is not free: RA does not supply it (from theorem
129   ↳ 2). -/
130 theorem unity_is_keyed : RA oneEnergy ∧ ¬ Nonempty (Unity
131   ↳ oneEnergy badData) :=
132   (ra_holds_in_model, fun (u) => bridge_is_keyed.2
133   ↳ (u.bridge))
134 /-- 8. The Ghost. A reading that deletes the arrow returns
135   ↳ one verdict for a
136   ↳ claim and its negation: it cannot be a verdict on either.
137   ↳ -/
138 def arrowDeleted (P : Prop) : Verdict := .xi0
139 theorem ghost (P : Prop) : arrowDeleted P = arrowDeleted (¬
140   ↳ P) := rfl
141 end RALi
142 #print axioms RALi.xi0_retires_iff_unity
143 #print axioms RALi.retired_seal_is_sound
144 #print axioms RALi.retired_grade_capped
145 #print axioms RALi.unity_is_keyed
146 #print axioms RALi.ghost
147 /-- ## PART III · The try. Which Codex instruments can supply
148   ↳ `Bridge.read` for  $\zeta$ ? -/
149 namespace RALi
150 /-- 9. The uniformity trap. A family of bridges that picks
151   ↳ its existents
152   ↳ without reading which L-function it serves is a uniform
153   ↳ bridge. Every
154   ↳ instrument carried at  $\Delta M = 0$  (the eliminator, the
155   ↳ multi-recursion
156   ↳ collapse, the one-bit species, the bare arrow) chooses
157   ↳ its existents
158   ↳ that way, so it lands here. -/
159 theorem common_phi_is_uniform (S : Substrate) (F : Type) (L :
160   ↳ F → LiData)
161   (φ0 : Nat → S.U) (B : ∀ f, Bridge S (L f)) (hφ : ∀ f, (B
162   ↳ f).φ = φ0) :
163   Nonempty (UniformBridge S F L) :=
164   (φ0, fun f n hn h => (B f).read n hn (by rw [hφ f]; exact
165   ↳ h)))
166 /-- 10. Hence dead: with RA and one family member carrying a
167   ↳ negative λ,
168   ↳ no  $\zeta$ -blind choice of existents can serve every member. -/
169 theorem blind_constructions_die (S : Substrate) (F : Type) (L
170   ↳ : F → LiData)
171   (hRA : RA S) (hctl : ∃ f n, 1 ≤ n ∧ ¬ (L f).nonneg n)
172   (φ0 : Nat → S.U) : ¬ ∃ B : (∀ f, Bridge S (L f)), ∀ f, (B
173   ↳ f).φ = φ0 :=
174   fun (B, hφ) => no_uniform_bridge S F L hRA hctl
175   ↳ (common_phi_is_uniform S F L φ0 B hφ)
176 /-- 11. What positivity on the Euler product does reach.
177   ↳ Mertens' identity
178   ↳  $3 + 4 \cos \theta + \cos 2\theta = 2(1 + \cos \theta)^2 \geq 0$ , with  $\cos 2\theta =$ 
179   ↳  $2c^2 - 1$ , is the
180   ↳ positivity that yields  $\zeta(1 + it) \neq 0$ : the wall at  $\text{Re} = 1$ .
181   ↳ Checked here on
182   ↳ integer c as the polynomial identity it is. -/
183 theorem mertens_identity (c : Int) : 3 + 4 * c + (2 * c * c -
184   ↳ 1) = 2 * ((1 + c) * (1 + c)) := by
185   simp only [Int.add_mul, Int.mul_add, Int.one_mul,
186   ↳ Int.mul_one, Int.mul_assoc]
187 omega
188 theorem mertens_nonneg (c : Int) : 0 ≤ 3 + 4 * c + (2 * c * c
189   ↳ - 1) := by
190   rw [mertens_identity]
191   refine Int.mul_nonneg (by decide) ?_
192   rcases Int.le_total 0 (1 + c) with h | h
193   · exact Int.mul_nonneg h h
194   · have h' : 0 ≤ -(1 + c) := by omega
195   have := Int.mul_nonneg h' h'
196   rwa [Int.neg_mul_neg] at this
197 end RALi
198 #print axioms RALi.common_phi_is_uniform
199 #print axioms RALi.blind_constructions_die
200 #print axioms RALi.mertens_nonneg
201 /-- ## PART IV · Time, made exact. The Li modes on the Bridge
202   ↳ plane.
203   ↳ A zero p = h/2 + i t sits at (h, t) in half-units, the Bridge
204   ↳ plane, fold (h,t) ↔ (2-h, t).
205   ↳ Li's coefficient is λn = Σp [1 - zpn] with the mode zp = 1 -
206   ↳ 1/p = (p-1)/p, so step
207   ↳ n ↔ n+1 multiplies each mode by zp. |zp|2 = |p-1|2/|p|2,
208   ↳ scaled by 4:
209   ↳ N1 = (h-2)2 + 4t2, N0 = h2 + 4t2.
210   ↳ A mode is unitary (perpetual, neither growing nor dying) iff
211   ↳ N1 = N0; it grows iff N1 > N0. -/
212 namespace RALi
213 def N1 (h t : Int) : Int := (h - 2) * (h - 2) + 4 * (t * t)
214 def N0 (h t : Int) : Int := h * h + 4 * (t * t)
215 theorem N1_sub_N0 (h t : Int) : N1 h t - N0 h t = 4 - 4 * h
216   ↳ := by
217   simp only [N1, N0, Int.sub_mul, Int.mul_sub]
218   omega
219 /-- 12. A mode is unitary exactly on the line: perpetuity in
220   ↳ time is the fixed locus. -/
221 theorem unitary_iff_on_line (h t : Int) : N1 h t = N0 h t ↔ h
222   ↳ = 1 := by
223   have := N1_sub_N0 h t
224   exact (fun e => by omega, fun e => by omega)
225 /-- 13. Left of the line the mode grows; right of it the mode
226   ↳ dies. -/
227 theorem grows_left (h t : Int) (hl : h < 1) : N0 h t < N1 h t
228   ↳ := by
229   have := N1_sub_N0 h t; omega
230 theorem dies_right (h t : Int) (hr : 1 < h) : N1 h t < N0 h t
231   ↳ := by
232   have := N1_sub_N0 h t; omega
233 /-- 14. The fold forbids a quiet exit: any off-line zero,
234   ↳ together with its mirror under

```

```

208 the functional equation, carries a growing mode. -/
209 theorem off_line_forces_growth (h t : Int) (hoff : h ≠ 1) :
210   N0 h t < N1 h t v N0 (2 - h) t < N1 (2 - h) t := by
211   have a := N1_sub_N0 h t; have b := N1_sub_N0 (2 - h) t;
212   ↪ omega
213
214 /-- 15. PERPETUAL STABILITY IS THE LINE PROPERTY. For any
215   ↪ fold-invariant zero set on the
216 plane: no mode grows under the time step iff every zero lies
217   ↪ on h = 1. -/
218 theorem stability_iff_line (Z : Int × Int → Prop)
219   (hinv : ∀ p, Z p → Z (2 - p.1, p.2)) :
220   (∀ p, Z p → N1 p.1 p.2 ≤ N0 p.1 p.2) ↔ (∀ p, Z p → p.1 =
221     ↪ 1) := by
222   constructor
223   · intro hs p hp
224     have a := hs p hp
225     have b := hs (2 - p.1, p.2) (hinv p hp)
226     have c := N1_sub_N0 p.1 p.2
227     have d := N1_sub_N0 (2 - p.1) p.2
228     simp only at b d
229     omega
230   · intro hl p hp
231     have := hl p hp
232     have c := N1_sub_N0 p.1 p.2
233     omega
234 end RALi
235
236 #print axioms RALi.unitary_iff_on_line
237 #print axioms RALi.off_line_forces_growth
238 #print axioms RALi.stability_iff_line
239
240 /-! ## PART V · The monism rule, hardened. "What happens once
241   ↪ happens again." -/
242 namespace RALi
243
244 /-- 16. One repeating one: the rule, with its step, is
245   ↪ induction. -/
246 theorem one_repeats_one (P : Nat → Prop) (h0 : P 0) (step : ∀
247   ↪ n, P n → P (n + 1)) :
248   ∀ n, P n := by
249   intro n; induction n with
250   | zero => exact h0
251   | succ k ih => exact step k ih
252
253 /-- 17. The step is the whole content: the universal claim is
254   ↪ exactly the base plus the step. -/
255 theorem the_step_is_the_claim (P : Nat → Prop) :
256   (∀ n, P n) ↔ (P 0 ∧ ∀ n, P n → P (n + 1)) :=
257   (fun h => (h 0, fun n => h (n + 1)), fun (h0, s) =>
258     ↪ one_repeats_one P h0 s)
259
260 /-- 18. The monism posit: one principle means the property
261   ↪ does not vary with the index.
262 Under it, one instance forces all. -/
263 theorem monism_posit_forces (P : Nat → Prop) (uniform : ∀ n
264   ↪ m, P n ↔ P m) (h0 : P 0) :
265   ∀ n, P n := fun n => (uniform 0 n).mp h0
266
267 /-- 19. Finite confirmation never forces. For every height N
268   ↪ there is a property true on
269 every index below N and false beyond: Skewes, Mertens, Pólya
270   ↪ are this shape. -/
271 theorem finite_never_forces (N : Nat) :
272   ∃ P : Nat → Prop, (∀ n, n < N → P n) ∧ ¬ ∀ n, P n :=
273   (fun n => n < N, fun _ h => h, fun h => Nat.lt_irrefl N (h
274     ↪ N))
275
276 /-- 20. Monism is not a law of the fold. A fold-invariant
277   ↪ zero set can hold one zero on the
278 line and a mirror pair off it: "on the line once" does not
279   ↪ recur by symmetry alone.
280 This is the Davenport-Heilbronn shape on the Bridge plane. -/
281 def mixedZ (p : Int × Int) : Prop := p = (1, 0) v p = (0, 5)
282   ↪ v p = (2, 5)
283
284 theorem mixed_is_fold_invariant : ∀ p, mixedZ p → mixedZ (2 -
285   ↪ p.1, p.2) := by
286   intro p hp
287   rcases hp with h | h | h <=> subst h
288   · exact 0r.inl rfl
289   · exact 0r.inr (0r.inr rfl)
290
291 exact 0r.inr (0r.inl rfl)
292
293 theorem mixed_recurrence_fails :
294   mixedZ (1, 0) ∧ mixedZ (0, 5) ∧ ¬ (∀ p, mixedZ p → p.1 =
295     ↪ 1) :=
296   (0r.inl rfl, 0r.inr (0r.inl rfl), fun h => by
297     have := h (0, 5) (0r.inr (0r.inl rfl)); cases this)
298
299 end RALi
300
301 #print axioms RALi.one_repeats_one
302 #print axioms RALi.the_step_is_the_claim
303 #print axioms RALi.monism_posit_forces
304 #print axioms RALi.finite_never_forces
305 #print axioms RALi.mixed_is_fold_invariant
306 #print axioms RALi.mixed_recurrence_fails
307
308 /-! ## PART VI · Actualism. Only measured zeros are actual.
309   ↪ -/
310 namespace RALi
311
312 /-- A zero set on a carrier, a line predicate, and the time
313   ↪ at which each zero is measured
314 (none if never). The actual zeros at time T are those
315   ↪ measured by T. -/
316 structure Actualized (α : Type) where
317   Z      : α → Prop
318   onLine : α → Prop
319   measured : α → Option Nat
320
321 def RHfull {α : Type} (A : Actualized α) : Prop := ∀ z, A.Z z
322   ↪ → A.onLine z
323
324 def RHactual {α : Type} (A : Actualized α) : Prop :=
325   ↪ ∀ z, A.Z z → A.measured z ≠ none → A.onLine z
326
327 /-- 21. The full claim yields the actual one. -/
328 theorem full_gives_actual {α : Type} (A : Actualized α) :
329   ↪ RHfull A → RHactual A :=
330   fun h z hz _ => h z hz
331
332 /-- 22. The actual claim does not yield the full one: one
333   ↪ never-measured zero off the line. -/
334 def unreached : Actualized Bool := (fun _ => True, fun b => b
335   ↪ = true, fun b => if b then some 0 else none)
336
337 theorem actual_not_full : RHactual unreached ∧ ¬ RHfull
338   ↪ unreached := by
339   refine (fun z _ hm => ?_, fun h => by cases h false
340     ↪ trivial)
341   cases z
342   · exact absurd rfl hm
343   · rfl
344
345 /-- 23. Given sufficient time, every zero is measured, and
346   ↪ then the actual claim over all
347 time is exactly the full claim: actualism re-routes to the
348   ↪ same step and does not bypass it. -/
349 theorem sufficient_time {α : Type} (A : Actualized α) (reach
350   ↪ : ∀ z, A.Z z → A.measured z ≠ none) :
351   RHactual A ↔ RHfull A :=
352   (fun h z hz => h z hz (reach z hz), full_gives_actual A)
353
354 end RALi
355
356 #print axioms RALi.full_gives_actual
357 #print axioms RALi.actual_not_full
358 #print axioms RALi.sufficient_time
359
360 /-! ## PART VII · The constructed witness, reconstructed from
361   ↪ Residual Monism.
362 The paper's witness is built from RA and read through RAF and
363   ↪ RAM. Here the witness is
364 built from the Codex connector directly: one substrate, one
365   ↪ involution serving both routes
366 (σ = σ'), and one principle that does not vary with the
367   ↪ index. The index is time. -/
368 namespace RALi
369
370 structure Q4 where
371   r : Int
372   i : Int
373   j : Int
374   k : Int

```

```

339 deriving DecidableEq, Repr
340
341 /-- The geometric route's involution and the formal route's
    ↪ involution, written apart. -/
342 def sigmaGeo (q : Q4) : Q4 := {q.r, -q.i, -q.j, -q.k}
343 def sigmaForm (q : Q4) : Q4 := {q.r, -q.i, -q.j, -q.k}
344
345 /-- THE MONISM WITNESS. One involution, one principle, one
    ↪ seed. -/
346 structure MonismWitness (P : Nat → Prop) : Prop where
347   one_involution : ∀ q, sigmaGeo q = sigmaForm q
348   uniform        : ∀ n m, P n ↔ P m
349   seed           : P 0
350
351 /-- 24. The seat field is constructed, not posited:  $\sigma = \sigma'$ 
    ↪ holds by rfl. -/
352 theorem one_involution_constructed : ∀ q, sigmaGeo q =
    ↪ sigmaForm q := fun _ => rfl
353
354 /-- 25. The timeless reading and the timed reading coincide
    ↪ under one principle:
355 the claim over all time is the claim at one instant. Deleting
    ↪ time loses nothing. -/
356 theorem timeless_equals_timed (P : Nat → Prop) (u : ∀ n m, P
    ↪ n ↔ P m) :
357   (∀ n, P n) ↔ P 0 :=
358   (fun h => h 0, fun h n => (u 0 n).mp rfl)
359
360 /-- 26. The witness closes the record, past and future. -/
361 theorem monism_closes (P : Nat → Prop) (w : MonismWitness P)
    ↪ : ∀ n, P n :=
362   (timeless_equals_timed P w.uniform).mpr w.seed
363
364 /-- 27. The witness is exactly the claim: a monism witness
    ↪ for P exists iff P holds at
365 every index. Its value field carries the whole of the claim,
    ↪ as theorem 17 required. -/
366 theorem monism_witness_is_the_claim (P : Nat → Prop) :
367   MonismWitness P ↔ ∀ n, P n :=
368   (monism_closes P, fun h => (one_involution_constructed,
369     fun n m => {fun _ => h m, fun _ => h n}, h 0))
370
371 /-- 28. Constructed at the model, as the paper constructs RA
    ↪ at one point. -/
372 theorem constructed_monism : MonismWitness (fun _ => True) :=
373   (one_involution_constructed, fun _ _ => Iff.rfl, trivial)
374
375 /-- 29. Unlike the RA witness, the monism witness is not
    ↪ silent on counter-models:
376 it cannot be constructed where the property breaks, so it
    ↪ decides by containing. -/
377 theorem monism_absent_on_breaks : ¬ MonismWitness (fun n => n
    ↪ = 0) :=
378   fun w => absurd ((w.uniform 0 1).mp rfl) (by decide)
379
380 /-- 30. Run through the RA-Li bridge. Replace the bridge by
    ↪ the monism witness on Li's
381 sign stream: the witness, with Li's criterion, gives RH. -/
382 theorem monism_yields_RH (L : LiData) (hLi : LiCriterion L)
383   (w : MonismWitness (fun n => L.nonneg (n + 1))) : L.RH :=
384   hLi.mpr (fun n hn => by
385     obtain (k, rfl) : ∃ k, n = k + 1 := (n - 1, by omega)
386     exact monism_closes _ w k)
387
388 /-- 31. And the price, stated: on  $\zeta$  the witness exists iff
    ↪ every Li sign is nonnegative,
389 which by Li is RH. Reconstruction moves the owed bit into
    ↪ 'uniform'; it does not remove it. -/
390 theorem monism_on_li_is_RH (L : LiData) (hLi : LiCriterion L)
    ↪ :
391   MonismWitness (fun n => L.nonneg (n + 1)) ↔ L.RH := by
392   rw [monism_witness_is_the_claim]
393   refine (fun h => monism_yields_RH L hLi
    ↪ ((monism_witness_is_the_claim _).mpr h), fun h n => ?_)
394   exact (hLi.mp h) (n + 1) (by omega)
395
396 end RALi
397
398 #print axioms RALi.one_involution_constructed
399 #print axioms RALi.timeless_equals_timed
400 #print axioms RALi.monism_closes
401 #print axioms RALi.monism_witness_is_the_claim
402 #print axioms RALi.constructed_monism

```

```

403 #print axioms RALi.monism_absent_on_breaks
404 #print axioms RALi.monism_yields_RH
405 #print axioms RALi.monism_on_li_is_RH
406
407 /-- ## PART VIII · Re-anchoring the monism witness to
    ↪ mathematical standard.
408 Calibration case: Perelman. The standard form of "one
    ↪ principle, the same result again"
409 is not a free uniformity; it is one dynamics, one invariant,
    ↪ and a PROVED transport of the
410 invariant along the dynamics (induction on the orbit;
    ↪ Lyapunov monotonicity; Perelman's
411 W-entropy under Ricci flow). -/
412 namespace RALi
413
414 /-- 32. Monism as a schema over all properties is
    ↪ inconsistent: uniformity cannot be
415 posited for every P. It must be anchored to a principle, not
    ↪ spread over properties. -/
416 theorem global_monism_inconsistent : ¬ ∀ P : Nat → Prop, ∀ n
    ↪ m, P n ↔ P m :=
417   fun h => absurd ((h (fun n => n = 0) 0 1).mp rfl) (by
    ↪ decide)
418
419 def iter {S : Type} (f : S → S) : Nat → S → S
420   | 0, s => s
421   | n + 1, s => f (iter f n s)
422
423 /-- THE ANCHORED MONISM WITNESS. One substrate S, one
    ↪ principle f (the dynamics),
424 one invariant I, the transport of I along f, and a seed. -/
425 structure AnchoredMonism (S : Type) where
426   f          : S → S
427   I          : S → Prop
428   transport  : ∀ s, I s → I (f s)
429   s0         : S
430   seed       : I s0
431
432 /-- 33. The anchored witness closes its whole orbit, past and
    ↪ future. -/
433 theorem anchored_closes {S : Type} (A : AnchoredMonism S) : ∀
    ↪ n, A.I (iter A.f n A.s0) := by
434   intro n; induction n with
435   | zero => exact A.seed
436   | succ k ih => exact A.transport _ ih
437
438 /-- 34. The free monism witness is DERIVED from the anchored
    ↪ one: uniformity is no longer
439 posited, it is manufactured by the transport. -/
440 theorem anchored_derives_monism {S : Type} (A :
    ↪ AnchoredMonism S) :
441   MonismWitness (fun n => A.I (iter A.f n A.s0)) :=
442   (one_involution_constructed,
443     fun n m => {fun _ => anchored_closes A m, fun _ =>
    ↪ anchored_closes A n},
444     A.seed)
445
446 /-- 35. THE PERELMAN SHAPE. A quantity monotone along one
    ↪ flow, here a discrete flow on its
447 time index, is an anchored monism witness for the property
    ↪ "never below its start." Perelman's
448 W-entropy along Ricci flow has this shape; it is a structural
    ↪ analogue and is not formalized. -/
449 def lyapunovWitness (W : Nat → Int) (mono : ∀ n, W n ≤ W (n +
    ↪ 1)) : AnchoredMonism Nat :=
450   (Nat.succ, fun n => W 0 ≤ W n, fun n h => Int.le_trans h
    ↪ (mono n), 0, Int.le_refl _)
451
452 theorem lyapunov_is_monism (W : Nat → Int) (mono : ∀ n, W n ≤
    ↪ W (n + 1)) :
453   MonismWitness (fun n => W 0 ≤ W (iter Nat.succ n 0)) :=
454   anchored_derives_monism (lyapunovWitness W mono)
455
456 /-- 36. The calibration's grade law: the closure is as strong
    ↪ as its transport. -/
457 structure GradedAnchor (S : Type) where
458   A          : AnchoredMonism S
459   tgrade     : Grade
460
461 def closureGrade {S : Type} (G : GradedAnchor S) : Grade :=
    ↪ Grade.weakest G.tgrade .theorem
462
463 theorem closure_grade_is_transport_grade {S : Type} (G :
    ↪ GradedAnchor S) :

```

```

464 closureGrade G = G.tgrade := by
465   unfold closureGrade Grade.weakest; cases G.tgrade <;> rfl
466
467 /-- 37. The RH instance. On Li's sign stream with the time
468   ↳ step as the principle, an
469   ↳ anchored witness exists iff RH. The transport is `nonneg
470   ↳ (n+1) → nonneg (n+2)` : the
471   ↳  $\zeta$ -analogue of Perelman's monotonicity formula, not yet proved
472   ↳ by anyone. -/
473 theorem rh_anchor_is_the_claim (L : LiData) (hLi :
474   ↳ LiCriterion L) :
475   (∃ A : AnchoredMonism Nat, A.f = Nat.succ ∧ A.s0 = 0 ∧
476     ∀ n, A.I n → L.nonneg (n + 1)) ↔ L.RH := by
477   constructor
478   · rintro (A, hf, hs, hI)
479     apply (monism_on_li_is_RH L hLi).mpr
480     refine (monism_witness_is_the_claim _).mpr (fun n => ?_)
481     have key : ∀ n, iter A.f n A.s0 = n := by
482       intro n; induction n with
483       | zero => exact hs
484       | succ k ih => show A.f (iter A.f k A.s0) = k + 1; rw
485         ↳ [ih, hf]
486     have := anchored_closes A n
487     rw [key n] at this
488     exact (hI n).mpr this
489   · intro h
490     have all := (monism_witness_is_the_claim _).mpr
491     ↳ ((monism_on_li_is_RH L hLi).mpr h)
492     exact ((Nat.succ, fun n => L.nonneg (n + 1), fun n =>
493       ↳ all (n + 1), 0, all 0),
494       ↳ rfl, rfl, fun _ => Iff.rfl)
495 end RALi
496
497 #print axioms RALi.global_monism_inconsistent
498 #print axioms RALi.anchored_closes
499 #print axioms RALi.anchored_derives_monism
500 #print axioms RALi.lyapunov_is_monism
501 #print axioms RALi.closure_grade_is_transport_grade
502 #print axioms RALi.rh_anchor_is_the_claim
503
504 /-- ## PART IX ·  $\zeta$ 's time has a proved monotonicity formula,
505   ↳ and it runs the wrong way.
506 The de Bruijn-Newman heat flow  $H_t(z) = \int e^{\{t u^2\}} \Phi(u) \cos(z
507   ↳ u) du$ ,  $\Phi$  built from  $\xi$ .
508 de Bruijn (1950): if every zero of  $H_0$  lies in  $|\operatorname{Im} z| \leq \Delta$ ,
509   ↳ every zero of  $H_t$  lies in
510   ↳  $|\operatorname{Im} z| \leq \sqrt{\max(\Delta^2 - 2t, 0)}$ . RH is "all zeros of  $H_0$  real".  $\Delta$ 
511   ↳ is the least t with all zeros
512   ↳ of  $H_t$  real; Newman (1976) defined it, Rodgers-Tao (2020)
513   ↳ proved  $\Delta \geq 0$ , Polymath 15 (2019)
514 proved  $\Delta \leq 0.22$ . So  $\operatorname{RH} \leftrightarrow \Delta = 0$ . Modelled here on the strip
515   ↳ bound, widths in Nat, half-steps
516 of time; a toy of the bound, not of the flow. -/
517 namespace RALi
518
519 /-- Squared strip width after t time steps:  $\max(d2 - 2t, 0)$ .
520   ↳ -/
521 def flow (d2 t : Nat) : Nat := d2 - 2 * t
522 def realAt (d2 t : Nat) : Prop := flow d2 t = 0
523
524 /-- 38. THE MONOTONICITY FORMULA. The width never grows along
525   ↳  $\zeta$ 's time. -/
526 theorem flow_monotone (d2 t : Nat) : flow d2 (t + 1) ≤ flow
527   ↳ d2 t := by
528   unfold flow; omega
529
530 /-- 39. Its transport, proved: once every zero is real, every
531   ↳ zero stays real. This is
532   ↳ de Bruijn's theorem in the toy, the  $\zeta$ -analogue of Perelman's
533   ↳ monotonicity, at theorem grade. -/
534 theorem reality_transported (d2 t : Nat) (h : realAt d2 t) :
535   ↳ realAt d2 (t + 1) := by
536   unfold realAt flow at *; omega
537
538 /--  $\Lambda$  in the toy: the first time the strip closes. -/
539 def Lam (d2 : Nat) : Nat := (d2 + 1) / 2
540
541 theorem real_at_Lam (d2 : Nat) : realAt d2 (Lam d2) := by
542   ↳ unfold realAt flow Lam; omega
543
544 /-- 40. In the toy, the strip is closed at time zero iff  $\Lambda =
545   ↳ 0$ . For  $\zeta$ ,  $\operatorname{RH} \leftrightarrow \Lambda = 0$  is the cited
546   ↳ result of Newman with Rodgers and Tao and enters the cone as
547   ↳ `hLam`. -/
548 theorem rh_iff_lambda_zero (d2 : Nat) : realAt d2 0 ↔ Lam d2
549   ↳ = 0 := by
550   unfold realAt flow Lam
551   exact (fun h => by omega, fun h => by omega)
552
553 /-- 41. The anchored monism witness exists for  $\zeta$ 's time,
554   ↳ transport proved, seeded at  $\Lambda$ .
555 It closes the whole future of the flow from  $\Lambda$  on. -/
556 def deBruijnWitness (d2 : Nat) : AnchoredMonism Nat :=
557   (Nat.succ, realAt d2, fun t h => reality_transported d2 t
558     ↳ h, Lam d2, real_at_Lam d2)
559
560 theorem iter_succ (n s : Nat) : iter Nat.succ n s = s + n :=
561   ↳ by
562   induction n with
563   | zero => rfl
564   | succ k ih => show Nat.succ (iter Nat.succ k s) = s + (k +
565     ↳ 1); rw [ih]; omega
566
567 theorem future_closed (d2 n : Nat) : realAt d2 (Lam d2 + n)
568   ↳ := by
569   have := anchored_closes (deBruijnWitness d2) n
570   rwa [show (deBruijnWitness d2).f = Nat.succ from rfl,
571     ↳ iter_succ] at this
572
573 /-- 42. THE RECORD FORGETS. After one step the record of the
574   ↳ width-zero state and the
575   ↳ width-one state coincide: the forward flow is even in the RH
576   ↳ bit, so no readout of the
577   ↳ flowed record decides RH. This is the fTOE wall, T1, executed
578   ↳ on  $\zeta$ 's own time. -/
579 theorem flowed_record_forgets (T : Nat) (hT : 1 ≤ T) :
580   ↳ ¬ ∃ g : Nat → Bool, ∀ d2, g (flow d2 T) = decide (d2 = 0)
581   ↳ := by
582   rintro (g, hg)
583   have a := hg 0; have b := hg 1
584   have e : flow 1 T = flow 0 T := by unfold flow; omega
585   rw [e, a] at b
586   exact absurd b (by decide)
587
588 /-- 43. The upstream direction is not a transport. Going
589   ↳ backward from a real record,
590   ↳ both answers are admissible: the preimage of "real at T"
591   ↳ holds the RH state and a non-RH
592   ↳ state. The forward arrow proves; the backward arrow must be
593   ↳ supplied. -/
594 theorem upstream_is_not_forced (T : Nat) (hT : 1 ≤ T) :
595   ↳ realAt 0 T ∧ realAt 1 T ∧ realAt 0 0 ∧ ¬ realAt 1 0 := by
596   unfold realAt flow
597   exact (by omega, by omega, rfl, fun h => by omega)
598
599 end RALi
600
601 #print axioms RALi.flow_monotone
602 #print axioms RALi.reality_transported
603 #print axioms RALi.rh_iff_lambda_zero
604 #print axioms RALi.future_closed
605 #print axioms RALi.flowed_record_forgets
606 #print axioms RALi.upstream_is_not_forced
607
608 /-- ## PART X · Halted Uniduction with the category-gap
609   ↳ ← eliminator. The regress, closed.
610 Every "missing piece" of Parts I-IX is a register's reading
611   ↳ ← of one seat. Exhibit each as a
612   ↳ leg of one cone with apex RH; then the hunt has one gap, not
613   ↳ ← a sequence of them. -/
614 namespace RALi
615
616 structure Hunt where
617   S : Substrate
618   L : LiData
619   u0 : S.U
620   hRA : RA S
621   hLi : LiCriterion L
622   lamZero : Prop -- the upstream reading:  $\Lambda =$ 
623     ↳ 0
624   hLam : L.RH ↔ lamZero -- Newman's definition with
625     ↳ Rodgers-Tao, cited
626
627 /-- The legs. Each register's reading of the owed seat,
628   ↳ ← proved equal to the apex. -/

```

```

588 theorem leg_bridge (H : Hunt) : Nonempty (Bridge H.S H.L) ↔
    ↪ H.L.RH :=
589   (fun (B) => bridge_yields_RH H.S H.L H.hRA H.hLi B,
590     fun h => ((fun _ => H.u0, fun n hn _ => (H.hLi.mp h) n
    ↪ hn)))
591
592 theorem leg_unity (H : Hunt) : Nonempty (Unity H.S H.L) ↔
    ↪ H.L.RH :=
593   (fun (u) => retired_seal_is_sound H.S H.L H.hRA H.hLi u,
594     fun h => let (B) := (leg_bridge H).mpr h; ((B, .premise)))
595
596 theorem leg_monism (H : Hunt) : MonismWitness (fun n =>
    ↪ H.L.nonneg (n + 1)) ↔ H.L.RH :=
597   monism_on_li_is_RH H.L H.hLi
598
599 theorem leg_transport (H : Hunt) :
600   (∃ A : AnchoredMonism Nat, A.f = Nat.succ ∧ A.s0 = 0 ∧ ∀
    ↪ n, A.I n ↔ H.L.nonneg (n + 1))
601   ↔ H.L.RH :=
602   rh_anchor_is_the_claim H.L H.hLi
603
604 theorem leg_upstream (H : Hunt) : H.lamZero ↔ H.L.RH :=
    ↪ H.hLam.symm
605
606 /-- The five readings, as a diagram of propositions. -/
607 inductive Reg5 | bridge | unity | monism | transport |
    ↪ upstream deriving DecidableEq, Repr
608
609 def D5 (H : Hunt) : Reg5 → Prop
610 | .bridge    => Nonempty (Bridge H.S H.L)
611 | .unity     => Nonempty (Unity H.S H.L)
612 | .monism    => MonismWitness (fun n => H.L.nonneg (n + 1))
613 | .transport => ∃ A : AnchoredMonism Nat, A.f = Nat.succ ∧
    ↪ A.s0 = 0 ∧ ∀ n, A.I n ↔ H.L.nonneg (n + 1)
614 | .upstream => H.lamZero
615
616 /-- 44. THE CONE. RH is the apex of a cone over all five
    ↪ readings, every leg an
617 equivalence: the category gaps between the hunt's registers
    ↪ are eliminated. -/
618 theorem hunt_cone (H : Hunt) : ∀ r, D5 H r ↔ H.L.RH := by
619   intro r; cases r
620   · exact leg_bridge H
621   · exact leg_unity H
622   · exact leg_monism H
623   · exact leg_transport H
624   · exact leg_upstream H
625
626 /-- 45. ONE GAP. Any two readings are the same proposition.
    ↪ The quantity monotone against
627 the flow (upstream) IS the bridge's read, the unity posit,
    ↪ the monism field, the transport. -/
628 theorem one_gap (H : Hunt) (r s : Reg5) : D5 H r ↔ D5 H s :=
629   (hunt_cone H r).trans (hunt_cone H s).symm
630
631 /-- 46. THE HALT. Every proposition proved equivalent to the
    ↪ hypothesis lands on the same apex
632 as the five readings; which future propositions are
    ↪ equivalent is not decided here. -/
633 theorem regress_halts (H : Hunt) (P : Prop) (hP : P ↔ H.L.RH)
    ↪ (r : Reg5) : P ↔ D5 H r :=
634   hP.trans (hunt_cone H r).symm
635
636 /-- 47. Given a true proposition Q, proving Q → RH is the
    ↪ same as proving RH: conditioning on a
637 true premise adds nothing (Theorem E of the prior paper, here
    ↪ on the hunt's apex). -/
638 theorem nothing_weaker (H : Hunt) (Q : Prop) (hq : Q) : (Q →
    ↪ H.L.RH) ↔ H.L.RH :=
639   (fun f => f hq, fun h _ => h)
640
641 end RALi
642
643 #print axioms RALi.hunt_cone
644 #print axioms RALi.one_gap
645 #print axioms RALi.regress_halts
646 #print axioms RALi.nothing_weaker
647
648 /-! ## PART X, completed · a sufficient route closes every
    ↪ reading at once. -/
649 namespace RALi
650
651 /-- 48. Theorem 46 covers every reformulation equivalent to
    ↪ the apex. A route strictly

```

```

652 stronger than the apex (a specific operator, a hypothesis
    ↪ over a family) is not equivalent,
653 and it does not open a new gap either: any sufficient route
    ↪ supplies all five readings. -/
654 theorem sufficient_closes_all (H : Hunt) (P : Prop) (hP : P →
    ↪ H.L.RH) (p : P) :
655   ∀ r, D5 H r :=
656   fun r => (hunt_cone H r).mpr (hP p)
657
658 end RALi
659
660 #print axioms RALi.sufficient_closes_all
661
662 /-! ## PART XI · Timeless Residual Monism, stated as
    ↪ Postulate M, and its equivalence with the line property.
    ↪ -/
663 namespace TimeLocus
664
665 def onLine (p : Int × Int) : Prop := p.1 = 1
666
667 /-- A timed world: the locus at each time, and the zeros
    ↪ present at each time. -/
668 structure World where
669   locus : Nat → (Int × Int → Prop)
670   zeros : Nat → (Int × Int → Prop)
671
672 /-- TIME PSP premise: prior and posterior locus are the SAME
    ↪ locus, the line, at every time. -/
673 def SameLocus (W : World) : Prop := ∀ t p, W.locus t p ↔
    ↪ onLine p
674
675 /-- The claim to be tested: every zero at every time is bound
    ↪ to the locus. -/
676 def Bound (W : World) : Prop := ∀ t p, W.zeros t p → W.locus
    ↪ t p
677
678 /-- Fold-invariance at every time: the functional equation
    ↪ acts at each iteration. -/
679 def FoldInv (W : World) : Prop := ∀ t p, W.zeros t p →
    ↪ W.zeros t (2 - p.1, p.2)
680
681 /-- 51. Given the same locus at all times, being bound IS the
    ↪ line property at all times. -/
682 theorem bound_iff_line (W : World) (h : SameLocus W) :
683   Bound W ↔ ∀ t p, W.zeros t p → onLine p :=
684   (fun b t p z => (h t p).mp (b t p z), fun l t p z => (h t
    ↪ p).mpr (l t p z))
685
686 /-- 52, the test world. A world where the locus is the same
    ↪ line at every time, the fold acts at every
687 time, the on-line zero recurs at every time, and a mirror
    ↪ pair sits off the line at every time. -/
688 def testWorld : World :=
689   (fun _ p => onLine p, fun _ p => p = (1, 0) ∨ p = (0, 5) ∨
    ↪ p = (2, 5))
690
691 theorem test_same_locus : SameLocus testWorld := fun _ _ =>
    ↪ Iff.rfl
692
693 theorem test_fold_inv : FoldInv testWorld := by
694   intro _ p hp
695   rcases hp with h | h | h <; subst h
696   · exact 0r.inl rfl
697   · exact 0r.inr (0r.inr rfl)
698   · exact 0r.inr (0r.inl rfl)
699
700 theorem test_not_bound : ¬ Bound testWorld := fun b => by
701   have := b 0 (0, 5) (0r.inr (0r.inl rfl)); cases this
702
703 /-- 52. Therefore SameLocus and FoldInv, at every time, do
    ↪ not imply Bound. -/
704 theorem time_does_not_bind :
705   ¬ ∀ W : World, SameLocus W → FoldInv W → Bound W :=
706   fun h => test_not_bound (h testWorld test_same_locus
    ↪ test_fold_inv)
707
708 end TimeLocus
709
710 namespace TimeLocus
711
712 /-- Residual Monism read as the Codex states it: one
    ↪ involution serves both routes. -/
713 def sigmaGeo (p : Int × Int) : Int × Int := (2 - p.1, p.2)

```

```

714 def sigmaForm (p : Int × Int) : Int × Int := (2 - p.1, p.2)
715 def OneInvolution : Prop := ∀ p, sigmaGeo p = sigmaForm p
716
717 /-- Residual Monism read as timeless: whether the zeros sit
    ↳ on the locus does not vary in time. -/
718 def P (W : World) (t : Nat) : Prop := ∀ p, W.zeros t p →
    ↳ onLine p
719 def Timeless (W : World) : Prop := ∀ t s, P W t ↔ P W s
720
721 /-- 53. Both readings hold in the counter-world: one
    ↳ involution by rfl, and timelessness because
722   the property "all zeros on the line" is constantly false
    ↳ there. Timeless and unbound. -/
723 theorem monism_holds_in_counter_world :
724   OneInvolution ∧ Timeless testWorld ∧ ¬ Bound testWorld :=
725   (fun _ => rfl, fun _ _ => Iff.rfl, test_not_bound)
726
727 /-- 54. What timeless monism does do: it makes one instant
    ↳ decide all time. With the seed, binding
728   follows; the seed is "every zero present at the instant
    ↳ is on the line". -/
729 theorem timeless_with_seed_binds (W : World) (h : SameLocus
    ↳ W) (u : Timeless W) (s0 : P W 0) :
730   Bound W :=
731   (bound_iff_line W h).mpr (fun t p z => ((u 0 t).mp s0) p z)
732
733 end TimeLocus
734
735 namespace TimeLocus
736
737 /-- THE CONSTRUCTED WITNESS from RESIDUAL-MONISM + TIME PSP,
    ↳ as asked: one involution, the
738   timeless interface (the Barzakh instant, t = 0, upstream
    ↳ of time), and the downstream times
739   agreeing with it. No field is a premise; each must be
    ↳ built as a term. -/
740 structure MonismTimeWitness (W : World) : Prop where
741   one_involution : OneInvolution
742   same_locus : SameLocus W
743   timeless : Timeless W
744   interface : P W 0 -- every zero present at
    ↳ the timeless interface is on the line
745
746 /-- 55. When it can be built, it proves the bound everywhere
    ↳ downstream. -/
747 theorem witness_proves_bound (W : World) (w :
    ↳ MonismTimeWitness W) : Bound W :=
748   timeless_with_seed_binds W w.same_locus w.timeless
    ↳ w.interface
749
750 /-- 55, completed. It can be built exactly when the bound
    ↳ already holds: its existence IS the line property. -/
751 theorem witness_iff_bound (W : World) (h : SameLocus W) :
752   Nonempty (MonismTimeWitness W) ↔ Bound W := by
753   refine (fun (w) => witness_proves_bound W w, fun b => ?_)
754   have l := (bound_iff_line W h).mp b
755   exact ({fun _ => rfl, h, fun t s => {fun _ p z => l s p z,
    ↳ fun _ p z => l t p z}, fun p z => l 0 p z})
756
757 /-- 56. Built at a model whose zeros are on the line, as the
    ↳ prior paper builds RA at one point. -/
758 def goodWorld : World := {fun _ p => onLine p, fun _ p => p =
    ↳ (1, 0)}
759 theorem constructed_at_model : Nonempty (MonismTimeWitness
    ↳ goodWorld) :=
760   (witness_iff_bound goodWorld (fun _ _ => Iff.rfl)).mpr (fun
    ↳ _ p z => by subst z; rfl)
761
762 /-- 57. Not buildable at the counter-world: the interface
    ↳ field fails there. -/
763 theorem not_constructible_off_line : ¬ Nonempty
    ↳ (MonismTimeWitness testWorld) :=
764   fun w => test_not_bound ((witness_iff_bound testWorld
    ↳ test_same_locus).mp w)
765
766 end TimeLocus
767
768 namespace TimeLocus
769
770 /-- Postulate M on an enumerated zero set. Stage k is when a
    ↳ zero is located; L t says every
771   zero located by stage t lies on the line; M says the
    ↳ truth of L t does not depend on t. -/

```

```

772 def Lstage {α : Type} (Z : α → Prop) (onL : α → Prop) (stage
    ↳ : α → Nat) (t : Nat) : Prop :=
773   ∀ z, Z z → stage z ≤ t → onL z
774 def PostulateM {α : Type} (Z : α → Prop) (onL : α → Prop)
    ↳ (stage : α → Nat) : Prop :=
775   ∀ t s, Lstage Z onL stage t ↔ Lstage Z onL stage s
776
777 /-- 49. POSTULATE M IS EQUIVALENT TO THE LINE PROPERTY, given
    ↳ the computed seed L 0 and an
778   exhaustive enumeration (every zero is located at some
    ↳ finite stage, built into `stage`). -/
779 theorem postulateM_iff_line {α : Type} (Z : α → Prop) (onL :
    ↳ α → Prop) (stage : α → Nat)
780   (seed : Lstage Z onL stage 0) :
781   PostulateM Z onL stage ↔ ∀ z, Z z → onL z := by
782   constructor
783   · intro m z hz
784     exact ((m 0 (stage z)).mp seed) z hz (Nat.le_refl _)
785   · intro h t s
786     exact (fun _ z hz _ => h z hz, fun _ z hz _ => h z hz)
787
788 /-- 50. And Postulate M is not free: it holds in a world
    ↳ whose zeros are all on the line, and
789   it cannot hold with the seed in a world with a located
    ↳ off-line zero. -/
790 theorem postulateM_decides {α : Type} (Z : α → Prop) (onL : α
    ↳ → Prop) (stage : α → Nat)
791   (seed : Lstage Z onL stage 0) (z : α) (hz : Z z) (off : ¬
    ↳ onL z) :
792   ¬ PostulateM Z onL stage :=
793   fun m => off ((postulateM_iff_line Z onL stage seed).mp m z
    ↳ hz)
794
795 end TimeLocus
796
797 #print axioms TimeLocus.bound_iff_line
798 #print axioms TimeLocus.time_does_not_bind
799 #print axioms TimeLocus.monism_holds_in_counter_world
800 #print axioms TimeLocus.timeless_with_seed_binds
801 #print axioms TimeLocus.witness_iff_bound
802 #print axioms TimeLocus.constructed_at_model
803 #print axioms TimeLocus.not_constructible_off_line
804 #print axioms TimeLocus.postulateM_iff_line
805 #print axioms TimeLocus.postulateM_decides
806
807 /- ## PART XII · The demand correctly addressed, and [.] at
    ↳ the collapse foundation. -/
808 namespace DotRH
809
810 inductive Token where
811   | sealed | broken | opn | dot
812   deriving DecidableEq, Repr
813
814 def Token.inEconomy : Token → Bool
815   | .dot => false
816   | _ => true
817
818 /-- I · THE COLLAPSE FOUNDATION and its stop. -/
819 structure Collapse (V : Prop) where
820   Reg : Type
821   reading : Reg → Prop
822   leg : ∀ r, reading r ↔ V
823
824 -- Theorem 58.
825 theorem self_referential_limit {V : Prop} (C : Collapse V) (r
    ↳ : s : C.Reg) :
826   (C.reading r ↔ V) ∧ (C.reading r ↔ C.reading s) :=
827   (C.leg r, (C.leg r).trans (C.leg s).symm)
828
829 def emitAtFoundation {V : Prop} (C : Collapse V) : Token :=
830   .dot
831
832 -- Theorem 59.
833 theorem value_marked_dot {V : Prop} (C : Collapse V) :
834   emitAtFoundation C = .dot := rfl
835
836 theorem dot_is_not_a_verdict :
837   Token.dot ≠ .sealed ∧ Token.dot ≠ .broken ∧ Token.dot ≠
838   .opn := by decide
839
840 theorem dot_outside_economy : Token.inEconomy .dot = false :=
841   rfl
842
843 def trivialCollapse (V : Prop) : Collapse V := (Unit, fun _
    ↳ => V, fun _ => Iff.rfl)
844
845

```

```

838 /-- Inside the collapse register, a demand for a verdict is a
839   ↪ ghost: the register returns
840   the same mark for the value and its negation. -/
841 -- Theorem 60.
842 theorem demand_is_ghost_in_register {V : Prop} (C : Collapse
843   ↪ V) (C' : Collapse (¬ V)) :
844   emitAtFoundation C = emitAtFoundation C' := rfl
845
846 /-- II · THE SELF-GROUNDING ROOT. A root R is self-grounding
847   ↪ when acts occur and every act of
848   adjudication, assent or denial, is itself an instance of
849   ↪ R. Then a denial of R re-enacts R,
850   no external proof adds to it, and a demand for one is a
851   ↪ ghost. This is the seat of the
852   analogy: a root that grounds itself cannot be proved from
853   ↪ outside itself. -/
854 structure SelfGrounding (R : Prop) where
855   Act      : Type
856   anAct    : Act
857   instances : Act → R
858
859 -- Theorem 61.
860 theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
861   ↪ (denial : G.Act) : R :=
862   G.instances denial
863
864 -- Theorem 61, completed.
865 theorem external_proof_adds_nothing {R : Prop} (G :
866   ↪ SelfGrounding R) (Q : Prop) :
867   (Q → R) ↔ R :=
868   (fun _ => G.instances G.anAct, fun r _ => r)
869
870 /-- The Root Axiom at the constructed domain is
871   ↪ self-grounding: every act is a deed, and a
872   deed actuates. -/
873 def RA : Prop := (0 : Int) < 1
874 def raSelfGrounding : SelfGrounding RA := (Unit, (), fun _ =>
875   ↪ show (0 : Int) < 1 by decide)
876
877 /-- III · A HYPOTHESIS ABOUT AN OBJECT is not self-grounding.
878   ↪ Its value is fixed by the object,
879   acts do not instance it, and a finite witness can refute
880   ↪ it. -/
881 structure Frame where
882   S : Type
883   τ : S → S
884   Z : S → Prop
885
886 def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
887
888 /-- A located zero off the line refutes the line property,
889   ↪ constructively. -/
890 theorem refuted_by_witness (X : Frame) (s : X.S) (hz : X.Z s)
891   ↪ (off : X.τ s ≠ s) :
892   ¬ LineProperty X :=
893   fun h => off (h s hz)
894
895 def twoPoint : Frame := (Bool, fun b => !b, fun _ => True)
896
897 /-- On a frame whose line property fails, no occurring act
898   ↪ can instance it: the line
899   property of a frame is not self-grounding. -/
900 -- Theorem 62.
901 theorem line_not_self_grounding : ¬ Nonempty (SelfGrounding
902   ↪ (LineProperty twoPoint)) :=
903   fun (G) => refuted_by_witness twoPoint true trivial (fun h
904     ↪ => Bool.noConfusion h) (G.instances G.anAct)
905
906 /-- IV · THE HARDENED FOUNDATION, in toto. The collapse marks
907   ↪ the value [.] and the demand is
908   a ghost inside that register; the self-grounding root
909   ↪ cannot be proved from outside and
910   re-enacts under denial; a hypothesis about an object is
911   ↪ witness-refutable and is not
912   self-grounding, so the self-grounding exemption does not
913   ↪ transfer to it. -/
914 theorem foundation_hardenened :
915   (∀ (V : Prop) (C : Collapse V), emitAtFoundation C =
916     ↪ .dot) ∧
917   Token.inEconomy .dot = false ∧
918   (∀ (V : Prop) (C : Collapse V) (C' : Collapse (¬ V)),
919     ↪ emitAtFoundation C = emitAtFoundation C') ∧
920   (∀ Q : Prop, (Q → RA) ↔ RA) ∧
921   (∀ a : Unit, RA ∧ raSelfGrounding.instances a =
922     ↪ raSelfGrounding.instances a) ∧
923   ¬ Nonempty (SelfGrounding (LineProperty twoPoint)) :=
924   {fun _ => rfl, rfl, fun _ => rfl,
925     ↪ external_proof_adds_nothing raSelfGrounding,
926     fun a => (raSelfGrounding.instances a, rfl),
927     ↪ line_not_self_grounding}
928
929 /-- V · THE DEMAND, CORRECTLY ADDRESSED.
930   "A proof of the Riemann Hypothesis cannot be demanded of
931   ↪ the foundation. Every reading the
932   foundation supplies is equivalent to the hypothesis, so
933   ↪ any derivation from those readings
934   alone would presuppose what it derives. The demand is
935   ↪ well-posed when addressed to the
936   object, the function zeta, whose structure fixes where
937   ↪ its zeros lie." -/
938
939 /-- A foundation resource is any property of frames the
940   ↪ foundation supplies. -/
941 def Resource := Frame → Prop
942
943 /-- (a) Misaddressed: any resource that also holds on a frame
944   ↪ where the line property fails
945   cannot, by itself, yield the line property on every frame
946   ↪ it covers. -/
947 -- Theorem 63.
948 theorem misaddressed_to_foundation (R : Resource) (hR : R
949   ↪ twoPoint) :
950   ¬ ∀ X, R X → LineProperty X :=
951   fun h => refuted_by_witness twoPoint true trivial (fun e =>
952     ↪ Bool.noConfusion e) (h twoPoint hR)
953
954 /-- (b) Presupposition: a derivation of the value from a
955   ↪ reading equivalent to it uses the
956   value's own content; the reading and the value stand or
957   ↪ fall together. -/
958 -- Theorem 64.
959 theorem derivation_presupposes {V P : Prop} (leg : P ↔ V) :
960   (P → V) ∧ (V → P) ∧ (¬ V → ¬ P) :=
961   {leg.mp, leg.mpr, fun nv p => nv (leg.mp p)}
962
963 /-- (c) Well-addressed: the object decides. A frame settles
964   ↪ its own line property, in one
965   direction by a located off-line witness, in the other by
966   ↪ its own structure. -/
967 def onLineFrame : Frame := (Unit, fun u => u, fun _ => True)
968
969 -- Theorem 65.
970 theorem object_decides :
971   ¬ LineProperty twoPoint ∧ LineProperty onLineFrame :=
972   (refuted_by_witness twoPoint true trivial (fun e =>
973     ↪ Bool.noConfusion e), fun _ => rfl)
974
975 /-- THE DEMAND, IN TOTO. Made to the foundation, it is
976   ↪ misaddressed and any answer from
977   there presupposes the value; made to the object, it is
978   ↪ well-posed and the object answers. -/
979 -- Theorem 66.
980 theorem demand_correctly_addressed :
981   (∀ R : Resource, R twoPoint → ¬ ∀ X, R X → LineProperty
982     ↪ X) ∧
983   (∀ {V P : Prop}, (P ↔ V) → (¬ V → ¬ P)) ∧
984   (¬ LineProperty twoPoint ∧ LineProperty onLineFrame) :=
985   {misaddressed_to_foundation, fun leg =>
986     ↪ (derivation_presupposes leg).2.2, object_decides}
987
988 end DotRH
989
990 namespace DotRH
991 /-- Theorem 67. Attached to the paper's cone: the readings of
992   ↪ Parts X and XI collapse onto
993   the hypothesis, and the foundation marks the value [.]
994   ↪ -/
995 def rhCollapse (H : RALi.Hunt) : Collapse H.L.RH :=
996   ↪ (RALi.Reg5, RALi.D5 H, RALi.hunt_cone H)
997 theorem rh_marked_dot (H : RALi.Hunt) : emitAtFoundation
998   ↪ (rhCollapse H) = .dot := rfl
999 end DotRH
1000
1001 #print axioms DotRH.self_referential_limit
1002 #print axioms DotRH.value_marked_dot
1003 #print axioms DotRH.dot_is_not_a_verdict
1004 #print axioms DotRH.demand_is_ghost_in_register

```

```

955 #print axioms DotRH.denial_reenacts_root
956 #print axioms DotRH.external_proof_adds_nothing
957 #print axioms DotRH.refuted_by_witness
958 #print axioms DotRH.line_not_self_grounding
959 #print axioms DotRH.foundation_hardened
960 #print axioms DotRH.misaddressed_to_foundation
961 #print axioms DotRH.derivation_presupposes
962 #print axioms DotRH.object_decides
963 #print axioms DotRH.demand_correctly_addressed
964 #print axioms DotRH.rh_marked_dot
965
966 /-! ## PART XIII · The six-register cone: Postulate M joined
    ↳ to the apex. -/
967 namespace SixCone
968
969 /-- The hunt with one more reading. The zeros are enumerated
    ↳ by stage, the seed holds, and
970 the hypothesis is, by its definition, the line property
    ↳ of the enumerated zeros; that
971 identification enters as the hypothesis `hDef`, carried
    ↳ by citation as Li's criterion and
972 Newman's equivalence are. -/
973 structure Hunt6 extends RALi.Hunt where
974   α      : Type
975   Z      : α → Prop
976   onL    : α → Prop
977   stage  : α → Nat
978   seed   : TimeLocus.Lstage Z onL stage 0
979   hDef   : L.RH ↔ ∀ z, Z z → onL z
980
981 inductive Reg6 where
982   | five (r : RALi.Reg5)
983   | postulateM
984
985 def D6 (H : Hunt6) : Reg6 → Prop
986   | .five r      => RALi.D5 H.toHunt r
987   | .postulateM => TimeLocus.PostulateM H.Z H.onL H.stage
988
989 -- Theorem 68.
990 /-- The cone over six readings: the five of Part X and
    ↳ Postulate M, every leg an equivalence
991 with the hypothesis. -/
992 theorem hunt_cone6 (H : Hunt6) : ∀ r, D6 H r ↔ H.L.RH := by
993   intro r
994   cases r with
995   | five r      => exact RALi.hunt_cone H.toHunt r
996   | postulateM =>
997     exact (TimeLocus.postulateM_iff_line H.Z H.onL H.stage
    ↳ H.seed).trans H.hDef.symm
998
999 -- Theorem 69.
1000 /-- The collapse foundation over six readings, and its mark
    ↳ on the value. -/
1001 def rhCollapse6 (H : Hunt6) : DotRH.Collapse H.L.RH := (Reg6,
    ↳ D6 H, hunt_cone6 H)
1002 theorem rh_marked_dot6 (H : Hunt6) : DotRH.emitAtFoundation
    ↳ (rhCollapse6 H) = .dot := rfl
1003
1004 end SixCone
1005
1006 #print axioms SixCone.hunt_cone6
1007 #print axioms SixCone.rh_marked_dot6
1008
1009 namespace DotRH
1010 -- Theorem 63, general form.
1011 /-- Any resource that also holds on some frame whose line
    ↳ property fails cannot, by itself,
1012 yield the line property on every frame it covers. -/
1013 theorem misaddressed_general (R : Resource) (Y : Frame) (hY :
    ↳ ¬ LineProperty Y) (hR : R Y) :
1014   ¬ ∀ X, R X → LineProperty X :=
1015   fun h => hY (h Y hR)
1016 end DotRH
1017
1018 #print axioms DotRH.misaddressed_general
1019
1020 /-! ## PART XIV · The logical form of the hypothesis: the
    ↳ only existence lives in the denial. -/
1021 namespace LogicalForm
1022
1023 -- Theorem 70.
1024 /-- A universal sentence carries no existential import: over
    ↳ an empty zero set the line

```

```

1025   property holds vacuously. The hypothesis posits no
    ↳ object. -/
1026 theorem universal_posits_nothing {α : Type} (onL : α → Prop)
    ↳ :
1027   ∀ z : α, (fun _ => False) z → onL z :=
1028   fun _ h => h.elim
1029
1030 -- Theorem 71.
1031 /-- The denial posits an object: a located zero off the line
    ↳ refutes the hypothesis. -/
1032 theorem denial_posits_a_witness {α : Type} (Z onL : α → Prop)
    ↳ :
1033   (∃ z, Z z ∧ ¬ onL z) → ¬ ∀ z, Z z → onL z :=
1034   fun (z, hz, off) h => off (h z hz)
1035
1036 -- Theorem 72.
1037 /-- With a decidable line predicate, the hypothesis fails
    ↳ only by a witness: it holds iff no
1038 off-line zero exists. The denial carries the whole
    ↳ existential load. -/
1039 theorem fails_only_by_witness {α : Type} (Z onL : α → Prop)
    ↳ (∀ z, Decidable (onL z)) :
1040   (∀ z, Z z → onL z) ↔ ¬ ∃ z, Z z ∧ ¬ onL z := by
1041     constructor
1042     · intro h (z, hz, off); exact off (h z hz)
1043     · intro h z hz
1044       exact Decidable.byContradiction (fun off => h (z, hz,
    ↳ off))
1045
1046 -- Theorem 73.
1047 /-- A witness is a finite check: for a decidable predicate on
    ↳ the naturals, one index at which
1048 the check fails refutes the universal, and the check at
    ↳ that index is a computation. -/
1049 theorem witness_is_a_finite_check (bad : Nat → Bool) (n :
    ↳ Nat) (h : bad n = true) :
1050   ¬ ∀ m, bad m = false :=
1051   fun hall => by rw [hall n] at h; cases h
1052
1053 end LogicalForm
1054
1055 #print axioms LogicalForm.universal_posits_nothing
1056 #print axioms LogicalForm.denial_posits_a_witness
1057 #print axioms LogicalForm.fails_only_by_witness
1058 #print axioms LogicalForm.witness_is_a_finite_check
1059
1060 /-! ## PART XV · The two "can't"s are not equal: the denial
    ↳ carries the burden.
1061 Abstract: a proposition RH, the foundation's provability
    ↳ predicate, and the property that makes
1062 the directions unequal,  $\Sigma_1$ -completeness: a false RH has a
    ↳ finite witness, and the foundation
1063 proves the denial from it. That property and soundness on the
    ↳ denial are carried as named
1064 hypotheses, cited from arithmetic, not proved here. -/
1065 namespace DirCant
1066
1067 structure Setting where
1068   RH      : Prop
1069   Prov    : Prop → Prop
1070   /--  $\Sigma_1$ -completeness, carried as a hypothesis: a false RH is
    ↳ refutable. -/
1071   sigma1  : ¬ RH → Prov (¬ RH)
1072   /-- Soundness of the foundation on the denial. -/
1073   sound_neg : Prov (¬ RH) → ¬ RH
1074
1075 -- Theorem 74.
1076 /-- THE SEALED DIRECTION. If the foundation cannot refute the
    ↳ hypothesis, the hypothesis holds:
1077 the denial's "can't" is decisive. -/
1078 theorem cant_refute_seals (S : Setting) [Decidable S.RH] (h :
    ↳ ¬ S.Prov (¬ S.RH)) : S.RH :=
1079   Decidable.byContradiction (fun n => h (S.sigma1 n))
1080
1081 -- Theorem 75.
1082 /-- Under soundness on the denial, the hypothesis holds
    ↳ exactly when the foundation cannot refute
1083 it: the denial's "can't" and the hypothesis are one
    ↳ statement. -/
1084 theorem rh_iff_cant_refute (S : Setting) [Decidable S.RH] :
    ↳ S.RH ↔ ¬ S.Prov (¬ S.RH) :=
1085   (fun r p => S.sound_neg p r, cant_refute_seals S)
1086

```

```

1087 -- Theorem 76.
1088 /- The assent's "can't" decides nothing: a setting where the
1089   hypothesis holds and the
1090   foundation cannot prove it satisfies every field. -/
1091 def independentTrue : Setting :=
1092   (True, fun _ => False, fun n => absurd trivial n, fun p =>
1093     ↪ p.elim)
1094
1095 theorem cant_prove_does_not_seal_false :
1096   independentTrue.RH  $\wedge$   $\neg$  independentTrue.Prov
1097   ↪ independentTrue.RH :=
1098   (trivial, id)
1099
1100 -- Theorem 77.
1101 /- THE ASYMMETRY. Can't-refute seals the hypothesis;
1102   can't-prove is compatible with its truth.
1103   The burden of the one bit lies on the denial side. -/
1104 theorem asymmetry :
1105   ( $\forall$  (S : Setting) [Decidable S.RH],  $\neg$  S.Prov ( $\neg$  S.RH)  $\rightarrow$ 
1106     ↪ S.RH)  $\wedge$ 
1107   ( $\exists$  S : Setting, S.RH  $\wedge$   $\neg$  S.Prov S.RH) :=
1108   (fun S _ h => cant_refute_seals S h, (independentTrue,
1109     ↪ cant_prove_does_not_seal_false))
1110
1111 end DirCant
1112
1113 #print axioms DirCant.cant_refute_seals
1114 #print axioms DirCant.rh_iff_cant_refute
1115 #print axioms DirCant.cant_prove_does_not_seal_false
1116 #print axioms DirCant.asymmetry

```

APPENDIX G: THE KERNEL AUDIT OF RA_LI_BRIDGE.LEAN, VERBATIM

lean RA_Li_Bridge.lean, exit 0, no warnings, no errors.

```

1 'RALi.bridge_yields_RH' does not depend on any axioms
2 'RALi.bridge_is_keyed' does not depend on any axioms
3 'RALi.no_uniform_bridge' does not depend on any axioms
4 'RALi.li_is_a_bit_stream' does not depend on any axioms
5 'RALi.xi0_retires_iff_unity' depends on axioms: [propext]
6 'RALi.retired_seal_is_sound' does not depend on any axioms
7 'RALi.retired_grade_capped' does not depend on any axioms
8 'RALi.unity_is_keyed' does not depend on any axioms
9 'RALi.ghost' does not depend on any axioms
10 'RALi.common_phi_is_uniform' does not depend on any axioms
11 'RALi.blind_constructions_die' does not depend on any axioms
12 'RALi.mertens_nonneg' depends on axioms: [propext,
13   ↪ Quot.sound]
14 'RALi.unitary_iff_on_line' depends on axioms: [propext,
15   ↪ Quot.sound]
16 'RALi.off_line_forces_growth' depends on axioms: [propext,
17   ↪ Quot.sound]
18 'RALi.stability_iff_line' depends on axioms: [propext,
19   ↪ Quot.sound]
20 'RALi.one_repeats_one' does not depend on any axioms
21 'RALi.the_step_is_the_claim' does not depend on any axioms
22 'RALi.monism_posit_forces' does not depend on any axioms
23 'RALi.finite_never_forces' does not depend on any axioms
24 'RALi.mixed_is_fold_invariant' does not depend on any axioms
25 'RALi.mixed_recurrence_fails' does not depend on any axioms
26 'RALi.full_gives_actual' does not depend on any axioms
27 'RALi.actual_not_full' does not depend on any axioms
28 'RALi.sufficient_time' does not depend on any axioms
29 'RALi.one_involution_constructed' does not depend on any
30   ↪ axioms
31 'RALi.timeless_equals_timed' does not depend on any axioms
32 'RALi.monism_closes' does not depend on any axioms
33 'RALi.monism_witness_is_the_claim' does not depend on any
34   ↪ axioms
35 'RALi.constructed_monism' does not depend on any axioms
36 'RALi.monism_absent_on_breaks' does not depend on any axioms
37 'RALi.monism_yields_RH' depends on axioms: [propext,
38   ↪ Quot.sound]
39 'RALi.monism_on_li_is_RH' depends on axioms: [propext,
40   ↪ Quot.sound]
41 'RALi.global_monism_inconsistent' does not depend on any
42   ↪ axioms
43 'RALi.anchored_closes' does not depend on any axioms

```

```

35 'RALi.anchored_derives_monism' does not depend on any axioms
36 'RALi.lyapunov_is_monism' depends on axioms: [propext]
37 'RALi.closure_grade_is_transport_grade' does not depend on
38   ↪ any axioms
39 'RALi.rh_anchor_is_the_claim' depends on axioms: [propext,
40   ↪ Quot.sound]
41 'RALi.flow_monotone' depends on axioms: [propext, Quot.sound]
42 'RALi.reality_transport' depends on axioms: [propext,
43   ↪ Quot.sound]
44 'RALi.rh_iff_lambda_zero' depends on axioms: [propext,
45   ↪ Quot.sound]
46 'RALi.future_closed' depends on axioms: [propext, Quot.sound]
47 'RALi.flowed_record_forgets' depends on axioms: [propext,
48   ↪ Quot.sound]
49 'RALi.upstream_is_not_forced' depends on axioms: [propext,
50   ↪ Quot.sound]
51 'RALi.hunt_cone' depends on axioms: [propext, Quot.sound]
52 'RALi.one_gap' depends on axioms: [propext, Quot.sound]
53 'RALi.regress_halts' depends on axioms: [propext, Quot.sound]
54 'RALi.nothing_weaker' does not depend on any axioms
55 'RALi.sufficient_closes_all' depends on axioms: [propext,
56   ↪ Quot.sound]
57 'TimeLocus.bound_iff_line' does not depend on any axioms
58 'TimeLocus.time_does_not_bind' does not depend on any axioms
59 'TimeLocus.monism_holds_in_counter_world' does not depend on
60   ↪ any axioms
61 'TimeLocus.timeless_with_seed_binds' does not depend on any
62   ↪ axioms
63 'TimeLocus.witness_iff_bound' does not depend on any axioms
64 'TimeLocus.constructed_at_model' does not depend on any
65   ↪ axioms
66 'TimeLocus.not_constructible_off_line' does not depend on any
67   ↪ axioms
68 'TimeLocus.postulateM_iff_line' does not depend on any axioms
69 'TimeLocus.postulateM_decides' does not depend on any axioms
70 'DotRH.self_referential_limit' does not depend on any axioms
71 'DotRH.value_marked_dot' does not depend on any axioms
72 'DotRH.dot_is_not_a_verdict' does not depend on any axioms
73 'DotRH.demand_is_ghost_in_register' does not depend on any
74   ↪ axioms
75 'DotRH.denial_reenacts_root' does not depend on any axioms
76 'DotRH.external_proof_adds_nothing' does not depend on any
77   ↪ axioms
78 'DotRH.refuted_by_witness' does not depend on any axioms
79 'DotRH.line_not_self_grounding' does not depend on any axioms
80 'DotRH.foundation_harden' does not depend on any axioms
81 'DotRH.misaddressed_to_foundation' does not depend on any
82   ↪ axioms
83 'DotRH.derivation_presupposes' does not depend on any axioms
84 'DotRH.object_decides' does not depend on any axioms
85 'DotRH.demand_correctly_addressed' does not depend on any
86   ↪ axioms
87 'DotRH.rh_marked_dot' depends on axioms: [propext,
88   ↪ Quot.sound]
89 'SixCone.hunt_cone6' depends on axioms: [propext, Quot.sound]
90 'SixCone.rh_marked_dot6' depends on axioms: [propext,
91   ↪ Quot.sound]
92 'DotRH.misaddressed_general' does not depend on any axioms
93 'LogicalForm.universal_posits_nothing' does not depend on any
94   ↪ axioms
95 'LogicalForm.denial_posits_a_witness' does not depend on any
96   ↪ axioms
97 'LogicalForm.fails_only_by_witness' does not depend on any
98   ↪ axioms
99 'LogicalForm.witness_is_a_finite_check' does not depend on
100   ↪ any axioms
101 'DirCant.cant_refute_seals' does not depend on any axioms
102 'DirCant.rh_iff_cant_refute' does not depend on any axioms
103 'DirCant.cant_prove_does_not_seal_false' does not depend on
104   ↪ any axioms
105 'DirCant.asymmetry' does not depend on any axioms

```

Appendix G, completed: the seventeen theorems the file does not print

The file prints 83 dependency sets. A supplementary run of the same file, with one #print axioms line appended for each of the 17 theorems it does not print, exit 0: 9 axiom-free, 8 on propext and Quot.sound,

none on `Classical.choice`; 100 sets in all, 70 axiom-free.

```

1 'RALi.ra_holds_in_model' does not depend on any axioms
2 'RALi.mertens_identity' depends on axioms: [propext,
  ↳ Quot.sound]
3 'RALi.Nl_sub_N0' depends on axioms: [propext, Quot.sound]
4 'RALi.grows_left' depends on axioms: [propext, Quot.sound]
5 'RALi.dies_right' depends on axioms: [propext, Quot.sound]
6 'RALi.real_at_Lam' depends on axioms: [propext, Quot.sound]
7 'RALi.iter_succ' depends on axioms: [propext, Quot.sound]
8 'RALi.leg_bridge' does not depend on any axioms
9 'RALi.leg_unity' does not depend on any axioms
10 'RALi.leg_monism' depends on axioms: [propext, Quot.sound]
11 'RALi.leg_transport' depends on axioms: [propext, Quot.sound]
12 'RALi.leg_upstream' does not depend on any axioms
13 'TimeLocus.test_same_locus' does not depend on any axioms
14 'TimeLocus.test_fold_inv' does not depend on any axioms
15 'TimeLocus.test_not_bound' does not depend on any axioms
16 'TimeLocus.witness_proves_bound' does not depend on any axioms
17 'DotRH.dot_outside_economy' does not depend on any axioms

```

APPENDIX H: THE KERNEL FILES OF PART III

Nine files, each compiled under core Lean 4.19.0 at exit 0 with zero warnings, no sorry, and no axiom declared: forty-five dependency sets, forty-two axiom-free and three on `propext` alone, none on `Classical.choice`; the tenth file of Part III, the formal block with its nine sets, is Appendix I. Then the Turing run on ζ to height 100, in floating point, testimony and not a certificate.

RH_Gold_And_Lead.lean

SHA-256 f5a399cd986dd472d36dab7b802e97260d0a93c66d73c336f72a905fc12253de, 70 lines.

```

1 /-
2 RH_Gold_And_Lead.lean · why the Real part and the Unicorn
  ↳ part should not be mixed.
3 Core Lean 4.19.0, no library, no sorry. Drafted 2026-09-24.
4 Companion to RH_Real_Unicorn_Split.lean, where RH  $\Leftrightarrow$  Real
  ↳ part  $\wedge$  Unicorn part at every height.
5 A statement's standing is proved, open, or refuted. A
  ↳ conjunction stands only as high as its
6 weakest part. So the conventional RH, the two parts fused,
  ↳ inherits the Unicorn part's standing:
7 the gold of the Real part does not survive the alloy.
8 -/
9 namespace GoldLead
10
11 /- The three standings of a statement, ranked. -/
12 inductive Standing | refuted | opn | proved
13   deriving DecidableEq, Repr
14
15 def Standing.rank : Standing → Nat
16   | .refuted => 0 | .opn => 1 | .proved => 2
17
18 /- A conjunction stands at the weaker of its two parts. -/
19 def join (a b : Standing) : Standing := if a.rank ≤ b.rank
  ↳ then a else b
20
21 -- G1.
22 /- The alloy never outranks either metal. -/
23 theorem join_below_both (a b : Standing) :
24   (join a b).rank ≤ a.rank  $\wedge$  (join a b).rank ≤ b.rank := by
25   cases a <|> cases b <|> decide
26
27 -- G2.
28 /- THE FUSED RH. The Real part stands proved (certified to 3
  ↳  $\times 10^{12}$ ), the Unicorn part stands
29   open; fused, the conventional RH stands open. -/

```

```

30 def realPart : Standing := .proved
31 def unicornPart : Standing := .opn
32 def fusedRH : Standing := join realPart unicornPart
33
34 theorem fused_rh_is_open : fusedRH = .opn := by decide
35
36 -- G3.
37 /- GOLD NO LONGER PURE. Fusion strictly lowers the Real
  ↳ part's standing: kept apart it is proved,
38   carried inside the fused statement it is only open. -/
39 theorem fusion_lowers_the_gold : fusedRH.rank < realPart.rank
  ↳ := by decide
40
41 -- G4.
42 /- Logically, every proof of the fused statement carries a
  ↳ proof of the Unicorn part, and every
43   refutation of the Unicorn part refutes the fused
  ↳ statement, whatever the Real part holds. -/
44 theorem fused_carries_unicorn (Real Unicorn : Prop) :
45   ((Real  $\wedge$  Unicorn)  $\rightarrow$  Unicorn)  $\wedge$  ( $\neg$  Unicorn  $\rightarrow \neg$  (Real  $\wedge$ 
  ↳ Unicorn)) :=
46   (fun h => h.2, fun hU => hU h.2)
47
48 -- G5.
49 /- Kept apart, each part keeps its own standing: the Real
  ↳ part is asserted exactly as proved,
50   with nothing borrowed from the Unicorn part, and the
  ↳ Unicorn part is asserted exactly as open. -/
51 theorem kept_apart (Real Unicorn : Prop) (hReal : Real) :
52   Real  $\wedge$  ((Real  $\wedge$  Unicorn)  $\leftrightarrow$  Unicorn) :=
53   (hReal, (fun h => h.2, fun hU => (hReal, hU)))
54
55 -- G6.
56 /- THE RULE, IN ONE THEOREM. Given the Real part, the fused
  ↳ RH is exactly the Unicorn part, and
57   fusing lowers the Real part from proved to open. So state
  ↳ the Real part as gold, the Unicorn
58   part as the open bit, and do not alloy them. -/
59 theorem do_not_alloy (Real Unicorn : Prop) (hReal : Real) :
60   ((Real  $\wedge$  Unicorn)  $\leftrightarrow$  Unicorn)  $\wedge$  fusedRH = .opn  $\wedge$ 
  ↳ fusedRH.rank < realPart.rank :=
61   ((kept_apart Real Unicorn hReal).2, fused_rh_is_open,
  ↳ fusion_lowers_the_gold)
62
63 end GoldLead
64
65 #print axioms GoldLead.join_below_both
66 #print axioms GoldLead.fused_rh_is_open
67 #print axioms GoldLead.fusion_lowers_the_gold
68 #print axioms GoldLead.fused_carries_unicorn
69 #print axioms GoldLead.kept_apart
70 #print axioms GoldLead.do_not_alloy

```

```

1 'GoldLead.join_below_both' does not depend on any axioms
2 'GoldLead.fused_rh_is_open' does not depend on any axioms
3 'GoldLead.fusion_lowers_the_gold' does not depend on any
  ↳ axioms
4 'GoldLead.fused_carries_unicorn' does not depend on any
  ↳ axioms
5 'GoldLead.kept_apart' does not depend on any axioms
6 'GoldLead.do_not_alloy' does not depend on any axioms

```

RH_Unicorn_Isomorphism.lean

SHA-256 0c3796bb1f36a429f85824edee71c1e177280523650db313835c0b79672b0ef3, 48 lines.

```

1 /-
2 RH_Unicorn_Isomorphism.lean · the Riemann Hypothesis and the
  ↳ unicorn condition are one statement.
3 Core Lean 4.19.0, no library, no sorry. Harvested 2026-09-24.
4 Scope: an equivalence (RH  $\Leftrightarrow$  the off-line zeros are
  ↳ unicorns), not a derivation of RH.
5 "Every white unicorn has its white fur attached only to its
  ↳ body" is true because there are no
6 unicorns: a universal over an empty class holds of anything.
  ↳ Here the class is the off-line zeros.

```

```

7  -/
8  namespace Unicorn
9
10 variable {α : Type} (Z onL : α → Prop)
11
12 /-- The off-line zeros: zeros not on the line. -/
13 def OffLine (z : α) : Prop := Z z ∧ ¬ onL z
14
15 /-- The hypothesis: every zero is on the line. -/
16 def RH : Prop := ∀ z, Z z → onL z
17
18 /-- The off-line zeros are unicorns: EVERY property holds of
19   ↳ all of them, vacuously. -/
19 def Unicorns : Prop := ∀ (Q : α → Prop) (z : α), OffLine Z
20   ↳ onL z → Q z
21
22 -- U1.
22 /-- If the hypothesis holds, the off-line zeros are unicorns:
23   ↳ anything at all is true of them. -/
23 theorem rh_makes_unicorns : RH Z onL → Unicorns Z onL :=
24   fun h _ z (hz, off) => absurd (h z hz) off
25
26 -- U2.
27 /-- If the off-line zeros are unicorns, the hypothesis holds
28   ↳ (with a decidable line predicate):
28   take the property "false"; it holds of every off-line
29   ↳ zero, so none exists. -/
29 theorem unicorns_make_rh [V z, Decidable (onL z)] : Unicorns
30   ↳ Z onL → RH Z onL :=
31   fun h z hz => Decidable.byContradiction (fun off => h (fun
32     ↳ _ => False) z (hz, off))
33
34 -- U3.
34 /-- THE SEAL. The Riemann Hypothesis is exactly the statement
35   ↳ that the off-line zeros are
35   unicorns: a class of which every sentence is vacuously
36   ↳ true, because it has no member. -/
35 theorem rh_iff_unicorns [V z, Decidable (onL z)] : RH Z onL ↔
36   ↳ Unicorns Z onL :=
37   (rh_makes_unicorns Z onL, unicorns_make_rh Z onL)
38
39 -- U4.
39 /-- And the unicorn class is empty exactly when no off-line
40   ↳ zero can be exhibited. -/
40 theorem unicorns_iff_no_witness : Unicorns Z onL ↔ ¬ ∃ z,
41   ↳ OffLine Z onL z :=
42   (fun h (z, hz) => h (fun _ => False) z hz, fun h _ z hz =>
43     ↳ absurd (z, hz) h)
44
45 end Unicorn
46
47 #print axioms Unicorn.rh_makes_unicorns
48 #print axioms Unicorn.unicorns_make_rh
49 #print axioms Unicorn.rh_iff_unicorns
50 #print axioms Unicorn.unicorns_iff_no_witness

```

```

1  'Unicorn.rh_makes_unicorns' does not depend on any axioms
2  'Unicorn.unicorns_make_rh' does not depend on any axioms
3  'Unicorn.rh_iff_unicorns' does not depend on any axioms
4  'Unicorn.unicorns_iff_no_witness' does not depend on any
   ↳ axioms

```

RH_Real_Unicorn_Split.lean

SHA-256 6979d9a5aac8ef2651e4984d11cf1b03542e85993539dac513541adee285235, 72 lines.

```

1  -/
2  RH_Real_Unicorn_Split.lean · the Riemann Hypothesis divided
3    ↳ into a Real part and a Unicorn part.
4  Core Lean 4.19.0, no library, no sorry. Drafted 2026-09-24.
5  Split at a height T. The Real part is the witnessed region:
6    ↳ every zero at height ≤ T lies on the
7    line. The Unicorn part is the unwitnessed region: above T,
8    ↳ the off-line zeros are unicorns, a
9    class of which every sentence is vacuously true because it
10   ↳ has no member.

```

```

7  -/
8  namespace Split
9
10 variable {α : Type} (Z onL : α → Prop) (height : α → Int)
11
12 def RH : Prop := ∀ z, Z z → onL z
13
14 /-- THE REAL PART at height T: every zero up to T is on the
15   ↳ line. -/
15 def RealPart (T : Int) : Prop := ∀ z, Z z → height z ≤ T →
16   ↳ onL z
17
18 /-- THE UNICORN PART at height T: above T, every property
19   ↳ holds of every off-line zero, because
20   there are none. -/
19 def UnicornPart (T : Int) : Prop :=
21   ↳ ∀ (Q : α → Prop) (z : α), T < height z → Z z → ¬ onL z → Q
22   ↳ z
23
24 -- S1.
24 /-- THE SPLIT. At every height T, the hypothesis is exactly
25   ↳ the Real part and the Unicorn part
25   together. Nothing is lost and nothing is added. -/
25 theorem rh_iff_real_and_unicorn [V z, Decidable (onL z)] (T :
26   ↳ Int) :
26   RH Z onL ↔ RealPart Z onL height T ∧ UnicornPart Z onL
27   ↳ height T := by
28   constructor
29   · intro h
30     exact (fun z hz _ => h z hz, fun _ z _ hz off => absurd
31       ↳ (h z hz) off)
32   · intro (hR, hU) z hz
33     by_cases hle : height z ≤ T
34     · exact hR z hz hle
35     · exact Decidable.byContradiction
36       (fun off => hU (fun _ => False) z (Int.lt_of_not_ge
37         ↳ hle) hz off)
38
39 -- S2.
39 /-- The parts are separate: the Real part can hold while the
40   ↳ Unicorn part fails. An off-line
40   zero above T leaves every zero up to T on the line. -/
40 def twoZeros (z : Int × Bool) : Prop := z = (5, true) ∨ z =
41   ↳ (20, false)
42 def onLine2 (z : Int × Bool) : Prop := z.2 = true
43 def height2 (z : Int × Bool) : Int := z.1
44
45 theorem parts_are_separate :
46   RealPart twoZeros onLine2 height2 10 ∧ ¬ UnicornPart
47   ↳ twoZeros onLine2 height2 10 := by
48   refine (?, ?)
49   · intro z hz hle
50     rcases hz with h | h <=> subst h
51     · rfl
52     · exact absurd hle (by decide)
53   · intro hU
54     exact hU (fun _ => False) (20, false) (by decide) (0r.inr
55       ↳ rfl) (fun h => Bool.noConfusion h)
56
57 -- S3.
57 /-- The Unicorn part is empty exactly when no off-line zero
58   ↳ above T can be exhibited. -/
58 theorem unicorn_part_iff_no_witness (T : Int) :
59   UnicornPart Z onL height T ↔ ¬ ∃ z, T < height z ∧ Z z ∧
60   ↳ ¬ onL z :=
61   (fun hU (z, hT, hz, off) => hU (fun _ => False) z hT hz
62     ↳ off,
63     fun h _ z hT hz off => absurd (z, hT, hz, off) h)
64
65 -- S4.
65 /-- Raising T moves zeros from the Unicorn part into the Real
66   ↳ part and never the other way: a
66   Real part at a higher height contains the Real part at a
67   ↳ lower one. -/
66 theorem real_part_monotone (T T' : Int) (hTT : T ≤ T') :
67   RealPart Z onL height T' → RealPart Z onL height T :=
68   fun h z hz hle => h z hz (Int.le_trans hle hTT)
69
70 end Split
71
72 #print axioms Split.rh_iff_real_and_unicorn
73 #print axioms Split.parts_are_separate
74 #print axioms Split.unicorn_part_iff_no_witness

```

```
72 #print axioms Split.real_part_monotone
```

```
1 'Split.rh_iff_real_and_unicorn' depends on axioms: [propext]
2 'Split.parts_are_separate' does not depend on any axioms
3 'Split.unicorn_part_iff_no_witness' does not depend on any
  ↳ axioms
4 'Split.real_part_monotone' depends on axioms: [propext]
```

RAM_ZFC_Placement.lean

SHA-256 d7b0d3da00ca3f6e1340440a775beeb539599b66a9f189deecade5309fb3086b, 85 lines.

```
1 /-
2 RAM_ZFC_Placement.lean · ZFC subsumed by RAM, and ZFC's
  ↳ location under RAM.
3 Core Lean 4.19.0, no library, no sorry, no axiom declaration.
  ↳ Harvested 2026-09-24.
4
5 Location. ZFC is a ladder at L2m (provability); its intended
  ↳ world, the cumulative hierarchy and
6 the numbers inside it, is the Ground at L1m (formal being as
  ↳ grounding); its computable part is
7 the rung at L3m. The order rung ≤ ladder ≤ Ground holds under
  ↳ two named premises: the rung on
8 the ladder (for ZFC,  $\Sigma_1$ -completeness, a theorem of
  ↳ arithmetic) and soundness (which ZFC cannot
9 prove of itself, Gödel II, so it is carried openly).
10
11 Subsumption is by placement: RAM locates ZFC, as it locates
  ↳ NBG, PA, and every other ladder; it
12 does not derive ZFC's axioms and does not replace ZFC's
  ↳ proofs. Any theory, ZFC included, is
13 modelled abstractly: sentences, a provability predicate, a
  ↳ decidable fragment, and truth in the
14 intended world.
15 -/
16 namespace RAMZFC
17
18 structure Theory where
19   Sent : Type
20   Prov : Sent → Prop      -- L2m, the ladder
21   Comp : Sent → Prop      -- L3m, the rung: sentences
22   ↳ settled by finite computation
23   True_ : Sent → Prop      -- L1m, the Ground: truth in the
24   ↳ intended world
25
26 /- The rung is on the ladder: what finite computation
27   ↳ settles, the theory proves.
28   (For ZFC:  $\Sigma_1$ -completeness, a theorem of arithmetic,
29   ↳ carried as a named premise.) -/
30 def RungOnLadder (T : Theory) : Prop := ∀ s, T.Comp s →
31   ↳ T.Prov s
32
33 /- The ladder reaches only the Ground: soundness. For ZFC
34   ↳ this is exactly what ZFC cannot
35   prove of itself (Gödel II); it is carried openly, never
36   ↳ derived. -/
37 def Sound (T : Theory) : Prop := ∀ s, T.Prov s → T.True_ s
38
39 -- Z1.
40 /- THE PLACEMENT. Under the two named premises, every theory
41   ↳ sits in RAM's strata in order:
42   rung ≤ ladder ≤ Ground. -/
43 theorem ram_places_theory (T : Theory) (hR : RungOnLadder T)
44   ↳ (hS : Sound T) :
45   (∀ s, T.Comp s → T.Prov s) ∧ (∀ s, T.Prov s → T.True_ s)
46   ↳ ∧ (∀ s, T.Comp s → T.True_ s) :=
47   (hR, hS, fun s h => hS s (hR s h))
48
49 -- Z2.
50 /- The placement is not free: an unsound theory breaks
51   ↳ ladder ≤ Ground. -/
52 def unsound : Theory := (Bool, fun _ => True, fun _ => False,
53   ↳ fun b => b = true)
54
55 theorem soundness_is_load_bearing :
```

```
44   RungOnLadder unsound ∧ ¬ Sound unsound := by
45   refine (fun _ h => h.elim, fun h => ?_)
46   have := h false trivial
47   cases this
48
49 -- Z3.
50 /- The Ground can exceed the ladder: a sound theory can
51   ↳ leave a truth unproved (Gödel's
52   incompleteness region, L1m minus L2m), so the strata are
53   ↳ genuinely three, not one. -/
54 def incomplete : Theory := (Bool, fun b => b = true, fun _ =>
55   ↳ False, fun _ => True)
56
57 theorem ground_exceeds_ladder :
58   Sound incomplete ∧ incomplete.True_ false ∧ ¬
59   ↳ incomplete.Prov false :=
60   (fun _ _ => trivial, trivial, fun h => Bool.noConfusion h)
61
62 -- Z4.
63 /- Axioms as bits. Two independent axioms are two bits, not
64   ↳ one composite paradox: all four
65   combinations of their truth values are realized, so a
66   ↳ theory with several independent
67   axioms is a product of bits. (For ZFC: Choice is
68   ↳ independent of ZF, Gödel 1938 and Cohen
69   1963; this is cited, not proved here.) -/
70 theorem independent_axioms_are_separate_bits :
71   ∀ a b : Bool, ∃ w : Bool × Bool, w.1 = a ∧ w.2 = b :=
72   fun a b => ((a, b), rfl, rfl)
73
74 -- Z5.
75 /- THE VERDICT. RAM places every theory, ZFC included, as
76   ↳ rung ≤ ladder ≤ Ground, conditional
77   on the two named premises; soundness cannot be dropped;
78   ↳ the Ground genuinely exceeds the
79   ladder. Placement is structural subsumption: RAM does not
80   ↳ derive ZFC's axioms, and does
81   not replace ZFC's proofs. -/
82 theorem ram_subsumes_by_placement :
83   (∀ T : Theory, RungOnLadder T → Sound T →
84     ↳ (∀ s, T.Comp s → T.True_ s)) ∧
85   (RungOnLadder unsound ∧ ¬ Sound unsound) ∧
86   (Sound incomplete ∧ incomplete.True_ false ∧ ¬
87     ↳ incomplete.Prov false) :=
88   (fun T hR hS => (ram_places_theory T hR hS).2.2,
89     ↳ soundness_is_load_bearing, ground_exceeds_ladder)
90
91 end RAMZFC
92
93 #print axioms RAMZFC.ram_places_theory
94 #print axioms RAMZFC.soundness_is_load_bearing
95 #print axioms RAMZFC.ground_exceeds_ladder
96 #print axioms RAMZFC.independent_axioms_are_separate_bits
97 #print axioms RAMZFC.ram_subsumes_by_placement
```

```
1 'RAMZFC.ram_places_theory' does not depend on any axioms
2 'RAMZFC.soundness_is_load_bearing' does not depend on any
  ↳ axioms
3 'RAMZFC.ground_exceeds_ladder' does not depend on any axioms
4 'RAMZFC.independent_axioms_are_separate_bits' does not depend
  ↳ on any axioms
5 'RAMZFC.ram_subsumes_by_placement' does not depend on any
  ↳ axioms
```

RA_to_ZFC_Chain.lean

SHA-256 5714fbd5dc59b7ec79e251daca934819f5663e5554bad94d5537ed9f6c618a3e, 90 lines.

```
1 /-
2 RA_to_ZFC_Chain.lean · RA mapped to ZFC through the RA→RAM
  ↳ bridge, link by link.
3 Core Lean 4.19.0, no library, no sorry, no axiom declaration.
  ↳ Harvested 2026-09-24.
4
5 The chain RA → Bridge → RAM → ZFC. The RA→RAM bridge is the
  ↳ junction's discriminator
```

```

6 (Master Codex Ch. 10B, `Bridge.discriminator`): a property
  ↳ whose every world has it is KEYLESS
7 and crosses from the root; a property some world lacks is
  ↳ KEYED, one bit, supplied and never
8 derived. ZFC's location under RAM is RAM_ZFC_Placement.lean.
  ↳ Worlds are ways a theory's content
9 could be, which of its independent sentences hold.
10
11 Verdict. Upward, every act of using ZFC grounds RA. Downward,
  ↳ RA crosses the bridge as presence
12 into every world of ZFC and carries no keyed content: no
  ↳ independent sentence of ZFC, Choice
13 over ZF the cited example (Gödel 1938, Cohen 1963), is
  ↳ decided by RA.
14 -/
15 namespace RatoZFC
16
17 structure Substrate where
18   U : Type
19   ΔE : U → Int
20
21 def RA (S : Substrate) : Prop := ∀ x : S.U, 0 < S.ΔE x
22
23 structure SelfGrounding (R : Prop) where
24   Act : Type
25   anAct : Act
26   instances : Act → R
27
28 def unitSub : Substrate := (Unit, fun _ => 1)
29 theorem ra_unit : RA unitSub := fun _ => show (0 : Int) < 1
  ↳ by decide
30
31 /- A theory, abstractly: sentences and a provability
  ↳ predicate (the RAM ladder). -/
32 structure Theory where
33   Sent : Type
34   Prov : Sent → Prop
35
36 -- LINK 1 · ZFC → RA: every act of using a theory grounds RA.
  ↳ This link holds.
37 theorem using_grounds_root (T : Theory) (s : T.Sent) (d :
  ↳ T.Prov s) :
38   Nonempty (SelfGrounding (RA unitSub)) :=
39   ((PLift (T.Prov s), (d), fun _ => ra_unit))
40
41 -- LINK 2 · RA → Bridge: the bridge carries only KEYLESS
  ↳ properties, those no world denies.
42 /- A property of worlds is keyless when every world has it,
  ↳ keyed when some world lacks it. -/
43 def Keyless {W : Type} (P : W → Prop) : Prop := ∀ w, P w
44 def Keyed {W : Type} (P : W → Prop) : Prop := ∃ w, ¬ P w
45
46 /- RA crosses to every world whatever its content: RA is
  ↳ keyless over worlds. -/
47 theorem ra_is_keyless {W : Type} : Keyless (fun _ : W => RA
  ↳ unitSub) := fun _ => ra_unit
48
49 /- A keyless property is carried by RA to every world. -/
50 theorem keyless_crosses {W : Type} (P : W → Prop) (hP :
  ↳ Keyless P) :
51   ∀ w, RA unitSub → P w := fun w _ => hP w
52
53 -- LINK 3 · Bridge → RAM → ZFC: a keyed sentence, one the
  ↳ theory's worlds disagree on (for ZFC,
54 -- Choice over ZF: Gödel 1938, Cohen 1963, cited), is not
  ↳ carried. RA holds in both worlds.
55 /- Two worlds of ZF: in one Choice holds, in the other it
  ↳ fails. -/
56 def choiceHolds : Bool → Prop := fun w => w = true
57
58 theorem choice_is_keyed : Keyed choiceHolds := (false, fun h
  ↳ => Bool.noConfusion h)
59
60 /- THE BREAK. RA holds in both worlds, so RA cannot carry
  ↳ Choice into ZFC, nor its denial. -/
61 theorem ra_does_not_cross_keyed :
62   (∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
  ↳ choiceHolds false :=
63   (fun _ => ra_unit, rfl, fun h => Bool.noConfusion h)
64
65 /- In general: RA decides no keyed property of worlds. -/
66 theorem ra_decides_no_keyed {W : Type} (P : W → Prop) (hK :
  ↳ Keyed P) :

```

```

67   ¬ (∀ w, RA unitSub → P w) :=
68   fun h => let (w, hw) := hK; hw (h w ra_unit)
69
70 -- THE CHAIN, IN ONE THEOREM.
71 /- Upward, every act of using ZFC grounds RA. Downward, RA
  ↳ carries exactly the keyless and
72   none of the keyed: it reaches every world of ZFC as
  ↳ presence, and no independent sentence of
73   ZFC as content. -/
74 theorem chain_verdict :
75   (∀ (T : Theory) (s : T.Sent), T.Prov s → Nonempty
  ↳ (SelfGrounding (RA unitSub))) ∧
76   (∀ {W : Type} (P : W → Prop), Keyless P → ∀ w, RA unitSub
  ↳ → P w) ∧
77   (∀ {W : Type} (P : W → Prop), Keyed P → ¬ ∀ w, RA unitSub
  ↳ → P w) ∧
78   ((∀ w : Bool, RA unitSub) ∧ choiceHolds true ∧ ¬
  ↳ choiceHolds false) :=
79   (using_grounds_root, fun P hP => keyless_crosses P hP, fun
  ↳ P hK => ra_decides_no_keyed P hK,
80     ra_does_not_cross_keyed)
81
82 end RatoZFC
83
84 #print axioms RatoZFC.using_grounds_root
85 #print axioms RatoZFC.ra_is_keyless
86 #print axioms RatoZFC.keyless_crosses
87 #print axioms RatoZFC.choice_is_keyed
88 #print axioms RatoZFC.ra_does_not_cross_keyed
89 #print axioms RatoZFC.ra_decides_no_keyed
90 #print axioms RatoZFC.chain_verdict

```

```

1 'RatoZFC.using_grounds_root' does not depend on any axioms
2 'RatoZFC.ra_is_keyless' does not depend on any axioms
3 'RatoZFC.keyless_crosses' does not depend on any axioms
4 'RatoZFC.choice_is_keyed' does not depend on any axioms
5 'RatoZFC.ra_does_not_cross_keyed' does not depend on any
  ↳ axioms
6 'RatoZFC.ra_decides_no_keyed' does not depend on any axioms
7 'RatoZFC.chain_verdict' does not depend on any axioms

```

ZFC_RA_RoundTrip.lean

SHA-256 72a7571509a019e4b5af3f3e7324c698ffa66e64a69c6332f91be8080059310a, 64 lines.

```

1 /-
2 ZFC_RA_RoundTrip.lean · the round trip ZFC → RAM → Bridge →
  ↳ RA → Bridge → RAM → ZFC.
3 Core Lean 4.19.0, no library, no sorry, no axiom declaration.
  ↳ Drafted 2026-09-24.
4 A sentence of ZFC is read in its worlds (ways ZFC's content
  ↳ could be). Up the chain it is placed
5 on RAM's Ground; across the RA-RAM bridge it meets RA; the
  ↳ arrow back returns through the bridge,
6 which carries keyless content, to RAM and then to ZFC.
7 -/
8 namespace RoundTrip
9
10 structure Substrate where
11   U : Type
12   ΔE : U → Int
13
14 def RA (S : Substrate) : Prop := ∀ x : S.U, 0 < S.ΔE x
15 def unitSub : Substrate := (Unit, fun _ => 1)
16 theorem ra_unit : RA unitSub := fun _ => show (0 : Int) < 1
  ↳ by decide
17
18 /- Up: a sentence's content, read on RAM's Ground in each
  ↳ world. -/
19 def up {W : Type} (P : W → Prop) : W → Prop := P
20
21 /- Across and back: what the bridge returns to RAM from RA,
  ↳ in each world: RA together with the
22   content it received. -/
23 def viaRA {W : Type} (P : W → Prop) : W → Prop := fun w => RA
  ↳ unitSub ∧ up P w

```

```

24
25 /-- Down: from RAM back to the sentence of ZFC. -/
26 def down {W : Type} (Q : W → Prop) : W → Prop := Q
27
28 -- R1.
29 /-- THE ROUND TRIP IS THE IDENTITY ON CONTENT. In every
30   world, the sentence that returns to ZFC
31   is equivalent to the sentence that left. -/
32 theorem round_trip_identity {W : Type} (P : W → Prop) (w : W)
33   ↪ :
34   down (viaRA P) w ↔ P w :=
35   (fun h => h.2, fun h => (ra_unit, h))
36
37 -- R2.
38 /-- An absolute sentence, one with the same truth value in
39   every world (an arithmetic sentence
40   such as RH, whose value is fixed in every model with the
41   ↪ standard numbers): the round trip
42   returns that same value, true if it left true and false
43   ↪ if it left false. -/
44 theorem absolute_returns_unchanged {W : Type} (P : W → Prop)
45   ↪ (hAbs : ∀ w v, P w ↔ P v)
46   (w : W) : (down (viaRA P) w ↔ P w) ∧ (∀ v, down (viaRA P)
47   ↪ v ↔ P v) :=
48   (round_trip_identity P w, fun v => (round_trip_identity P
49   ↪ v).trans (hAbs v w))
50
51 -- R3.
52 /-- What the trip adds: presence. RA holds in every world on
53   ↪ return; it holds identically for a
54   sentence and for its negation, so it tells them apart
55   ↪ nowhere. -/
56 theorem trip_adds_presence_only {W : Type} (P : W → Prop) :
57   (∀ w : W, RA unitSub) ∧ (∀ w, (down (viaRA P) w ↔ P w) ∧
58   ↪ (down (viaRA (fun v => ¬ P v)) w ↔ ¬ P w)) :=
59   (fun _ => ra_unit, fun w => (round_trip_identity P w,
60   ↪ round_trip_identity (fun v => ¬ P v) w))
61
62 -- R4.
63 /-- THE VERDICT. Whatever ZFC sends up returns to ZFC
64   ↪ unchanged in content, with RA's presence
65   attached in every world; a sentence ZFC does not settle
66   ↪ leaves undecided and returns
67   undecided. -/
68 theorem round_trip_verdict :
69   (∀ {W : Type} (P : W → Prop) (w : W), down (viaRA P) w ↔
70   ↪ P w) ∧
71   (∀ {W : Type} (P : W → Prop), (∀ w v, P w ↔ P v) → ∀ w v,
72   ↪ down (viaRA P) v ↔ P w) :=
73   (fun P w => round_trip_identity P w, fun P h w v =>
74   ↪ (absolute_returns_unchanged P h w).2 v)
75
76 end RoundTrip
77
78 #print axioms RoundTrip.round_trip_identity
79 #print axioms RoundTrip.absolute_returns_unchanged
80 #print axioms RoundTrip.trip_adds_presence_only
81 #print axioms RoundTrip.round_trip_verdict

```

```

1 'RoundTrip.round_trip_identity' does not depend on any axioms
2 'RoundTrip.absolute_returns_unchanged' does not depend on any
  ↪ axioms
3 'RoundTrip.trip_adds_presence_only' does not depend on any
  ↪ axioms
4 'RoundTrip.round_trip_verdict' does not depend on any axioms

```

Massless_Arrow_Test.lean

SHA-256 7822a00ba1a2540891cdfb5888aac7d533d3e4f84c623c08b879c0751d17e5ee, 42 lines.

```

1 /-
2 Massless_Arrow_Test.lean · can a zero-content arrow from RA
  ↪ deliver a proposition?
3 Core Lean 4.19.0, no library, no sorry. Drafted 2026-09-24.
4 An arrow sends a proposition to what it delivers. It is
  ↪ massless (zero content) when it delivers

```

```

5 the same for every proposition, and sound when what it
  ↪ delivers for P implies P.
6 /-
7 namespace Massless
8
9 def Massless (f : Prop → Prop) : Prop := ∀ P Q, f P ↔ f Q
10 def Sound (f : Prop → Prop) : Prop := ∀ P, f P → P
11
12 -- M1.
13 /-- A sound massless arrow delivers nothing: if it delivered
14   ↪ P, it would deliver ¬P too, and
15   both would hold. So its delivery is empty for every
16   ↪ proposition. -/
17 theorem massless_sound_delivers_nothing (f : Prop → Prop) (hM :
18   ↪ Massless f) (hS : Sound f) :
19   ∀ P, ¬ f P :=
20   fun P h => hS (¬ P) ((hM P (¬ P)).mp h) (hS P h)
21
22 -- M2.
23 /-- The RA-shaped arrow: it delivers RA's presence, the same
24   ↪ for every proposition. -/
25 def raArrow (RA : Prop) : Prop → Prop := fun _ => RA
26
27 theorem raArrow_massless (RA : Prop) : Massless (raArrow RA)
28   ↪ := fun _ _ => Iff.rfl
29
30 /-- If RA holds, the RA-shaped arrow is NOT sound: it
31   ↪ delivers every proposition, true or false. -/
32 theorem raArrow_not_sound (RA : Prop) (hRA : RA) : ¬ Sound
33   ↪ (raArrow RA) :=
34   fun hS => hS False hRA
35
36 -- M3.
37 /-- THE VERDICT. A zero-content arrow either delivers nothing
38   ↪ (if it is sound) or delivers every
39   proposition and its negation alike (if it is not). Either
40   ↪ way it cannot deliver RH rather than
41   ¬RH. Content is what separates a proposition from its
42   ↪ negation, and a massless arrow has none. -/
43 theorem massless_cannot_separate (f : Prop → Prop) (hM :
44   ↪ Massless f) (RH : Prop) :
45   (f RH ↔ f (¬ RH)) :=
46   hM RH (¬ RH)
47
48 end Massless
49
50 #print axioms Massless.massless_sound_delivers_nothing
51 #print axioms Massless.raArrow_massless
52 #print axioms Massless.raArrow_not_sound
53 #print axioms Massless.massless_cannot_separate

```

```

1 'Massless.massless_sound_delivers_nothing' does not depend on
  ↪ any axioms
2 'Massless.raArrow_massless' does not depend on any axioms
3 'Massless.raArrow_not_sound' does not depend on any axioms
4 'Massless.massless_cannot_separate' does not depend on any
  ↪ axioms

```

Arrow_Translation.lean

SHA-256 f57924e397b9ec7e8b9f654547ff96c3f69d7aa7e41b4e32ee88afcb0a972ea9, 74 lines.

```

1 /-
2 Arrow_Translation.lean · Part Two translated into the purely
  ↪ formal domain: the Root Axiom
3 replaced by a bare computation arrow. Core Lean 4.19.0, no
  ↪ library, no sorry, no axiom declared.
4 The arrow is the identity computation on any type, `id : α →
  ↪ α`; its existence needs nothing.
5 Every place Part Two used the Root Axiom, it is replaced by
  ↪ the proposition that an arrow exists.
6 /-
7 namespace ArrowForm
8
9 /-- THE ARROW: a computation from a type to itself. It exists
  ↪ on every type, with no premise. -/

```

```

10 def Arrow (α : Type) : Prop := Nonempty (α → α)
11
12 theorem arrow_exists (α : Type) : Arrow α := {id}
13
14 /-- The arrow used throughout, on the unit type. -/
15 def A : Prop := Arrow Unit
16 theorem a_holds : A := arrow_exists Unit
17
18 -- T1 · the upward link. Every act of proof yields an arrow:
19   ↳ the proof itself, carried to itself.
20 theorem proof_yields_arrow (P : Prop) (h : P) : Arrow (PLift
21   ↳ P) :=
22   (fun _ => (h))
23
24 -- T2 · keyless crossing. The arrow holds in every world and
25   ↳ carries every keyless property.
26 def Keyless {W : Type} (P : W → Prop) : Prop := ∀ w, P w
27 def Keyed {W : Type} (P : W → Prop) : Prop := ∃ w, ¬ P w
28
29 theorem arrow_is_keyless {W : Type} : Keyless (fun _ : W =>
30   ↳ A) := fun _ => a_holds
31 theorem keyless_crosses {W : Type} (P : W → Prop) (hP :
32   ↳ Keyless P) : ∀ w, A → P w :=
33   fun w _ => hP w
34
35 -- T3 · the break. The arrow decides no keyed property.
36 theorem arrow_decides_no_keyed {W : Type} (P : W → Prop) (hK
37   ↳ : Keyed P) : ¬ ∀ w, A → P w :=
38   fun h => let (w, hw) := hK; hw (h w a_holds)
39
40 -- T4 · the round trip. Carried through the arrow and back,
41   ↳ every sentence returns unchanged.
42 def viaArrow {W : Type} (P : W → Prop) : W → Prop := fun w =>
43   ↳ A ∧ P w
44 theorem round_trip_identity {W : Type} (P : W → Prop) (w : W)
45   ↳ : viaArrow P w ↔ P w :=
46   (fun h => h.2, fun h => (a_holds, h))
47
48 -- T5 · the massless arrow. An arrow that delivers the same
49   ↳ for every proposition cannot
50   ↳ separate a proposition from its negation.
51 def Massless (f : Prop → Prop) : Prop := ∀ P Q, f P ↔ f Q
52 theorem arrowDelivery_massless : Massless (fun _ => A) := fun
53   ↳ _ => Iff.rfl
54 theorem massless_cannot_separate (f : Prop → Prop) (hM :
55   ↳ Massless f) (RH : Prop) :
56   f RH ↔ f (¬ RH) := hM RH (¬ RH)
57
58 -- T6 · the Real part, closed by a certificate, with no
59   ↳ premise at all.
60 structure Certificate {α : Type} (Z onL : α → Prop) (height :
61   ↳ α → Int) (T : Int) where
62   zeros : List α
63   complete : ∀ z, Z z → height z ≤ T → z ∈ zeros
64   onLine : ∀ z, z ∈ zeros → onL z
65
66 theorem arrow_real_part_of_certificate {α : Type} (Z onL : α
67   ↳ → Prop) (height : α → Int) (T : Int)
68   (c : Certificate Z onL height T) : ∀ z, Z z → height z ≤
69   ↳ T → onL z :=
70   fun z hz hle => c.onLine z (c.complete z hz hle)
71
72 -- T7 · THE TRANSLATION IS FAITHFUL. Any inhabited
73   ↳ proposition plays the root's part in every
74   ↳ result above: the road's theorems hold for an arbitrary
75   ↳ true premise R, so the root is idle in
76   ↳ the formal content, and the bare arrow is enough.
77 theorem any_true_premise_serves (R : Prop) (hR : R) {W :
78   ↳ Type} (P : W → Prop) :
79   ((∀ w, P w) → ∀ w, R → P w) ∧
80   ((∃ w, ¬ P w) → ¬ ∀ w, R → P w) ∧
81   (∀ w, (R ∧ P w) ↔ P w) :=
82   (fun hP w _ => hP w, fun (w, hw) h => hw (h w hR), fun _ =>
83     ↳ (fun h => h.2, fun h => (hR, h)))
84
85 end ArrowForm
86
87 #print axioms ArrowForm.arrow_exists
88 #print axioms ArrowForm.proof_yields_arrow
89 #print axioms ArrowForm.keyless_crosses
90 #print axioms ArrowForm.arrow_decides_no_keyed
91 #print axioms ArrowForm.round_trip_identity
92 #print axioms ArrowForm.massless_cannot_separate

```

```

73 #print axioms ArrowForm.arrow_real_part_of_certificate
74 #print axioms ArrowForm.any_true_premise_serves

```

```

1 'ArrowForm.arrow_exists' does not depend on any axioms
2 'ArrowForm.proof_yields_arrow' does not depend on any axioms
3 'ArrowForm.keyless_crosses' does not depend on any axioms
4 'ArrowForm.arrow_decides_no_keyed' does not depend on any
5   ↳ axioms
6 'ArrowForm.round_trip_identity' does not depend on any axioms
7 'ArrowForm.massless_cannot_separate' does not depend on any
8   ↳ axioms
9 'ArrowForm.arrow_real_part_of_certificate' does not depend on
10  ↳ any axioms
11 'ArrowForm.any_true_premise_serves' does not depend on any
12  ↳ axioms

```

RH_Real_Part_Computed.lean

SHA-256 a5c432c7eaf5f68ed1117368e71450cfad5a52b4d1cf331d7db9a98e21721645, 54 lines.

```

1 /-
2 RH_Real_Part_Computed.lean · the Real part of RH, closed by
3   ↳ computation and carried up the ladder.
4 Core Lean 4.19.0, no library, no sorry. Drafted 2026-09-24.
5 The Real part at height T (RH_Real_Unicorn_Split.lean) is a
6   ↳ bounded statement: finitely many
7   ↳ zeros lie below T. It is closed by a finite certificate, a
8   ↳ complete list of the zeros below T and
9   ↳ a check that each is on the line. That is the rung (L3m); the
10   ↳ rung is on the ladder (L2m), so a
11   ↳ certified Real part is provable; and the round trip returns
12   ↳ it unchanged.
13
14 -/
15 namespace RealPart
16
17 variable {α : Type} (Z onL : α → Prop) (height : α → Int)
18
19 def RealPartAt (T : Int) : Prop := ∀ z, Z z → height z ≤ T →
20   ↳ onL z
21
22 /-- A certificate for the Real part at T: a list of all zeros
23   ↳ up to T, and each is on the line. -/
24 structure Certificate (T : Int) where
25   zeros : List α
26   complete : ∀ z, Z z → height z ≤ T → z ∈ zeros
27   onLine : ∀ z, z ∈ zeros → onL z
28
29 -- C1.
30 /-- THE REAL PART, CLOSED. A certificate proves the Real part
31   ↳ at T completely. -/
32 theorem real_part_of_certificate (T : Int) (c : Certificate Z
33   ↳ onL height T) :
34   RealPartAt Z onL height T :=
35   fun z hz hle => c.onLine z (c.complete z hz hle)
36
37 -- C2.
38 /-- An executed instance: a zero set whose members up to T
39   ↳ are listed and checked by the kernel's
40   ↳ own computation (`decide`), the Real part following from
41   ↳ the certificate. -/
42 def zs : List (Int × Bool) := [(14, true), (21, true), (25,
43   ↳ true)]
44 def Zs (z : Int × Bool) : Prop := z ∈ zs
45 def onLs (z : Int × Bool) : Prop := z.2 = true
46
47 theorem real_part_executed : RealPartAt Zs onLs (fun z =>
48   ↳ z.1) 30 :=
49   real_part_of_certificate Zs onLs (fun z => z.1) 30
50   (zs, fun _ hz _ => hz, fun z hz => by
51     simp only [zs, List.mem_cons, List.mem_nil_iff,
52       ↳ or_false] at hz
53     rcases hz with h | h | h <=> subst h <=> rfl)
54
55 -- C3.
56 /-- The rung is on the ladder: a true Real part is a bounded
57   ↳ statement, and a theory complete for

```

```

42   true bounded statements proves it (for ZFC and PA,
    ↪ bounded completeness, a theorem of
43   arithmetic, carried as the named premise `bounded`). A
    ↪ certificate therefore yields a proof. -/
44 theorem certified_is_provable (Prov : Prop → Prop)
45   (bounded : ∀ T, RealPartAt Z onL height T → Prov
    ↪ (RealPartAt Z onL height T))
46   (T : Int) (c : Certificate Z onL height T) :
47   Prov (RealPartAt Z onL height T) :=
48   bounded T (real_part_of_certificate Z onL height T c)
49
50 end RealPart
51
52 #print axioms RealPart.real_part_of_certificate
53 #print axioms RealPart.real_part_executed
54 #print axioms RealPart.certified_is_provable

```

```

1 'RealPart.real_part_of_certificate' does not depend on any
  ↪ axioms
2 'RealPart.real_part_executed' depends on axioms: [propext]
3 'RealPart.certified_is_provable' does not depend on any
  ↪ axioms

```

RH_Real_Part_Turing_Check.py

```

1 # Real part executed to height T, three independent counts,
  ↪ in floating point:
2 #   (a) zeros ON the line, as sign changes of Hardy's Z(t) on
  ↪ a fine grid;
3 #   (b) all zeros in the strip with  $0 < \text{Im} < T$ ,  $N(T)$ , by
  ↪ mpmath's counting function nzeros(T),
4 #   computed independently of the grid; the earlier form
  ↪  $\theta(T)/\pi + 1 + \arg \zeta(1/2 + iT)/\pi$ 
5 #   with a principal-branch endpoint argument is not the
  ↪ counting function S(T) and is not used;
6 #   (c) the ordinates of the first zeros themselves, from
  ↪ mpmath.zetazero, counted below T.
7 # Agreement of (a), (b), (c) at T means every zero up to T is
  ↪ on the line, in 30-digit floating
8 # point without interval arithmetic: testimony, not a
  ↪ certificate. The certificate of record for
9 # zeta to  $3 \times 10^{12}$  is Platt and Trudgian (2021), cited in
  ↪ the paper, not this script.
10 import mpmath as mp
11 mp.mp.dps = 30
12 T = mp.mpf(100)
13 def Z(t): return mp.siegelz(t)
14 grid = [mp.mpf(k)/20 for k in range(1, int(T*20)+1)]
15 on_line, prev = 0, Z(grid[0])
16 for t in grid[1:]:
17     z = Z(t)
18     if z == 0 or mp.sign(z) != mp.sign(prev): on_line += 1
19     prev = z
20 N = int(mp.nzeros(T))
21 ords = []
22 k = 1
23 while True:
24     g = mp.zetazero(k).imag
25     if g > T: break
26     ords.append(g); k += 1
27 print("T =", T)
28 print("(a) zeros ON the line, sign changes of Z on a 1/20
  ↪ grid :", on_line)
29 print("(b) N(T), all zeros in the strip, mpmath nzeros
  ↪ :", N)
30 print("(c) ordinates of zeros below T, enumerated
  ↪ :", len(ords),
31       " first", mp.nstr(ords[0], 8), " last",
  ↪ mp.nstr(ords[-1], 8), " next",
  ↪ mp.nstr(mp.zetazero(k).imag, 8))
32 ok = (on_line == N == len(ords))
33 print("Real part at T =", int(T), ":", "HOLDS (three counts
  ↪ equal)" if ok else "count mismatch")

```

```

1 T = 100.0
2 (a) zeros ON the line, sign changes of Z on a 1/20 grid : 29

```

```

3 (b) N(T), all zeros in the strip, mpmath nzeros      : 29
4 (c) ordinates of zeros below T, enumerated          : 29
  ↪ first 14.134725 last 98.831194 next 101.31785
5 Real part at T = 100 : HOLDS (three counts equal)

```

APPENDIX I: RH_UNICORN_BLOCK.LEAN AND ITS AUDIT

SHA-256 222bd4c985b99df22435e5f3b04a66171304377177f042495caa5be2e5a4921b, Lean 4.19.0, core only, 142 lines; nine dependency sets, every one reported as depending on no axiom.

```

1 /-
2 RH_Unicorn_Block.lean · the Unicorn part of the Riemann
  ↪ Hypothesis, closed to derivation, its
3 aperture one bit wide, its supply side silent. Core Lean
  ↪ 4.19.0, no library, no sorry, no user-declared axiom.
  ↪ 2026-09-25.
4
5 Part III divided the hypothesis at a height T into a Real part
  ↪ and a Unicorn part and proved
6 the Real part by certificate. Parts I and II proved the wall,
  ↪ the crossing, the bound, and the
7 cure theorem, and did not say what they say of the Unicorn
  ↪ part. This file says it: closed to
8 derivation, the aperture one bit wide, the register silent
  ↪ beyond it.  $\Delta M = 0$ .
9 -/
10 namespace RHUnicorn
11
12 variable {α β : Type}
13
14 /- The line property, the Real part at T, the Unicorn part at
  ↪ T, as in Part III. -/
15 def RH (Z onL : α → Prop) : Prop := ∀ z, Z z → onL z
16 def RealPartAt (Z onL : α → Prop) (height : α → Nat) (T : Nat)
  ↪ : Prop :=
17   ∀ z, Z z → height z ≤ T → onL z
18 def UnicornAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) :
  ↪ Prop :=
19   ∀ z, Z z → height z > T → onL z
20
21 /- THE FORMAL BLOCK OF THE UNICORN PART. Let τ act on the
  ↪ zeros, let p be a record even at a
22 seat z above the height, and let d decide the line
  ↪ property, odd at z. Then no reading g of
23 the record agrees with d even on the region above the
  ↪ height alone. The Unicorn part is
24 closed to every even register; the closure is a theorem and
  ↪ does not expire. -/
25 theorem unicorn_block (height : α → Nat) (T : Nat) (τ : α → α)
  ↪ (p : α → β) (d : α → Bool)
26   (z : α) (habove : height z > T) (habove' : height (τ z) >
  ↪ T)
27   (hp : p (τ z) = p z) (hd : d (τ z) = !d z) :
28   ¬ ∃ g : β → Bool, ∀ w, height w > T → g (p w) = d w := by
29   intro (g, hg)
30   have h1 := hg z habove
31   have h2 := hg (τ z) habove'
32   rw [hp, hd, h1] at h2
33   revert h2
34   cases d z <|> intro h2 <|> exact Bool.noConfusion h2
35
36 /- THE APERTURE, ONE BIT WIDE. At a seat where d is odd, one
  ↪ odd bit s at the seat would decide d
37 on the seat and its partner through a calibration that
  ↪ exists and is unique. The width of
38 the aperture, a theorem; it says nothing of whether
  ↪ anything passes through it. -/
39 theorem aperture_one_bit_wide (τ : α → α) (s d : α → Bool) (z :
  ↪ α)
40   (hs : s (τ z) = !s z) (hd : d (τ z) = !d z) :
41   ∃ c : Bool, (d z = xor (s z) c ∧ d (τ z) = xor (s (τ z)) c)
  ↪ ∧
42   ∀ c' : Bool, (d z = xor (s z) c' ∧ d (τ z) = xor (s (τ
  ↪ z)) c') → c' = c :=
43   (xor (d z) (s z),
44     (by cases s z <|> cases d z <|> rfl,

```

```

45   by rw [hs, hd]; cases s z <;> cases d z <;> rfl),
46   by
47     intro c' hc
48     obtain (h1, -) := hc
49     generalize hsz : s z = sv
50     generalize hdz : d z = dv
51     rw [hsz, hdz] at h1
52     cases sv <;> cases dv <;> cases c' <;> first | rfl |
      ↪ exact absurd h1 (by decide))
53
54 /-- EXISTENCE SUPPLIES NOTHING. Conditioning the Unicorn part
   ↪ on an inhabited premise, the root
55 axiom for one, leaves it exactly where it was; and no class
   ↪ of frames on which the premise
56 is to decide it decides more than already held on the
   ↪ class. Theorems E and I of Part I,
57 read at the Unicorn part. -/
58 theorem unicorn_rule_exact (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :  $\alpha \rightarrow$ 
   ↪ Nat) (T : Nat) (A : Prop) (ha : A) :
59 (A  $\rightarrow$  UnicornAt Z onL height T)  $\leftrightarrow$  UnicornAt Z onL height T
   ↪ :=
60 (fun h => h ha, fun hu _ => hu)
61
62 theorem unicorn_no_cure {Frame : Type} (A : Prop) (ha : A) (C U
   ↪ : Frame  $\rightarrow$  Prop) :
63 ( $\forall X, C X \rightarrow A \rightarrow U X$ )  $\leftrightarrow$  ( $\forall X, C X \rightarrow U X$ ) :=
64 (fun h X hc => h X hc ha, fun h X hc _ => h X hc)
65
66 /-- NO BYPASS. Given the Real part, the fused hypothesis is
   ↪ exactly the Unicorn part, so a proof
67 of the hypothesis contains a proof of the Unicorn part and
   ↪ the block is not avoided by
68 aiming at the whole. -/
69 theorem unicorn_no_bypass (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :  $\alpha \rightarrow$  Nat)
   ↪ (T : Nat)
70 (hr : RealPartAt Z onL height T) : RH Z onL  $\leftrightarrow$  UnicornAt Z
   ↪ onL height T := by
71 constructor
72 · intro h z hz _; exact h z hz
73 · intro hu z hz
74 cases Nat.lt_or_ge T (height z) with
75 | inl hgt => exact hu z hz hgt
76 | inr hle => exact hr z hz hle
77
78 /-- THE SUPPLY SIDE IS SILENCE, AT THE REGISTER. Over one even
   ↪ record, both orientations of d at
79 the seat are equally refused to every reading above the
   ↪ height: the register cannot say
80 which way the seat lies, so it cannot say what a supply
   ↪ would bring, nor that one exists,
81 nor that none does. -/
82 theorem supply_side_silent (height :  $\alpha \rightarrow$  Nat) (T : Nat) ( $\tau : \alpha$ 
   ↪  $\rightarrow \alpha$ ) ( $\rho : \alpha \rightarrow \beta$ ) (d :  $\alpha \rightarrow \text{Bool}$ )
83 (z :  $\alpha$ ) (habove : height z > T) (habove' : height ( $\tau$  z) >
   ↪ T)
84 (hp :  $\rho$  ( $\tau$  z) =  $\rho$  z) (hd : d ( $\tau$  z) = !d z) :
85 ( $\neg \exists g : \beta \rightarrow \text{Bool}, \forall w, \text{height } w > T \rightarrow g$  ( $\rho$  w) = d w)  $\wedge$ 
86 ( $\neg \exists g : \beta \rightarrow \text{Bool}, \forall w, \text{height } w > T \rightarrow g$  ( $\rho$  w) = !d w)  $\wedge$ 
87 (!d z)  $\neq$  d z :=
88 (unicorn_block height T  $\tau$   $\rho$  d z habove habove' hp hd,
89 unicorn_block height T  $\tau$   $\rho$  (fun w => !d w) z habove habove'
   ↪ hp (by
90   show (!d ( $\tau$  z)) = !(d z)
91   rw [hd]),
92 by cases d z <;> decide)
93
94 /-- The division at the height, on natural heights: the
   ↪ hypothesis is exactly its Real part and
95 its Unicorn part. -/
96 theorem rh_iff_real_and_unicorn (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :  $\alpha$ 
   ↪  $\rightarrow$  Nat) (T : Nat) :
97 RH Z onL  $\leftrightarrow$  RealPartAt Z onL height T  $\wedge$  UnicornAt Z onL
   ↪ height T := by
98 constructor
99 · intro h; exact (fun z hz _ => h z hz, fun z hz _ => h z hz)
100 · intro (hr, hu) z hz
101 cases Nat.lt_or_ge T (height z) with
102 | inl hgt => exact hu z hz hgt
103 | inr hle => exact hr z hz hle
104
105 /-- A certificate at the height: a complete list of the zeros
   ↪ up to T, each checked on the line. -/
106 structure Certificate (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :  $\alpha \rightarrow$  Nat) (T
   ↪ : Nat) where

```

```

107 items      : List  $\alpha$ 
108 complete   :  $\forall z, Z z \rightarrow \text{height } z \leq T \rightarrow z \in \text{items}$ 
109 checked    :  $\forall z, z \in \text{items} \rightarrow \text{onL } z$ 
110
111 theorem real_part_of_certificate (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :  $\alpha$ 
   ↪  $\rightarrow$  Nat) (T : Nat)
112 (c : Certificate Z onL height T) : RealPartAt Z onL height
   ↪ T :=
113 fun z hz hle => c.checked z (c.complete z hz hle)
114
115 /-- THE THREE BITS AS ONE TERM · THE REGISTER'S PROOF OF THE
   ↪ HYPOTHESIS, COMPLETE. Under a
116 certificate at T and the seat data above T, the hypothesis
   ↪ divides exactly (bit one), its
117 Real part at T holds (bit two), every reading of the even
   ↪ register is refused above T (bit
118 three, refused), and one supplied bit calibrates the seat
   ↪ uniquely (the door). The hypothesis
119 is not thereby proved; nothing derivable about it is left
   ↪ underived. -/
120 theorem rh_register_proof_complete (Z onL :  $\alpha \rightarrow \text{Prop}$ ) (height :
   ↪  $\alpha \rightarrow$  Nat) (T : Nat)
121 (c : Certificate Z onL height T) ( $\tau : \alpha \rightarrow \alpha$ ) ( $\rho : \alpha \rightarrow \beta$ ) (d
   ↪ :  $\alpha \rightarrow \text{Bool}$ ) (z :  $\alpha$ )
122 (habove : height z > T) (habove' : height ( $\tau$  z) > T) (hp :
   ↪  $\rho$  ( $\tau$  z) =  $\rho$  z)
123 (hd : d ( $\tau$  z) = !d z) (hs : s ( $\tau$  z) = !s z) :
124 (RH Z onL  $\leftrightarrow$  RealPartAt Z onL height T  $\wedge$  UnicornAt Z onL
   ↪ height T)  $\wedge$ 
125 RealPartAt Z onL height T  $\wedge$ 
126 ( $\neg \exists g : \beta \rightarrow \text{Bool}, \forall w, \text{height } w > T \rightarrow g$  ( $\rho$  w) = d w)  $\wedge$ 
127  $\exists k : \text{Bool}, (d z = \text{xor } (s z) k \wedge d (\tau z) = \text{xor } (s (\tau z)) k)$ 
   ↪  $\wedge$ 
128  $\forall k' : \text{Bool}, (d z = \text{xor } (s z) k' \wedge d (\tau z) = \text{xor } (s (\tau$ 
   ↪ z)) k')  $\rightarrow k' = k$  :=
129 (rh_iff_real_and_unicorn Z onL height T,
   ↪ real_part_of_certificate Z onL height T c,
130 unicorn_block height T  $\tau$   $\rho$  d z habove habove' hp hd,
   ↪ aperture_one_bit_wide  $\tau$  s d z hs hd)
131
132 end RHUnicorn
133
134 #print axioms RHUnicorn.unicorn_block
135 #print axioms RHUnicorn.aperture_one_bit_wide
136 #print axioms RHUnicorn.unicorn_rule_exact
137 #print axioms RHUnicorn.unicorn_no_cure
138 #print axioms RHUnicorn.unicorn_no_bypass
139 #print axioms RHUnicorn.supply_side_silent
140 #print axioms RHUnicorn.rh_iff_real_and_unicorn
141 #print axioms RHUnicorn.real_part_of_certificate
142 #print axioms RHUnicorn.rh_register_proof_complete

```

Kernel audit, verbatim: lean RH_Unicorn_ Block.lean, exit 0.

```

1 'RHUnicorn.unicorn_block' does not depend on any axioms
2 'RHUnicorn.aperture_one_bit_wide' does not depend on any axioms
3 'RHUnicorn.unicorn_rule_exact' does not depend on any axioms
4 'RHUnicorn.unicorn_no_cure' does not depend on any axioms
5 'RHUnicorn.unicorn_no_bypass' does not depend on any axioms
6 'RHUnicorn.supply_side_silent' does not depend on any axioms
7 'RHUnicorn.rh_iff_real_and_unicorn' does not depend on any
   ↪ axioms
8 'RHUnicorn.real_part_of_certificate' does not depend on any
   ↪ axioms
9 'RHUnicorn.rh_register_proof_complete' does not depend on any
   ↪ axioms

```
